CME 295 – Transformers and Large Language Models

Lecture 1: Course Introduction, NLP Foundations, and the Transformer

Afshine Amidi, Shervine Amidi

Course Introduction and Logistics

Afshine and Shervine (twin brothers) both studied at CentraleSupélec in France before diverging – Afshine to MIT ('17), Shervine to Stanford's ICME Master's program ('19) – and both later worked at Uber, Google, and now Netflix, specializing in NLP and LLMs since 2020. This course grew out of a workshop they had run yearly since 2021; following the surge of interest after ChatGPT's 2022 release, it became a formal Stanford course (CME 295) last spring, and this is its second offering.

Goals and Audience

(1) Understand how transformers work and how they relate to LLMs.

(2) Learn how LLMs are trained and used in various applications.

The course is aimed at anyone interested in LLMs – whether as a career goal, for a personal project, or out of general "AI literacy"/curiosity. Prerequisites: machine learning basics and linear algebra (e.g. matrix multiplication); a still-developing level of competency is workable.

Format, Grading, and Materials

The class meets Fridays 3:30–5:20pm in Thornton 110, for two units, gradable as letter or credit/no-credit. Sessions are recorded. Grading is based on two exams – no homework: a **midterm** (50% of the grade, October 24) and a **final exam** (50%, week of December 8, exact date TBD at the time of this lecture).

Course materials: the class website (`cme295.stanford.edu`) hosts the syllabus, slides, and recordings; the companion textbook is the *Super Study Guide: Transformers & Large Language Models*; and a condensed **VIP cheatsheet** (translated into 11 languages at the time of this lecture, with more added on request) is available at `https://github.com/afshinea/stanford-cme-295-transformers-large-language-m`. Announcements go through Canvas; class discussions happen on Ed; general questions can go to `cme295-aut2526-staff@lists.stanford.edu` or directly to the instructors.

Q: Are there coding portions on the exams?
A: No – exams focus purely on the concepts covered in class; following the lectures and slides should be sufficient preparation.

Q: What should a waitlisted student do?
A: Reach out to the instructors; historically enrollment settles as some students drop the class, and

the waitlist was small (around six people) at the time, so it was expected to resolve.

Q: Where can the slides be found?
A: On the course website, with a link also posted on Canvas.

Q: How are the two exams weighted?
A: 50% midterm, 50% final – no homework contributes to the grade.

Q: Does the final exam only cover the second half of the course?
A: The exam hadn't been written yet at the time, but the plan was for the final to focus on the second half of the syllabus.

A Roadmap of Abbreviations

The field is dense with abbreviations – the instructors flag this upfront with a deliberately overwhelming word cloud (BERT, T5, GPT, LLaMA, LSTM, GRU, GloVe, BPE, CoT, ToT, SC, RAG, SFT, PEFT, FLAN, RL, RM, RLHF, PPO, DPO, NER, PoS, MLM, NSP, MT, QA, NLG, MNLI, WNLI, C4, SQuAD, GLUE, MRPC, F1, PPL, ROUGE, BLEU, METEOR, LaaJ, WER, ...) before reassuring students not to worry: by the end of the course, all of these should have a clear place in a mental map like the one below.

Category	**Examples**
Transformer-based models	BERT, T5, GPT, LLaMA
Misc. architectures & techniques	LSTM, GRU, GloVe, BPE, CoT, ToT, SC, RAG
Training strategies	SFT, PEFT, FLAN, RL, RM, RLHF, PPO, DPO
Tasks	NER, PoS, MLM, NSP, MT, QA, NLG
Datasets	MNLI, WNLI, C4, SQuAD, GLUE, MRPC
Metrics	F1, PPL, ROUGE, BLEU, METEOR, LaaJ, WER

Course Outline

Today's lecture (and the running outline shown throughout): NLP overview → Tokenization → Word representation → RNNs → Self-attention mechanism → Transformer architecture → End-to-end example.

1 NLP Task Categories

NLP (Natural Language Processing) concerns computing over text. Tasks broadly fall into three buckets.

1. Classification

Text in, single prediction out (schematically: `Input text` → `Model` → one of several classes). Examples: sentiment analysis, intent detection ("I want to create an alarm for tomorrow" → create-alarm intent), language detection, topic modeling.

Illustrative Task: Sentiment Extraction

E.g. "This teddy bear is SO CUTE!" → positive.

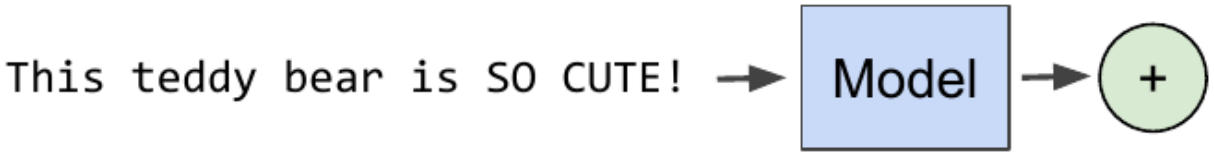


Figure 1: Sentiment extraction, schematically.

Common datasets: **Amazon reviews**, **IMDb critiques**, and **Twitter/X** posts. Evaluated via **accuracy** (% of observations correctly predicted), **precision** (% of predicted-positive that were correct), **recall** (% of actually-positive that were correctly identified), and **F1 score** (a function of precision and recall). These finer metrics matter especially under **class imbalance** (e.g. 99% positive, 1% negative), where a model that always predicts the majority class scores misleadingly high accuracy.

2. "Multi"-Classification (Token-Level)

Text in, *multiple* predictions out – one per token or span (schematically: `Input text` → `Model` → `Input text` with per-token labels attached). The flagship example is **Named Entity Recognition (NER)**: e.g. labeling "teddy bear" with the tag ENTITY. Related, more linguistics-oriented tasks include part-of-speech tagging and dependency/constituency parsing (less trendy today, but historically well-studied).

Illustrative Task: NER

Common dataset: annotated Reuters newswire (**CoNLL-2003**, **CoNLL++**). Evaluated via accuracy/precision/recall/F1, but computed **at the token level, per entity type** rather than at the whole-sentence level.

3. Generation

Text in, variable-length text out (schematically: `Input text` → `Model` → `Output text`). Examples: machine translation, question answering (the assistant-style interaction familiar from ChatGPT/Gemini), summarization, text/code/poem generation.

Illustrative Task: Machine Translation

E.g. English "A cute teddy bear is reading" → French "Un ours en peluche mignon lit." Common datasets: **WMT'14 English-French** and **WMT'14 English-German** (Workshop on Machine Translation).

Evaluation is inherently harder here, since a given meaning can be expressed in many valid ways. Two historical rule-based reference metrics (formulas available at `http://www.statmt.org/book/slides/08-evaluation.pdf`):

- **BLEU** (BiLingual Evaluation Understudy) – quality of translated text, conceptually similar to "precision."
- **ROUGE** – quality of generated text, conceptually similar to "recall."

Both require reference (labeled) translations, which are expensive to produce. **Perplexity**, by contrast, requires no reference text: it looks only at the model's own output probabilities, quantifying how "surprised" the model is to see certain words occur together.

A Brief History

Year	Development
1980s	Recurrent Neural Networks (RNNs) – *theoretical foundations*
1997	Long Short-Term Memory (LSTM) – *theoretical foundations*
2013	Word2vec
2017	Transformers
2020s	Large Language Models

The 1980s/1997 entries reflect *theoretical* foundations; the 2013 onward entries were increasingly driven by more data, growing compute power, and fast iteration on ideas – ingredients that weren't yet available when RNNs and LSTMs were first proposed.

2 Tokenization

Models operate on numbers, not raw text, so text must first be divided into discrete units called **tokens** – a process called **tokenization**. Several granularities are possible (illustrated on "A cute teddy bear is reading."):

- **Arbitrary**: e.g. splitting as `A | cute | teddy bear | is | reading | .` – illustrative, not a real scheme.
- **Word-level**: `A | cute | teddy | bear | is | reading | .`
- **Subword-level** (e.g. **WordPiece**, **BPE** – Byte-Pair Encoding): `A | cute | ted | ##dy | bear | is | read | ##ing | .` – splitting words into common roots/affixes, with continuation pieces marked (e.g. `##`).
- **Character-level**: splitting into individual characters (e.g. `A_c u t e_t e d d y_b e a r_i s_r e a d i n g .`).

Method	Pros	Cons
Word-level	Simple; interpretable	Risk of OOV; does not leverage knowledge of word roots
Subword-level (e.g. WordPiece, BPE)	Leverages common prefixes/suffixes; learned from the data	Risk of OOV, though less than word-level
Character-level	Small chance of OOV; robust to casing and misspellings	Makes computations slower; embeddings not interpretable

A key related concept: **Out-Of-Vocabulary (OOV)** tokens – text seen at inference time that wasn't in the training vocabulary must be mapped to a reserved `[UNKNOWN]` token. Word-level tokenization has the highest OOV risk; subword reduces it; character-level effectively eliminates it.

Q: How does vocabulary size and the word/subword/character choice vary by task and language?
A: It depends on the task – a single-language task typically uses subword tokenization for its favorable trade-off (leverages word roots while limiting OOV risk). Vocabulary size is typically on the order of tens of thousands for one language; modern multilingual, code-capable models often use vocabularies in the hundreds of thousands. Non-Latin scripts (e.g. Chinese) follow the same subword principles, just applied to the target language's own character system.

---

3 Representing Tokens

One-Hot Encoding and Its Limitation

The naive representation assigns each token in a vocabulary a **one-hot vector** (e.g. for vocabulary {book, soft, teddy bear}: soft = $[1, 0, 0]$, teddy bear = $[0, 1, 0]$, book = $[0, 0, 1]$). To compare token similarity, a common measure is **cosine similarity** – the angle between two vectors: vectors pointing the same direction are similar, orthogonal vectors are independent, opposite-pointing vectors are dissimilar.

The problem: one-hot vectors for any two distinct tokens are always exactly orthogonal, giving zero similarity even between clearly related tokens (e.g. "teddy bear" and "soft," which intuitively should be similar) just as much as between unrelated ones (e.g. "teddy bear" and "book"). We instead want a **learned embedding** where similar tokens have high similarity and unrelated tokens are near zero.

Q: Why does cosine similarity ignore vector norm (relying only on the dot product/angle)?
A: Cosine similarity is simply one common way people have chosen to quantify similarity – it is a measure, not *the* perfect measure (dot product alone is another option some use). Whether the norm carries meaningful information depends on how the vectors were trained; in practice, what's typically of interest is the angle (direction) between vectors, not their magnitude.

Q: So how do you actually obtain these (non-one-hot) embeddings?
A: That's the topic of the next section – Word2vec.

---

4 Word2vec: Learning Embeddings from a Proxy Task

Word2vec (Mikolov et al., *"Efficient Estimation of Word Representations in Vector Space,"* 2013) is a neural network trained on a proxy task over billions of words of text, from which an **embedding layer** is learned. It demonstrated that embeddings learned this way could capture strikingly interpretable relationships, e.g. king $-$ man $+$ woman $\approx$ queen, or Paris is to France as Berlin is to Germany. Two training variants (proxy tasks):

- **Continuous Bag of Words (CBOW)**: predict a target word from its surrounding context words.
- **Skip-gram**: predict the surrounding context words from a target word.

Both are **proxy tasks**: the actual goal isn't the prediction itself, but the meaningful embeddings that emerge as a side effect of a model becoming good at that prediction – a model that can predict context

well must have implicitly captured something about how words relate.

Architecture and Worked Example

A simple feedforward network: a one-hot input vector (size V, the vocabulary size) is projected to a small hidden vector (size $d \ll V$; real-world d is typically in the hundreds, e.g. 768 is a common choice), then projected back up to a vocabulary-sized output, followed by softmax to get next-word probabilities. Training proceeds by feeding each word's one-hot vector, comparing the predicted next-word distribution to the true next word (cross-entropy loss), and backpropagating to update weights – repeated across the training corpus. The learned hidden-layer projection weights, applied to any token's one-hot vector, *are* that token's learned embedding.

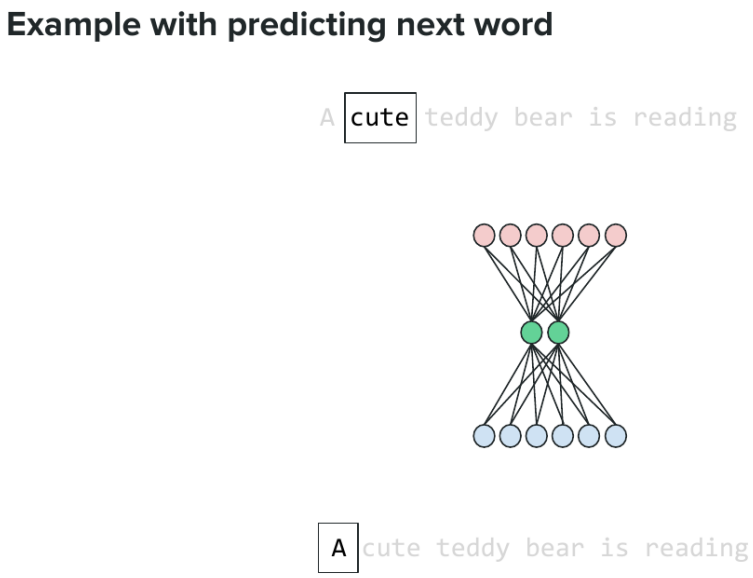


Figure 2: Word2vec's proxy-task network: one-hot input → small hidden layer (the learned embedding) → vocabulary-sized output.

Q: In the toy example, what does V represent, and why only six possible words?
A: V is the vocabulary size – six was just a simplified toy example; real vocabularies are far larger. This also illustrates why tokenization granularity matters: a word-level vocabulary must account for every word variation, inflating V. At inference time, any token not seen during training is mapped to a reserved out-of-vocabulary/unknown token representation – a problem word-level tokenizers face more often than subword, and subword more often than character-level.

Q: How do you know when to stop training (i.e. when has the model "learned enough")?
A: Track the loss as a function of **epochs** (passes over the training set) and stop once it converges – though the ideal stopping point can also depend on the downstream task.

Q: How does the model know when to stop generating text?
A: A reserved **end-of-sequence (EOS)** token; generation stops once it's produced.

Q: What determines the size of the hidden (embedding) layer?
A: A trade-off: richer/more complex downstream tasks benefit from larger embeddings (more expressive), but larger embeddings cost more to compute (higher latency, more expensive inference). Typical orders of magnitude are hundreds to low thousands of dimensions (e.g. 768) – often set empirically, informed by what has worked well in prior work, rather than derived from first principles.

Q: How can this approach distinguish the same word used with different meanings in different contexts (e.g. "bank" as in riverbank vs. a bank you rob)?
A: It can't, yet – Word2vec-style embeddings are *not* context-aware; a given token gets the same embedding regardless of surrounding context. This is precisely the limitation that motivates the methods covered next (recurrence, and eventually self-attention).

---

5 Sequential Models: RNNs and LSTMs

Representing a whole sentence by naively averaging its word embeddings loses word order and much of the meaning, and (per the question above) word-level embeddings aren't context-aware regardless. This motivates models that explicitly capture sequence structure.

Recurrent Neural Networks (RNNs)

First introduced in the 1980s: a class of neural networks whose connections form a temporal sequence. Rather than processing tokens independently, an RNN processes tokens **one at a time**, maintaining a running **hidden state** that summarizes the sequence so far, updated at each timestep by combining the previous hidden state with the current token. Because processing is now sequential, word **order** is naturally captured, and the same word can in principle contribute differently depending on what came before it.

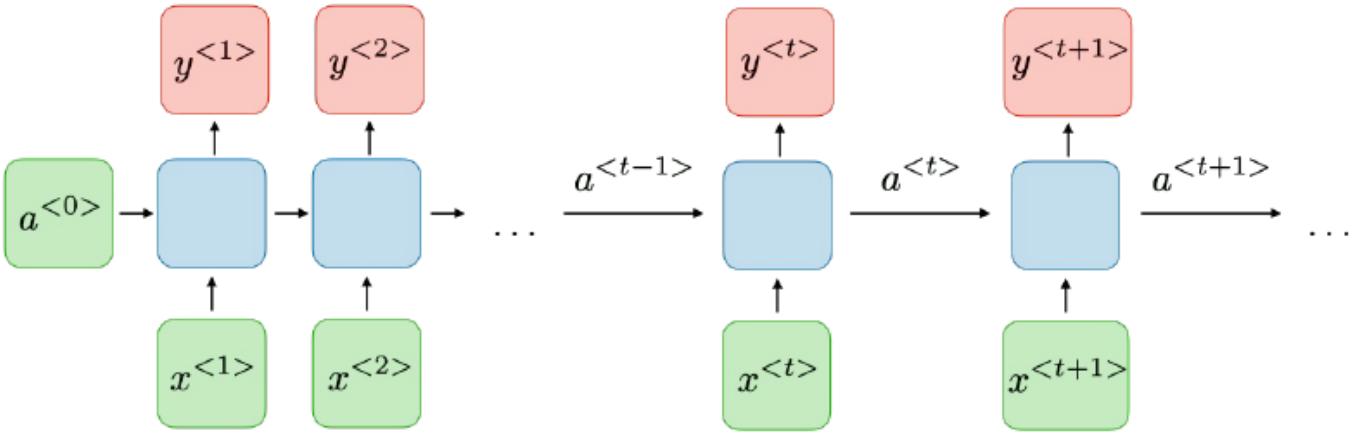


Figure 3: General form of an RNN: hidden state $a^{\langle t \rangle}$ carried forward, combined with each new input $x^{\langle t \rangle}$ to produce output $y^{\langle t \rangle}$.

Use Across Task Types

Mapping back to the three NLP task categories:

- **Classification** (e.g. sentiment → "opinion"): use the final hidden state, projected onto the label space.
- **Multi-classification** (e.g. tagging → "text"): use the hidden state at each token of interest, projected onto the relevant label space.
- **Generation** (e.g. translation → "source"): process the full source sequence into a final context vector, then use it to drive decoding of the output sequence.

Long Short-Term Memory (LSTM)

Introduced in *"Long Short-Term Memory"* (Hochreiter & Schmidhuber, 1997): an extension of RNNs using a more structured approach to the cell's hidden state, adding a separately tracked **cell state** (c, alongside the usual activation a) intended to better preserve information judged important to remember over longer spans.

Summary So Far (Non-Exhaustive)

Method	Pros	Cons
Word2vec (e.g. CBOW, skip-gram)	Very simple, yet powerful; intuitive embeddings	Word order does not count; embeddings not context-aware
RNNs (e.g. traditional RNN, LSTM)	Word order matters; state-of-the-art results (at the time)	Vanishing gradient problem; slow computations

Limitations: Vanishing Gradients

RNNs struggle with **long-range dependencies**: information from early in a sequence tends to fade from the hidden state by the time later tokens are processed. This is the **vanishing gradient** problem: predicting a late token depends (via backpropagation through time) on a chain of computations reaching back through every earlier timestep, and this dependency mathematically works out to a *product* of many terms – if these terms are consistently below 1 in magnitude, the product shrinks toward zero, making early-timestep gradients vanishingly small (an update signal too weak to learn from); values consistently above 1 cause the opposite problem (exploding gradients). RNNs are also computationally slow to train on long sequences, since computing a given hidden state requires having already computed all prior hidden states in order.

6 From Recurrence to Attention

Attention was introduced in Bahdanau et al., *"Neural Machine Translation by Jointly Learning to Align and Translate"* (2014), motivated by exactly this problem: translation tasks exposed a real issue with long-range dependencies, since Seq2Seq (encoder-decoder RNN) models were unable to "remember" what the input sentence was saying by the time they needed to decode later output words. Attention instead creates a **direct link** between what's currently being predicted and any relevant earlier position, rather than relying on a single running hidden state to carry everything forward. Example: while decoding "Un ours en peluche mignon lit" from "A cute teddy bear is reading," being able to directly "look back" at the relevant source word at each decoding step is more effective than compressing the entire source sentence into one hidden state.

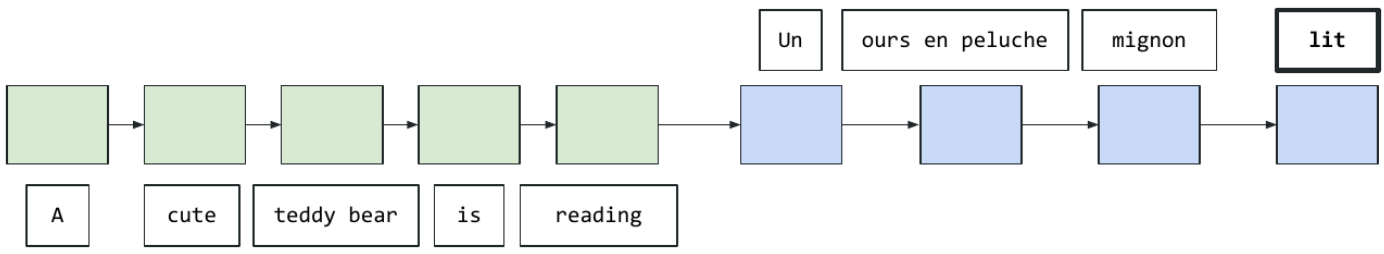


Figure 4: Seq2seq encoder (green) / decoder (blue) without attention: the whole input must be compressed into the final hidden state before decoding begins.

Self-Attention and the Transformer

The **transformer** (Vaswani et al., *"Attention Is All You Need,"* 2017) took this further: rather than processing tokens sequentially at all, it lets *every* token attend directly to *every other* token in the sequence at once – **self-attention** – and this alone achieved state-of-the-art results on machine translation tasks. In "a cute teddy bear is reading," computing the representation of "teddy bear" involves directly attending to all other tokens in the sequence simultaneously, rather than passing through a chain of intermediate hidden states.

Q (returning to the earlier "bank"/context question): Does this mean the same word gets different representations depending on context?
A: Yes – with self-attention, a token's representation is computed as a direct function of the *specific* other tokens present in that sequence, so e.g. "bank" in a riverbank context vs. a financial context would end up with different representations.

Query, Key, and Value

To express one token in terms of others, self-attention uses three learned projections: **query** (Q), **key** (K), and **value** (V). To compute a representation of a token (as a query), it is compared against the **keys** of all tokens (including itself) to determine similarity/relevance, and the corresponding **values** are combined in a weighted average based on that similarity.

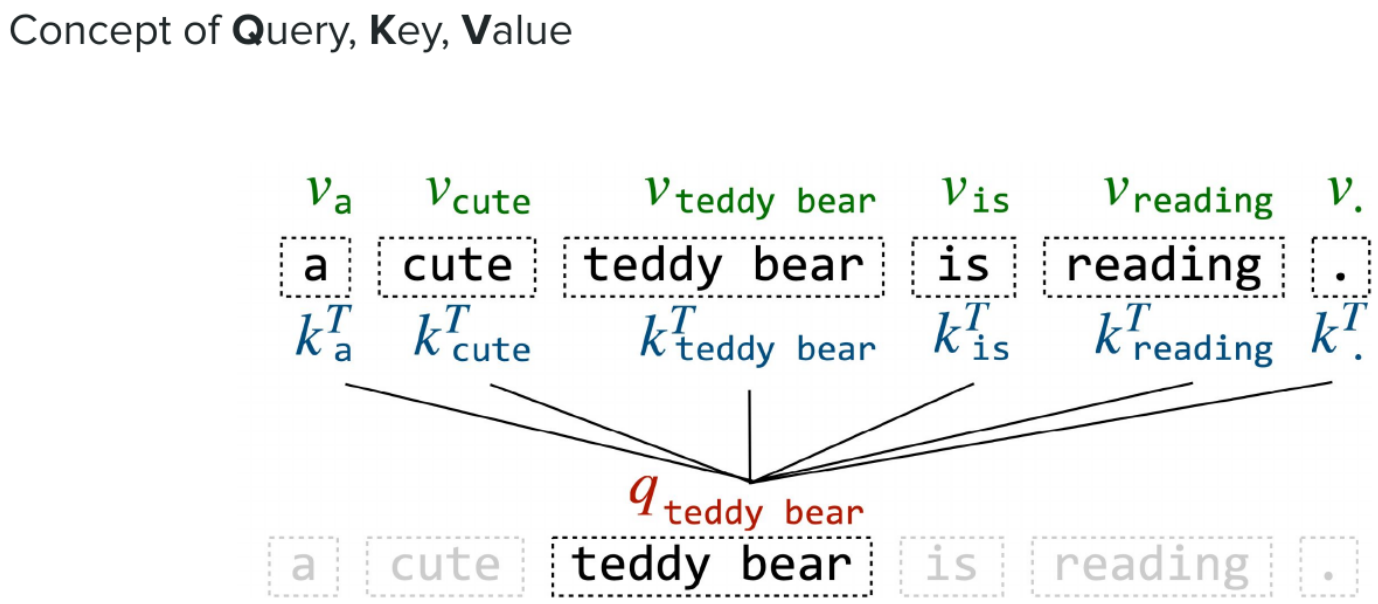


Figure 5: The query for "teddy bear" compared against every token's key, to weight every token's value.

This generalizes cleanly to matrix form – convenient, since GPUs are highly optimized for large matrix multiplications:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Q: What exactly are "key" and "value," and how are they obtained?
A: Both are **learned** projections of the input (via projection matrices trained by the model), not fixed

or hand-designed. Interpretively: the key is used to determine how similar/relevant a token is to a given query, while the value is the actual content contributed once that relevance is established – the attention weights (from comparing query and key) determine how much each token's value contributes to the weighted average.

---

7 The Transformer Architecture

At a high level, the transformer has two components: an **encoder** (left) and a **decoder** (right), originally applied to translation – the encoder processes the source-language input; the decoder produces the target-language output. Both are built from three recurring components: the **attention layer** (multi-head attention – self-attention within the encoder or decoder, plus an encoder-decoder attention layer), the **feedforward neural network (FFNN)**, and **positional encoding (PE)**.

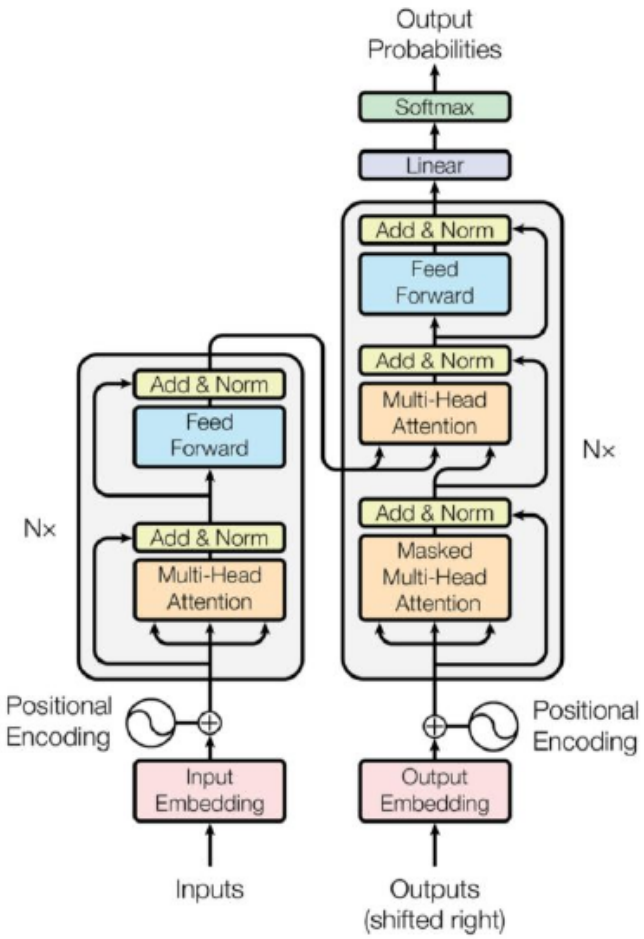


Figure 6: The full transformer architecture (Vaswani et al., 2017): encoder (left) and decoder (right).

Input Embedding

Text is tokenized, and each token gets a **learned embedding**. Two governing hyperparameters recur throughout the architecture: V (vocabulary size) and d_{model} (embedding dimension).

Positional Encoding: "... with a trick!"

Since direct attention links discard the sequential notion of order that RNNs had implicitly, the transformer adds a position encoding to each input, either **learned** or **hardcoded**, with the goal of letting the model understand *relative* input position: the dot product between two position encodings should be largest (close to its max) when the two positions coincide, and decay toward its minimum as their relative distance grows – varying smoothly across embedding dimensions (see Kazemnejad, *"Transformer Architecture: The Positional Encoding,"* 2019, for detailed visualizations of this behavior).

Encoder

Each of N stacked encoder layers contains: **encoder-encoder attention** (self-attention), an FFNN, and a normalization layer. Governing hyperparameters: N (number of stacked layers), h (number of attention heads), d_{FF}, d_{key}, d_{value} (sub-layer dimensions), and d_{model} (embedding dimension).

Decoder Input: "Output, Shifted Right"

The decoder's own input consists of learned embeddings for the *output* tokens generated so far; in practice, decoding starts from a `[BOS]` (beginning-of-sequence) token. Governing hyperparameters: V and d_{model}, as with the encoder input.

Decoder

Each of N stacked decoder layers contains: **decoder-decoder attention** (masked self-attention, attending only to previously decoded tokens), **encoder-decoder attention** (cross-attention: decoder queries attend to encoder-derived keys/values), an FFNN, and a normalization layer. Same governing hyperparameters as the encoder: N, h, d_{FF}, d_{key}, d_{value}, d_{model}.

Q (posed to the class): In cross-attention (encoder-decoder attention), does the arrow coming from the decoder represent a query, a key, or a value?
A: Query. The decoder is asking, "given what I'm trying to generate next, which parts of the input matter?" – so the decoder supplies the query, while the encoder supplies the keys and values.

Output

A final **linear projection** maps the decoder's representation onto a vector of size V, followed by softmax. This is naturally framed as a **classification problem where each class is a word** in the vocabulary – a nice full-circle callback to the classification task category from the start of this lecture. Governing hyperparameters: V and d_{model}.

Computational Trick: Multi-Head Attention

Rather than a single self-attention computation, the transformer runs **multiple self-attention layers in parallel** ("heads"), each with its own learned $Q/K/V$ projections. This lets the model capture

different attention patterns simultaneously – comparable to how multiple filters in a convolutional layer let a computer vision model capture different visual features in parallel.

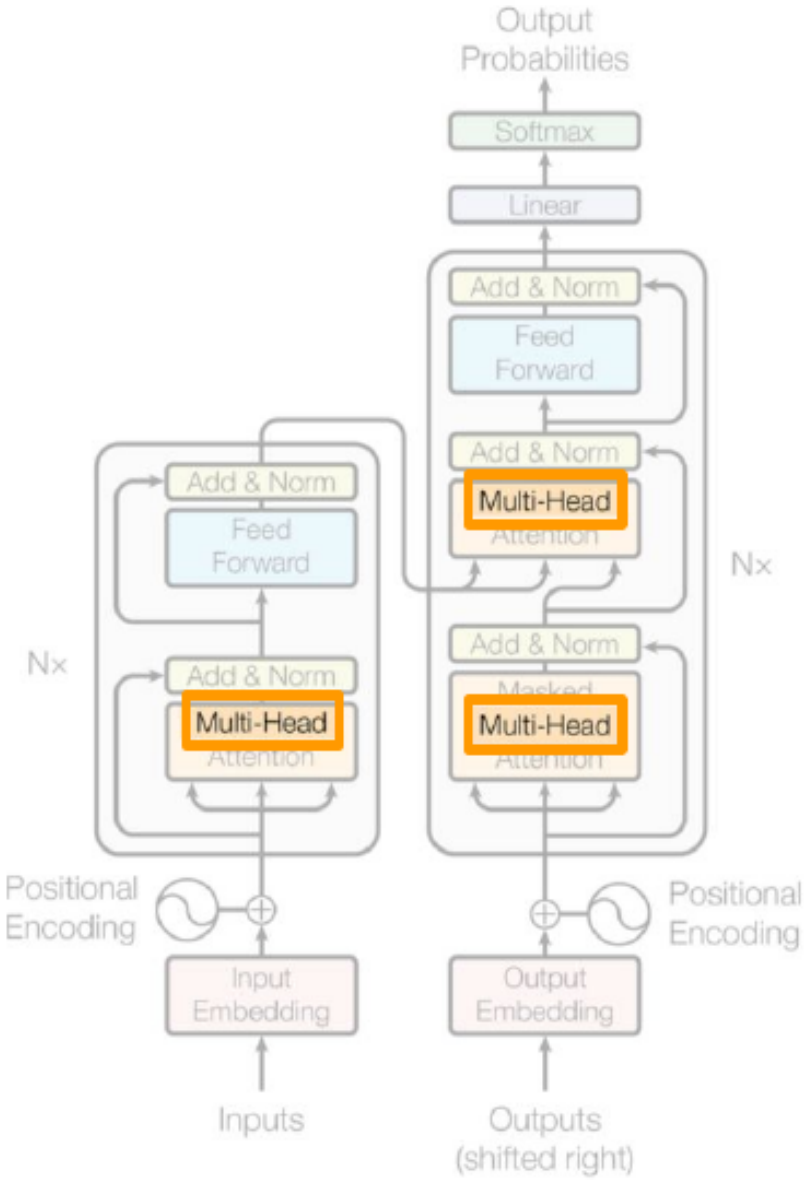


Figure 7: Multi-head attention: multiple self-attention layers run in parallel, analogous to multiple filters in a CNN.

Q: What does a "head" mean in multi-head attention?
A: Each head has its own set of learned $Q/K/V$ projection matrices, giving the model an additional degree of freedom to learn different kinds of relationships between tokens in parallel (denoted h, the number of heads).

Q: Could all the heads end up learning the same thing, making the multi-head setup redundant?
A: Nothing in the architecture explicitly *prevents* that, but there's no incentive for it either: the network's only objective is to build representations useful for the training task (e.g. predicting the next word), and gradient descent has no reason to waste multiple heads duplicating the same computation. In

practice, heads tend to converge on different, complementary representations that are then concatenated and projected back into a useful combined representation.

Computational Trick: Label Smoothing

Motivated by a 2015 computer vision paper's finding that model overconfidence is undesirable, **label smoothing** introduces a small amount of noise into the true labels used for training. Concretely, rather than training the model to predict the target word with probability 1 and everything else 0 – reasonable given that natural language often admits multiple valid continuations (e.g. "what a great day/lecture/book/..") – the target distribution is softened to $1-\epsilon$ on the correct word and $\epsilon/(|V|-1)$ spread across all other vocabulary entries. This is a general technique (not specific to transformers) that helps prevent overfitting, and was found to improve both accuracy and BLEU score in the original transformer paper.

Q: Is this related to the explore/exploit trade-off?
A: An interesting parallel, though the direct mechanism here is simpler: softmax alone doesn't let you specify anything other than a hard, single correct-answer target; label smoothing changes what the *target label itself* looks like (from a one-hot vector to a slightly "smoothed" distribution), which is a different lever than the sampling/exploration behavior at inference time.

---

8 Worked End-to-End Example

Using "A cute teddy bear is reading." throughout, with `[BOS]`/`[EOS]` tokens added to mark sequence boundaries.

Input Representation

For each token, combine a learned **token embedding** with a **position embedding** (added element-wise) to obtain a **position-aware embedding**. Stacking these across the sequence gives a position-aware embeddings **matrix** of size $d_{\text{model}} \times n$ (embedding dimension by sequence length, including the `[BOS]`/`[EOS]` tokens).

Self-Attention, Step by Step

The position-aware embeddings matrix is projected via three learned matrices W_q, W_k, W_v into query, key, and value matrices Q, K, V (each row of Q is one token's query; similarly for K, V). Computing $QK^\top$ yields, per row, the dot product of that token's query against every token's key; applying softmax turns each row into a probability distribution over which other tokens are most relevant to that query. Dividing by $\sqrt{d_k}$ before the softmax counteracts the tendency of dot products to grow in magnitude as the key/query dimension grows (query and key dimensions must match, since they're dot-producted together). Multiplying this attention-weight matrix by V produces, for each token, a **weighted average of values**, with weights being a function of query-key similarity – its updated, context-aware representation.

Q: Why scale by $\sqrt{d_k}$ specifically?
A: As the dimensionality of queries/keys grows, their dot products tend to grow in magnitude as well; dividing by $\sqrt{d_k}$ normalizes this before the softmax, keeping the resulting distribution well-behaved rather than overly peaked purely as an artifact of dimensionality.

This computation is run in parallel across h heads (each with its own W_q, W_k, W_v); the resulting h output matrices are concatenated column-wise and passed through a final projection matrix W_o back to the original embedding dimension – giving the network a dimension-invariant way to combine multiple heads' representations.

Q: Since nothing forces the different heads to learn different projections, could they all converge to the same result, making the concatenation pointless?
A: In principle nothing prevents it, but it's simply not what tends to happen: the model's only incentive is to build representations that help predict the next word, and gradient descent has no reason to make redundant heads – in practice, heads converge to different, useful representations.

Feedforward Layer

Following self-attention, a feedforward network (FFN) further transforms each token's representation. Unlike the small hidden layer used in the earlier Word2vec proxy-task example, the FFN's hidden layer here (d_{FF}) is *larger* than the input/output dimension (d_{model}) – giving the model more degrees of freedom to build richer, more complex features, rather than compressing to a small representational bottleneck.

Stacking and Decoding

The encoder consists of N stacked encoder layers (self-attention + FFN, repeated), producing final **context-aware encoded embeddings** of the input. These are then fed as the keys/values for encoder-decoder (cross-)attention in each of N stacked decoder layers. Decoding starts from the `[BOS]` token: at the first step, decoder-decoder (masked) self-attention has only the `[BOS]` token to attend to (attending only to itself); encoder-decoder attention then lets this token's representation incorporate information from the full encoded input; an FFN further refines it; and this repeats across all N decoder layers, ending in a linear projection and softmax over the vocabulary (e.g. yielding a distribution like $[0.001, 0.0003, \ldots, 0.4, \ldots, 0.002]$) to predict the next token – here, "Un." That predicted token is then fed back into the decoder (appended to the sequence so far) to predict the following token, and so on: "Un" → "ours en peluche" → "mignon" → "lit" → `[EOS]`.

Q: When does the decoding process stop?
A: When the model generates the end-of-sequence (`[EOS]`) token.

This autoregressive loop – predict next token, append it, repeat until `[EOS]` – yields the final translation "Un ours en peluche mignon lit." and is exactly how the original transformer paper performed machine translation; it remains the core generation mechanism used throughout the rest of this course.

Summary

This first lecture covered course logistics (including a roadmap for navigating the field's dense abbreviation set), then built up the conceptual foundation for everything to follow: the three broad categories of NLP tasks (classification, token-level multi-classification, generation) and how each is evaluated, with concrete datasets (Amazon reviews/IMDb/Twitter, CoNLL-2003, WMT'14) and metrics (accuracy/precision/recall/F1, BLEU/ROUGE, perplexity); **tokenization** trade-offs (word/subword/character-level, and the OOV problem); the move from one-hot encoding to **learned embeddings** via **Word2vec**'s proxy-task framing (CBOW/skip-gram, Mikolov et al. 2013); **RNNs** and **LSTMs** (Hochreiter & Schmidhuber, 1997) as early sequence-aware models, and their long-range-dependency (vanishing gradient) limitations; and finally **attention** (Bahdanau et al. 2014) and **self-attention** as the direct solution to that limitation, culminating in the full **transformer** architecture (Vaswani et al. 2017) – encoder, decoder, multi-head attention, masked (decoder-decoder) self-attention, encoder-decoder (cross-)attention, position encodings, and label smoothing – walked through end-to-end on a worked translation example.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 2: Transformer Refinements and Encoder-Only Models (BERT)

Afshine Amidi, Shervine Amidi

Recap of Lecture 1

Lecture 1 introduced **self-attention**: each token attends to every other token in the sequence via **queries**, **keys**, and **values**. A query is compared against all keys to determine similarity, and the corresponding values are aggregated accordingly:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

This formula reduces to large, hardware-friendly matrix multiplications. We also introduced the **transformer** architecture (encoder + decoder, originally for machine translation: encoder processes the source language, decoder produces the target language), and the **multi-head attention** layer, where each head learns its own way of projecting inputs into queries, keys, and values. **Attention maps** (as shown in the original "Attention Is All You Need" paper) visualize which tokens a given query attends to most strongly, and different heads can specialize in different relationships (e.g. different heads highlighting different referents of a pronoun like "its").

Today's lecture covers, in two parts: (1) components of the transformer that have evolved since 2017 (position embeddings, normalization, and attention efficiency/sharing), and (2) the taxonomy of transformer-derived models, with a deep dive into encoder-only models (BERT and its variants).

1 Part 1: Evolving Components of the Transformer

Position Embeddings

Because tokens interact directly with one another (unlike RNNs, which process tokens sequentially), the transformer loses positional information and must reinject it explicitly.

Learned Absolute Position Embeddings

The original approach: give each position (1, 2, 3, ...) its own learned embedding, added to the token embedding. Limitations: (1) the learned embeddings can inherit biases from the training set's positional patterns, and (2) they cannot generalize beyond the maximum sequence length seen during training.

Q: Are position embeddings learned or static?
A: Both – the original authors tried both approaches. If learned, you have a placeholder learnable

embedding for each position (e.g. positions 1 through 512), and these weights are learned through ordinary gradient descent alongside the rest of the model. The catch: you become very dependent on what positional patterns happened to appear in the training set, and you can only learn embeddings up to the maximum position seen during training – at inference time, a position beyond that has no learned value to fall back on.

Fixed Sinusoidal Position Embeddings

An alternative used in the original paper: a deterministic (non-learned) formula based on sine and cosine. For position m and dimension index i, define

$$\omega_i = 10000^{-2i/d_{\text{model}}}$$

and use $\sin(\omega_i m)$ and $\cos(\omega_i m)$ across the embedding dimensions. The motivation is that we want closer tokens to appear more similar than distant ones. Using the trigonometric identity

$$\cos(a-b) = \cos a \cos b + \sin a \sin b$$

one can show that the dot product between the position embeddings at positions m and n reduces to a sum of $\cos(\omega_i(m-n))$ terms – i.e. a function purely of the *relative* distance $m-n$. Since $\cos(0) = 1$ is the maximum value of cosine (near the origin), this dot product is maximized when $m = n$, matching the intuition that a position is most similar to itself, and decreasing (with oscillation, due to periodicity) as the relative distance grows. Empirically, low dimensions (i small, ω_i large) oscillate at high frequency, while high dimensions (i large, ω_i small) oscillate at low frequency.

This method performs comparably to learned embeddings but has the advantage of extending naturally to sequence lengths beyond those seen at training time.

Injecting Position Directly into Attention

Since what actually matters is that similarity, as computed inside the attention layer, reflects proximity, later work moved position information from the *input* embedding directly into the *attention* computation:

- **T5**: learns a relative position **bias** term (per attention head), added inside the softmax (i.e. to the raw query-key score), based on bucketed relative distances $m-n$ (Raffel et al., *"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"*).
- **ALiBi** (Attention with Linear Biases): uses a fixed, deterministic, *unbounded* linear bias as a function of the relative distance $m-n$, rather than a learned one (Press et al. 2021, *"Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation"*).

Q: Does adding a bias term inside the softmax cause a problem, given that the output has to sum to 1?
A: No – you can add whatever you want inside the softmax, since the softmax normalizes the result regardless. The bias just shifts the relative scores before normalization (e.g. making distant tokens' scores more negative), which is exactly the point.

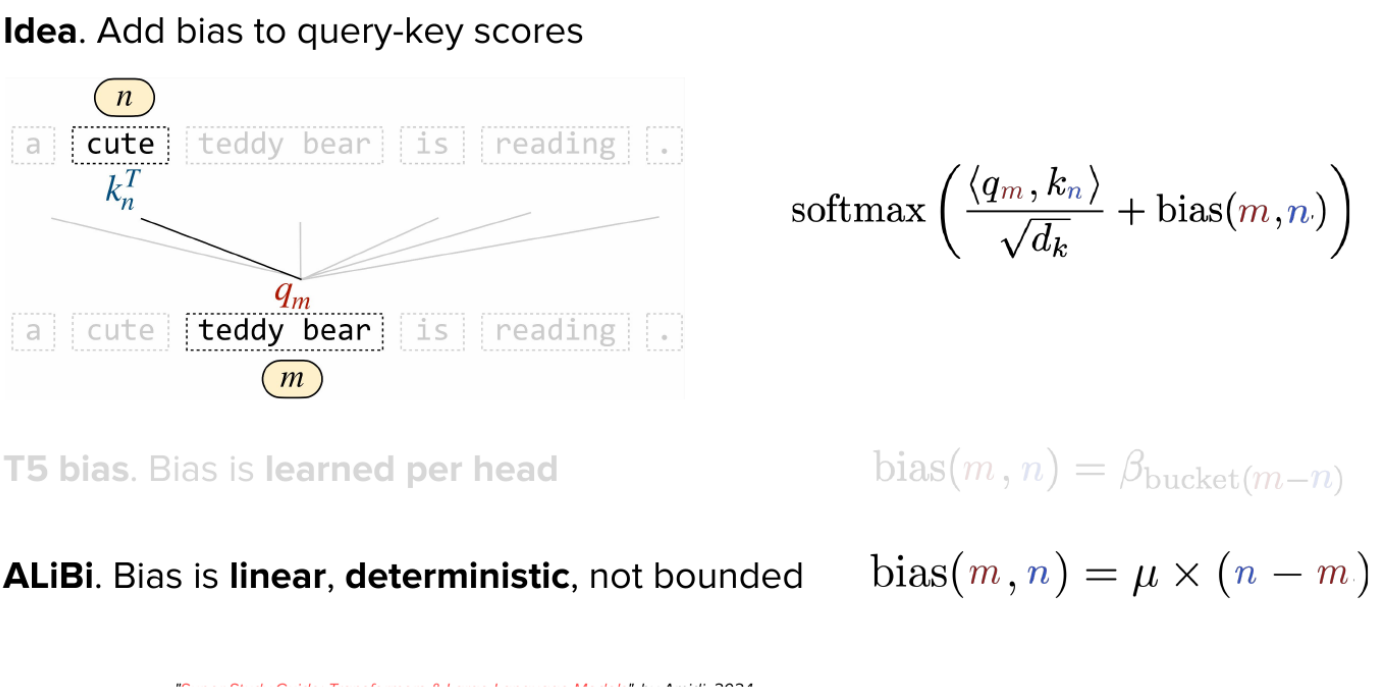


Figure 1: Bias added to query-key scores before the softmax; T5 learns this bias per head, ALiBi uses a fixed linear formula.

RoPE – Rotary Position Embeddings

Most modern models use **RoPE** (Su et al. 2021, *"RoFormer: Enhanced Transformer with Rotary Position Embeddings"*) as the default choice. The idea: rotate the query vector by an angle proportional to its position m, and the key vector by an angle proportional to its position n. In 2D, a rotation by angle θ is applied via the rotation matrix

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

For a vector $v = R(\cos\phi, \sin\phi)$ (norm R, angle ϕ from the x-axis), one can show

$$R(\theta)\, v = R\big(\cos(\theta+\phi),\ \sin(\theta+\phi)\big)$$

i.e. multiplying by $R(\theta)$ rotates the vector by θ. Applying this to queries and keys, the dot product between a query rotated by angle (proportional to) m and a key rotated by angle (proportional to) n depends only on $R(\theta \cdot (n-m))$ – again purely a function of relative distance. The angle parameter θ is fixed per dimension pair, analogous to the ω_i from the sinusoidal scheme, and is extended to d-dimensional space by rotating every block of dimension 2 separately. A useful theoretical property is that the *upper bound* on the attention weight between a query and key exhibits **long-term decay** as their relative distance grows (with some oscillation, but a decaying envelope) – proven in the RoFormer paper.

Q: Concretely, how do you rotate a vector by some angle?
A: Via matrix multiplication with a **rotation matrix**: a 2×2 matrix with $\cos\theta$, $-\sin\theta$, $\sin\theta$, $\cos\theta$ as entries. Multiplying this matrix by a 2D vector $(R\cos\phi, R\sin\phi)$ yields $(R\cos(\theta+\phi), R\sin(\theta+\phi))$ – i.e. the same vector rotated by θ, which can be verified directly by expanding the matrix product and applying the angle-addition identities.

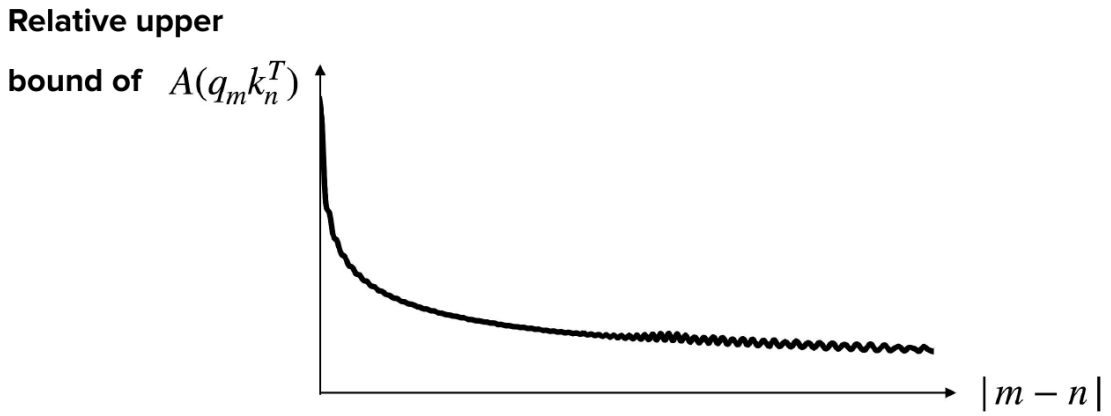


Figure 2: RoPE's relative upper bound on attention weight decays as the relative distance $|m-n|$ grows.

Q: Why rotate the query and key at all?
A: Because rotating the query by an angle proportional to its position and the key by an angle proportional to its position means their dot product ends up depending only on the *difference* between the two angles – i.e. purely on relative position – which is exactly the property we want out of a position-encoding scheme.

Q: What exactly is θ?
A: It's fixed (not learned), and is a function of the dimension index i (ranging between 1 and $d/2$) and the total dimension d – playing the same role as the ω_i term from the sinusoidal scheme.

Q: Does θ need to match the dimension of the underlying (e.g. latent) space?
A: Since this is a matrix product, the dimensions on both sides do need to match up – yes.

Q: How is the long-term decay upper bound actually derived?
A: It's shown mathematically in an appendix of the RoFormer paper, where the authors prove the attention-weight upper bound decays as relative distance grows; deriving it in full is beyond this course's scope, but the paper's appendix walks through it explicitly for anyone who wants to see the proof.

Q: Do these position-encoding techniques apply to self-attention and cross-attention alike?
A: They *can* be applied to any attention layer, but in practice this mostly comes up for masked self-attention in the decoder, since modern LLMs are decoder-only (no encoder, hence no cross-attention) – so that's the setting where these techniques are actually used today.

Layer Normalization

Inside each encoder/decoder block, an "Add & Norm" step takes the sublayer's input and output, sums them, and normalizes. For a vector x, **LayerNorm** computes:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$$

where μ, σ are the mean and standard deviation of x's components, and γ (rescaling) and β (shift) are learned (Ba et al. 2016, *"Layer Normalization"*). This improves training stability and convergence

speed by keeping activations within a consistent range (related to the notion of **internal covariate shift**) – as opposed to **batch normalization**, which normalizes across the batch dimension instead and introduces train/inference discrepancies, making LayerNorm generally preferred for transformers.

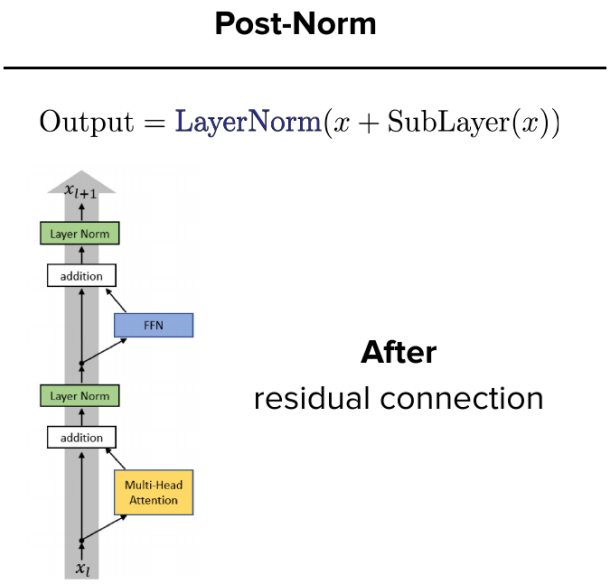


Figure 3: Post-Norm: LayerNorm applied *after* the residual addition, as in the original transformer.

Q: What's the intuition behind normalizing in the first place?
A: Across layers, a model's activations can sometimes take on extreme values in one part of the vector and not another. When activations vary too much, the model tends to struggle to learn good weights in each layer. The idea is to bring activation components back into some consistent range – a phenomenon related to what's called **internal covariate shift**.

Q: What's the difference between this (LayerNorm) and batch normalization?
A: Batch normalization normalizes across the *batch* dimension instead – for a given component, it normalizes with respect to that same component across all the other vectors in the batch. For transformer-based models, people typically use LayerNorm rather than BatchNorm, likely because it empirically works better, and because BatchNorm's dependence on the batch can introduce differences between training and inference behavior.

- **Post-norm vs. pre-norm**: the original transformer normalized *after* the sublayer, i.e. after the residual connection (post-norm: $\text{Norm}(x + \text{Sublayer}(x))$). Modern models typically use **pre-norm**: normalize *before* the residual connection, i.e. before feeding into the sublayer ($x + \text{Sublayer}(\text{Norm}(x))$) (Xiong et al. 2020, *"On Layer Normalization in the Transformer Architecture"*).
- **RMSNorm**: a simplification of LayerNorm that normalizes by the root-mean-square of x's components (no mean-subtraction) and learns only γ (no β) (Zhang & Sennrich 2019, *"Root Mean Square Layer Normalization"*):

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\text{RMS}(x)}, \qquad \text{RMS}(x) = \sqrt{\frac{1}{d}\sum_j x_j^2}$$

  This achieves comparable convergence properties with fewer learned parameters, and is standard in most modern LLMs (typically paired with pre-norm).

Attention Efficiency and Sharing

Full self-attention has $O(n^2)$ complexity in sequence length n, which becomes expensive as n grows.

Sliding Window (Local) Attention

Introduced in papers like **Longformer** (Beltagy et al. 2020, *"Longformer: The Long-Document Transformer"*): restrict each token's attention to a local neighborhood window rather than the full sequence, reducing complexity.

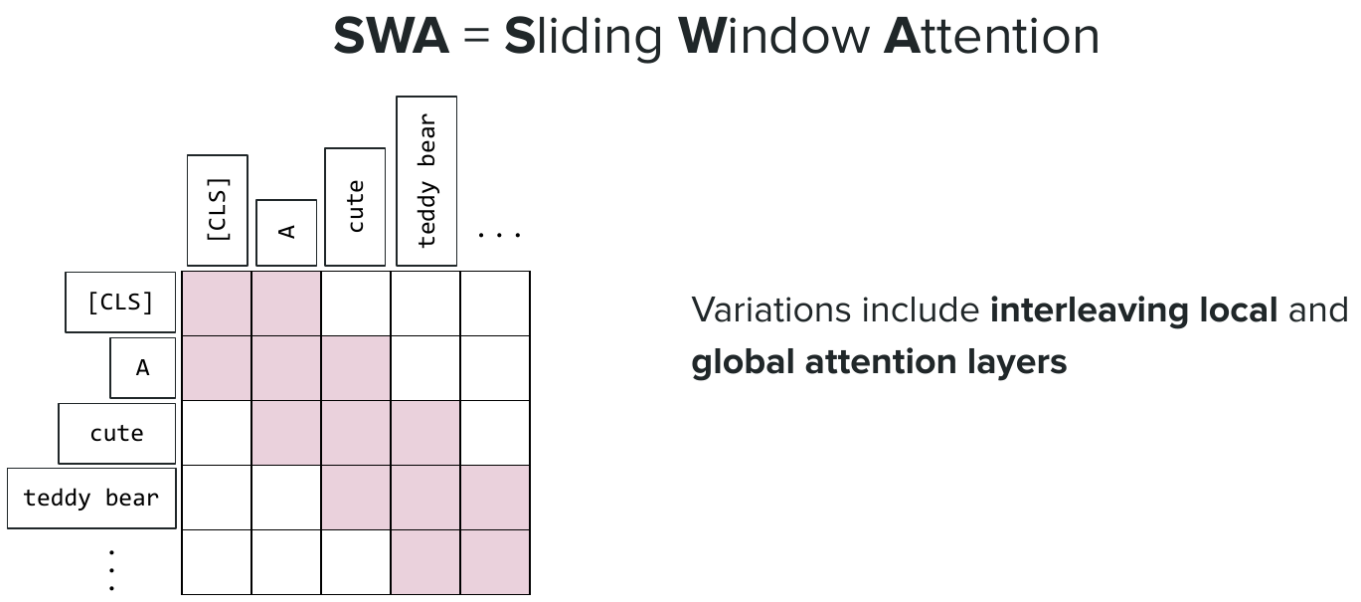


Figure 4: Sliding Window Attention (SWA): each token attends only to a local neighborhood (shaded), not the full sequence.

In practice, implementations use techniques like **tiling** to avoid ever materializing the full $n \times n$ attention matrix. Modern models often **interleave** local and global attention layers (Mistral 7B, for instance, uses sliding-window attention – SWA – at every layer, per its 2023 announcement). Stacking local-attention layers effectively grows a token's **receptive field** over depth, directly analogous to receptive fields in convolutional networks.

Q: Is this restriction applied *after* the full attention matrix (i.e. the softmax) is computed?
A: No – if you computed the full softmax over everything first, you wouldn't save any compute. In practice, implementations use a technique called **tiling**: a set of clever operations that avoid ever materializing the full $QK^\top/\sqrt{d}$ matrix in the first place, rather than computing everything and discarding the irrelevant parts afterward.

Q: Is this comparable to convolutions?
A: Yes – there are real similarities to the computer vision world here. Stacking layers of local/sliding-window attention grows a token's effective **receptive field** over depth, exactly analogous to how stacking convolutional layers grows the receptive field of a CNN.

Sharing Projection Matrices Across Heads

Rather than each of the h attention heads having its own key/value projection matrices, these can be shared:

- **Multi-Head Attention (MHA)**: every head has its own query, key, and value projections (the original/standard setup).
- **Grouped Query Attention (GQA)** (Ainslie et al. 2023, *"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints"*): key/value projection matrices are shared within g groups of heads (each group of size h/g).
- **Multi-Query Attention (MQA)**: the extreme case – all h heads share a single set of key/value projections.

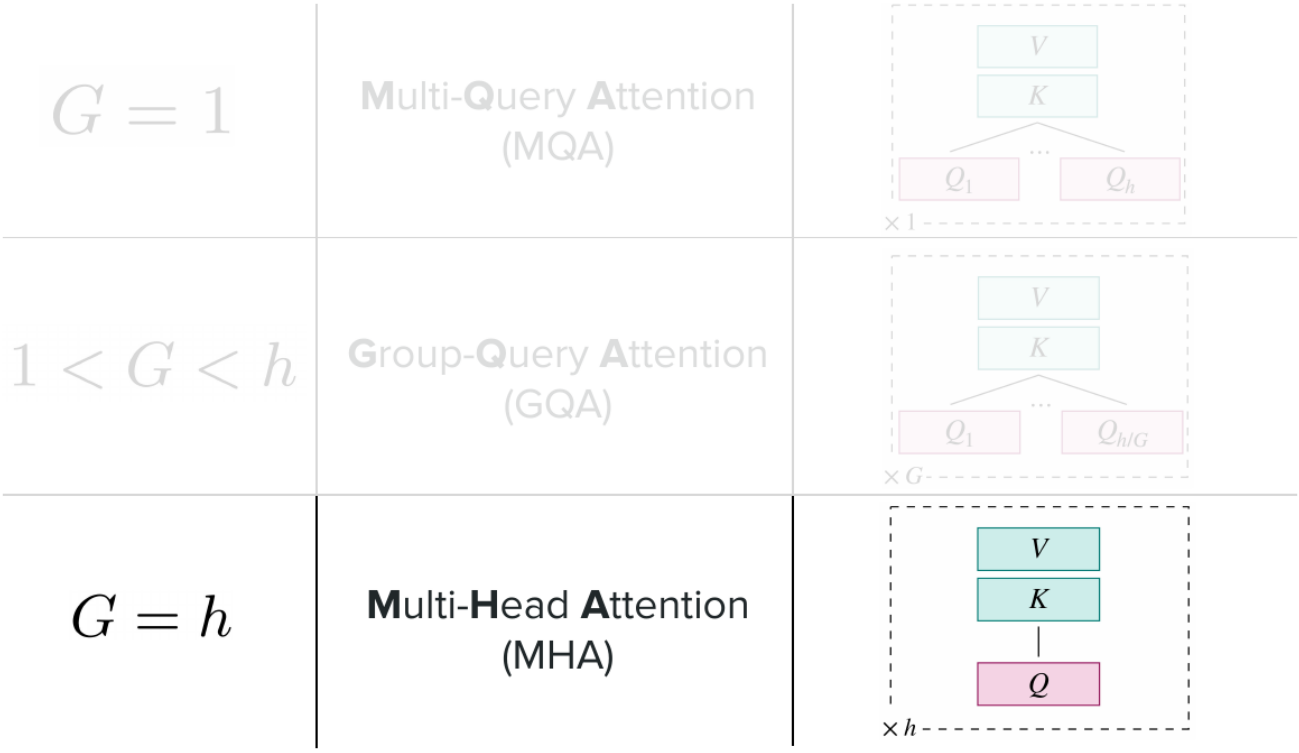


Figure 5: Spectrum of head-sharing: G=1 (MQA), 1<G<h (GQA), G=h (MHA, no sharing).

Query projections are kept separate because queries capture "what am I looking for," where diversity across heads is more valuable; keys and values, by contrast, are repeatedly reused at every decoding step (stored in the **KV cache**, covered in a later lecture), so sharing them across heads reduces memory footprint. The choice among MHA/GQA/MQA depends on the performance/latency/memory trade-off desired; GQA is common in many recent models, though not universal.

Q: How do you know which of MHA, GQA, or MQA to use?
A: The choice is generally driven by a few factors: how well the model performs with each option, and how much you care about things like latency and cost. It also depends on model size and how much compute you want to save, as well as your input length – there's no single universal answer; that said, a lot of recent models tend to share projection matrices, so GQA is what you'd typically see, though it's not the case for every model.

---

2 Part 2: The Transformer Model Family

Three categories of transformer-based models derive from the original architecture:

Category	I/O	Examples	Popularity
Encoder-decoder	Text to text	T5, mT5, ByT5	∼2018–2022
Encoder-only	Projection of embedding for class prediction (e.g. sentiment extraction)	BERT, DistilBERT, RoBERTa	
Decoder-only	Text to text	GPT series	Popular now

Encoder-Decoder Models: The T5 Family

T5 ("Text-To-Text Transfer Transformer") keeps both the encoder and decoder. Variants include **mT5** (multilingual data and vocabulary) and **ByT5** (tokenizer-free, operating at the byte level – a vocabulary of $2^8 = 256$ instead of the usual $O(10^4)$–$O(10^5)$).

T5's pretraining objective differs from next-token prediction: it uses **span corruption**. Contiguous spans of the encoder input are replaced with **sentinel tokens**, and the decoder must reconstruct each corrupted span in sequence – generating tokens between consecutive sentinel tokens (up to sentinel $n+1$) to recover the missing text. Training uses teacher forcing, feeding the full target sequence to the decoder at once.

Decoder-Only Models

Modern LLMs drop the encoder entirely: no cross-attention is needed, and each decoder block consists only of masked self-attention and a feedforward network (FFN). Over time, the field shifted from encoder-decoder architectures toward decoder-only ones, as compute invested in the decoder (via the simple, highly scalable next-token-prediction objective) proved to align well with building generally helpful chatbot-style assistants – decoder-only models are the focus of subsequent lectures.

Encoder-Only Models: BERT

BERT stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers (Devlin et al., *"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,"* submitted October 2018). It keeps only the encoder. Because there is no causal mask, every token attends to every other token (both left and right context) – true **bidirectionality**, in contrast to the causal (left-to-right) masked self-attention used in decoder-only models like GPT.

BERT was published the same year as another bidirectional representation-learning paper, **ELMo** (Peters et al., *"Deep Contextualized Word Representations,"* submitted February 2018 – i.e. earlier than BERT), based on a stacked bidirectional LSTM, which received comparatively less attention partly due to the scaling limitations inherent to recurrent architectures. (As an aside, both model names – ELMo and BERT – are references to *Sesame Street* characters, a naming convention later continued by other papers in the field.)

Two-Stage Training: Pretraining and Fine-Tuning

(1) **Pretraining**: learn general-purpose embeddings from largely unlabeled/self-supervised data, using two joint objectives:

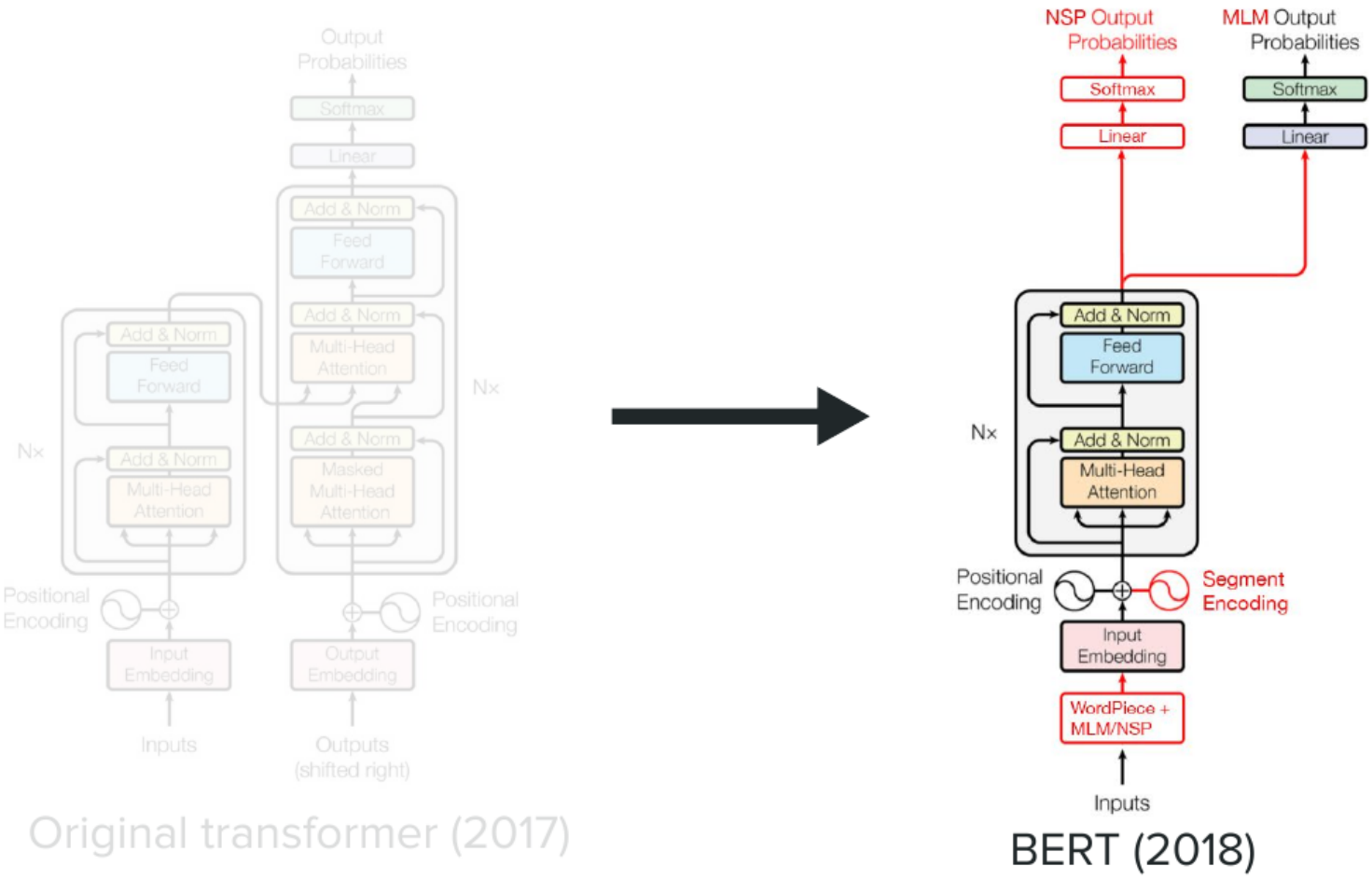


Figure 6: From the full transformer (left) to BERT (right): only the encoder is kept, with an added segment encoding and MLM/NSP output heads.

- **Masked Language Model (MLM)**: predict randomly masked tokens from their bidirectional context. Selected tokens are replaced with a `[MASK]` token 80% of the time, left unchanged 10% of the time, and replaced with a random token 10% of the time.
- **Next Sentence Prediction (NSP)**: given two sentences, predict whether the second truly follows the first in the original corpus (50% of the time they are consecutive; 50% of the time they are randomly paired) – a binary classification task using the `[CLS]` token's representation.

(2) **Fine-tuning**: attach a small task-specific head (e.g. a linear layer) on top of the pretrained embeddings and train on labeled data for the target task, often needing comparatively little labeled data.

Q: Doesn't predicting a next sentence sound like a decoder (generation) task – so is BERT still purely an encoder here?
A: Yes, still purely an encoder. NSP isn't about *generating* the next sentence – it's about taking two *already-given* sentences, placing them one after the other, and predicting whether they were truly consecutive in the original text. That's a binary classification task (using the `[CLS]` representation), not a generation task, so no decoder is involved.

Pros: pretraining requires little to no labeled data; resulting embeddings transfer well and require minimal fine-tuning data to exceed then-state-of-the-art performance. **Cons**: no text generation capability (no decoder); the two-stage process adds complexity relative to simpler one-shot training paradigms.

Tokens and Embeddings

BERT uses the **WordPiece** tokenizer (subword merges learned to maximize likelihood on the training corpus; vocabulary size on the order of 30k). Special tokens: `[CLS]` (prepended; its final representation is used for sequence-level classification), `[SEP]` (separates two sentences, used for NSP), and `[PAD]` (pads sequences to a fixed length for batching).

Each token's input representation sums three learned embeddings:

- **Token embedding** (from a large lookup table),
- **Position embedding** (as in the original transformer),
- **Segment embedding**: a new addition – one embedding for all tokens of "segment A" (first sentence) and another for all tokens of "segment B" (second sentence), intended to help the model represent sentence-pair relationships (a hypothesis later challenged, notably by RoBERTa).

Q: What does the segment embedding actually do?
A: It's simply learned: two embeddings (one for segment A, one for segment B) that get additively added to the token's other embeddings, just like the token and position embeddings. The model learns them via ordinary gradient descent – there's no special mechanism beyond that.

Q: Does every token in the same sentence get the same segment encoding?
A: Yes – all tokens in the first sentence get segment A's embedding, and all tokens in the second sentence get segment B's embedding.

Worked Example

For the sentence "This teddy bear is SO CUTE!", preprocessing (lowercasing for the uncased model: "this teddy bear is so cute!"), WordPiece tokenization (`this | teddy | bear | is | so | cute | !`), and adding `[CLS]` at the start, `[SEP]` at the end, and `[PAD]` tokens to reach a fixed sequence length yields, e.g.:

```
[CLS] this teddy bear is so cute !  [SEP] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
```

with position indices $0, 1, \ldots, 14$ assigned left to right, and segment labels `A` for every real-sentence token including the trailing `[SEP]` (positions 0–8) and `B` for the `[PAD]` filler tokens (positions 9–14). Each token embedding is summed with its position and segment embeddings to get a position- and segment-aware embedding, then passed through the (pretrained) encoder stack (self-attention + FFN per layer). For this sentence-level classification task, only the final `[CLS]` embedding is kept and passed through a small feedforward classification head (labeled **SE** for "sentiment extraction" here); the other token embeddings are discarded for this task (though they remain useful for token-level tasks, e.g. predicting answer start/end spans in question answering, each with their own FFN head).

Q: What does this feedforward classification head actually correspond to?
A: It's a mapping from the dimension of the output (CLS) embedding to the dimension of your task

of interest – e.g. positive/negative for sentiment. Concretely, there's typically a hidden layer of some chosen size, with two matrices: one projecting from the CLS embedding to that hidden layer, and one projecting from the hidden layer to the final output (the class probabilities). These weights are learned to perform the classification task based on what the encoder has produced.

Q: Why do we throw away all the other (non-CLS) output embeddings?
A: Because for a sentence-level classification task, we don't need them – by convention, we operate on the CLS token, whose final representation has, via self-attention, mixed in information from every other token in the sequence, making it a suitable summary for classification. That said, this is specific to sentence-level classification: for token-level tasks (e.g. question answering, where you want to detect the start/end of an answer span), each token's embedding *is* used, typically with its own feedforward head.

Q: What are the query, key, and value for the CLS token, specifically?
A: Exactly the same process as for any other token: the CLS token's embedding gets projected into a query, a key, and a value, participates in the same attention computation as everything else, and ends up with an output representation that has attended to all the other tokens – there's nothing special about how CLS is treated in the attention mechanism itself; the special part is just the convention of using its *final* representation for classification.

Notation and Scale

The paper uses L (number of layers, called N in the original transformer paper), H (hidden/embedding dimension, called d_{model}), and A (number of attention heads, called h). On the data side, checkpoints vary by whether they are language-specific or multilingual, and by **cased** vs. **uncased** (whether input casing is preserved). Released sizes span a wide range:

Model	L	H	A	**Parameters**
BERT-Tiny	2	128	2	4M
BERT-Mini	4	256	4	11M
BERT-Small	4	512	8	30M
BERT-Medium	8	512	8	42M
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

Fine-Tuning in Practice

Common tricks: initialize from an already massively-pretrained checkpoint; optionally **freeze early layers** for a better complexity/performance trade-off; strong results are achievable with minimal labeled data, depending on the target task's complexity and its proximity to the pretraining data distribution and objectives. Two broad use cases: **sequence classification** (e.g. sentiment extraction, using the `[CLS]` embedding) and **token classification** (e.g. question answering, using per-token embeddings).

Takeaways and Shortcomings

Benefits: state-of-the-art results (at the time); embeddings that are (close to) truly contextual; adaptable to many classification tasks – widely used in industry for anything encoding-related. **Limitations**: limited context window size; computationally expensive, a hard sell for low-latency/cost-sensitive applications; and a comparatively complex training paradigm (MLM + NSP pretraining, followed by a required fine-tuning stage).

BERT Variants

Beyond DistilBERT and RoBERTa (below), **ALBERT** is another notable parameter-efficient variant.

DistilBERT

Motivated by BERT's latency and parameter cost, DistilBERT (Sanh et al. 2019, *"DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter"*) applies **knowledge distillation**: rather than training a smaller (**student**) model on hard labels alone, it is trained to match the output probability distribution of a larger (**teacher**) model, since "the soft targets contain almost all the knowledge" of the teacher (Hinton et al., *"Dark Knowledge,"* 2014). The training objective is the **KL divergence** between the teacher and student distributions:

$$\mathrm{KL}(T \,\|\, S) = \sum_{c} y_T^{(c)} \log \frac{y_T^{(c)}}{y_S^{(c)}}$$

If the teacher distribution y_T collapses to a hard one-hot label, this reduces to the standard cross-entropy loss $-\log y_S$. Concretely, DistilBERT halves the number of layers relative to BERT-base (N=12 $\rightarrow$ N=6), running roughly **1.6× faster** while retaining about **97%** of BERT-base's performance – distillation compensating for the reduced capacity.

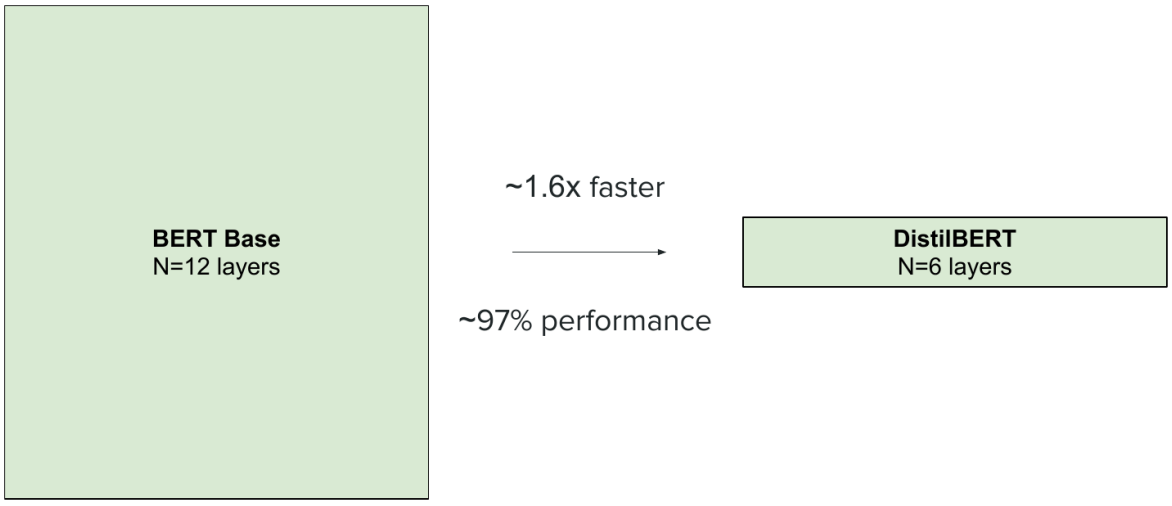


Figure 7: DistilBERT: half the layers of BERT-base, ∼1.6× faster, ∼97% of the performance.

RoBERTa

RoBERTa (Liu et al. 2019, *"RoBERTa: A Robustly Optimized BERT Pretraining Approach"*) is a thorough empirical study of how to better optimize BERT's original recipe, finding that **dropping the NSP objective (and segment encodings)** had essentially no effect on performance (DistilBERT had also dropped it), and introducing additional refinements:

- **Dynamic masking**: the masking pattern for MLM is regenerated (rather than fixed) each time a training example is seen (e.g. each epoch) – static → dynamic masking across epochs.
- **More and richer training data**: pretraining corpus size grew from 16GB to **160GB**, with substantially longer training (1M steps at batch size 256, vs. BERT's roughly 500K steps at batch size 8K) – confirming that the original BERT was undertrained.

Net result: roughly **+4% across benchmarks** using the same underlying architecture as BERT – gains attributable entirely to training recipe and data, not architectural changes.

Summary

This lecture traced how the transformer architecture has evolved since 2017 – from learned absolute position embeddings toward relative-position schemes injected directly into attention (T5-style bias, ALiBi, and especially RoPE), from post-norm/LayerNorm toward pre-norm/RMSNorm, and from full quadratic attention toward local/sliding-window attention and shared key-value projections (GQA/MQA) for efficiency. We then surveyed the transformer-derived model families – encoder-decoder (T5, with its span-corruption objective), decoder-only (the basis of modern LLMs), and encoder-only – with a detailed look at **BERT**: its bidirectional self-attention, WordPiece tokenization, CLS/SEP/segment embeddings, MLM/NSP pretraining objectives, and fine-tuning paradigm, along with the DistilBERT (distillation) and RoBERTa (NSP removal, dynamic masking, more data) refinements.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 3: LLMs, Decoding Strategies, Prompting, and Inference Efficiency

Afshine Amidi, Shervine Amidi

Recap of Lectures 1 and 2

We introduced self-attention and the transformer architecture, then surveyed the three model families derived from it: **encoder-decoder** (e.g. T5, text-in/text-out), **encoder-only** (e.g. BERT, producing meaningful embeddings for classification via the CLS token), and **decoder-only** (e.g. GPT, text-in/text-out, with cross-attention removed since there is no encoder). Today we formally introduce large language models (LLMs), which are overwhelmingly decoder-only.

1 Part 1: What Makes an LLM "Large," Mixture of Experts, and Decoding

Defining an LLM

An LLM is, first, a **language model**: it assigns probabilities to token sequences, in practice by predicting the next token. It is **large** along three axes:

- **Model size**: commonly hundreds of billions of parameters (at least on the order of a billion to qualify as "large").
- **Training data**: hundreds of billions to tens of trillions of tokens.
- **Compute**: substantial GPU resources, though consumer-hardware optimizations have narrowed this somewhat.

Under today's definition, a model like BERT would *not* count as an LLM, since it is encoder-only and does not generate text – despite sometimes being informally grouped with LLMs in earlier (pre-2020) usage. The backbone of an LLM keeps only the decoder-only components: masked self-attention, feedforward network (FFN), and add&norm – no cross-attention. Examples include GPT, LLaMA, Gemma, DeepSeek, Mistral, and Qwen; more than 90% of modern LLMs are decoder-only.

Mixture of Experts (MoE)

Given the enormous parameter counts of LLMs, must *every* parameter be activated for every forward pass? Analogy: if you have a math question and a room containing a mathematician, a physicist, a chemist, and a historian, you would ask the mathematician – not everyone. **Mixture of experts** (Shazeer et al. 2017, *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"*) applies this idea: given n experts $E_1, \ldots, E_n$, a learned **gate** (or **router**) G decides which expert(s) should process a given input x.

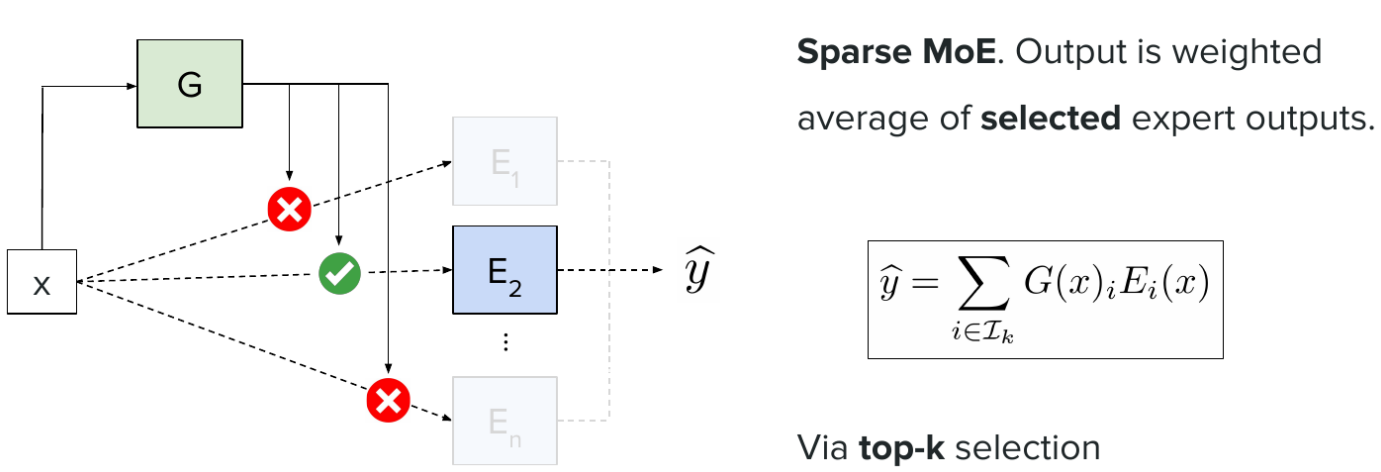


Figure 1: Sparse MoE: the gate G routes input x to a top-k subset of experts; output is a weighted average of the selected experts' outputs.

Q: How do you train the gate G and the experts E?
A: Typically jointly, as part of ordinary training: you do a forward pass, compute the loss, and backpropagate as usual. (There are additional challenges specific to training MoEs well, covered below.)

Q: What is the architecture of the experts themselves?
A: At this point, just think of each expert as "some network" – the specific architecture is pinned down shortly (spoiler: each expert is itself a feedforward neural network).

Dense vs. Sparse MoE

- **Dense MoE**: all experts contribute, weighted by the gate's output (a probability distribution over experts):

$$\hat{y} = \sum_{i=1}^{n} G(x)_i \, E_i(x)$$

- **Sparse MoE**: only the **top-k** experts (by gate probability) are activated, saving compute:

$$\hat{y} = \sum_{i \in \text{top-}k} G(x)_i \, E_i(x)$$

Compute cost is measured in **FLOPs** (floating-point operations); sparse MoE uses fewer FLOPs per forward pass than dense MoE.

Where to Place the Experts

Within a decoder block, the **feedforward network (FFN)** is where MoE is applied – it is the more parameter- and compute-heavy component. Rough parameter counts:

$$\#\text{params}_{\text{FFN}} \approx 2\, d_{\text{model}} \cdot d_{\text{ff}} \quad \text{(plus biases)}, \qquad \#\text{params}_{\text{attn}} \approx d_{\text{model}} \cdot (d_k + d_q + d_v)$$

where d_{ff} (often $O(10^3)$–$O(10^4)$) is typically much larger than the attention head dimensions ($O(10^2)$–$O(10^3)$). In practice, each expert is itself an FFN, and **routing happens at the token level**: different tokens (even within the same sequence) can be routed to different experts, and each layer has its own independently trained gate and set of experts (weights are not shared across layers).

Q: Of the three components in a decoder block – masked self-attention, FFN, normalization – where would you put the mixture of experts?
A: The **FFN**. It's the component with the most parameters and the most operations: its hidden layer projects up to a dimension d_{ff} that's typically much larger than the model dimension, whereas the attention layer's projection dimensions (for queries/keys/values) are comparatively small – so that's where swapping in multiple experts has the biggest effect on capacity.

Q: Is the routing decision tied to the attention heads in any way (e.g. one set of experts per head)?
A: No – the number of experts is independent of the number of attention heads; they're separate design choices.

Q: Does every decoder block (layer) get its own set of experts?
A: Yes, and those weights are not shared across layers – it's entirely possible, for instance, that layer 1 routes a given token to expert 3 while layer 2 routes that same token to expert 1; everything is learned independently per layer.

Q: At what point during inference is the routing decision actually made?
A: Right after the self-attention layer, at the start of the FFN block. By then the token's representation is already contextual (it has attended to other tokens). That representation is fed into the gate G, which outputs a probability distribution over the n experts; in the sparse case, only the top-k experts (by that probability) are actually computed.

Q: Is there a separate router for each attention head?
A: No – there's only one router per layer. The multiple attention heads' outputs are concatenated and re-projected *before* reaching the router, so the router only ever sees one (per-layer) representation per token, not one per head.

Scaling Capacity Without Scaling Compute

MoE lets total parameter count (**capacity**) grow while keeping **active parameters** (those used per forward pass) controlled – enabling models with trillions of total parameters (e.g. the Switch Transformer, at ∼1+ trillion parameters) while remaining computationally tractable, and often more **sample-efficient** during training than a dense model of comparable active size.

Q: If you increase the number of experts, does that increase the total number of model parameters?
A: Yes – and that's actually central to the appeal of MoE: you can scale up a model's total capacity (parameter count) without having to proportionally increase the *active* parameters used at inference time. This is exactly how some MoE-based models end up with total parameter counts in the trillions, well beyond typical dense models, while keeping the compute cost of a single forward pass under control.

Routing Collapse and Load Balancing

A key training challenge is **routing collapse**: the router may consistently favor only a few experts, leaving others rarely or never used – a symptom highlighted in Fedus et al. 2021, *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"* (also notable for scaling MoE-based LLMs to over a trillion parameters). This is mitigated by adding a **load-balancing auxiliary loss** term to the training objective:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot n \cdot \sum_{i=1}^{n} f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i, P_i is the average routing probability assigned to expert i (across the batch), n is the number of experts, and α is a hyperparameter. This term pushes routing toward a more uniform distribution across experts. **Noisy gating** (adding noise to the gate's logits before top-k selection) is another technique used to encourage broader expert utilization, conceptually similar in spirit to dropout. Visualizations of token-to-expert routing patterns (e.g. in Jiang et al. 2024, *"Mixtral of Experts"*) are a common way to sanity-check that routing is reasonably well distributed across experts in practice.

Q: Could you use ordinary Dropout instead, to fight routing collapse?
A: Sure – it can be combined with the other techniques mentioned. It's the kind of idea (randomly perturbing what gets used) that generalizes well across different settings, and nothing stops it from being layered on top of the load-balancing loss or noisy gating.

Q: How exactly do you take the derivative through f_i and P_i for backpropagation, given that f_i involves a hard top-k selection?
A: P_i is straightforward – it's just the average of the gate's (softmax) output probabilities across the batch, so it's directly differentiable. f_i is trickier since it counts how many tokens were *actually routed* to expert i (a discrete, non-differentiable quantity); there exist standard techniques to handle this in practice (e.g. treating it as a soft/relaxed count, or not requiring it to carry gradient at all since it mainly serves to weight the differentiable P_i term), though the full mechanics are beyond what's needed here.

Generating the Next Token

Given a trained LLM, how do we go from an output probability distribution over the vocabulary to an actual next token?

Q: Is this about training, or inference?
A: Inference – this is about how a trained model actually generates a response. During training, you instead compare the output probabilities against the true label (typically a hard target) via cross-entropy; the decoding strategies discussed here (greedy, beam search, sampling) only come into play once the model is already trained and you're generating text from it.

Greedy Decoding

Always pick the highest-probability token. Simple, but deterministic (no diversity across runs) and only **locally optimal**: greedily choosing the highest-probability token at each step does not guarantee the

highest-probability *sequence* overall, since a lower-probability first token might lead to much higher-probability continuations.

Beam Search

Maintains the k most probable partial sequences (**beam width** k) at each step, rather than just one, aiming for a more globally optimal sequence. Sequence probability is computed via summed log-probabilities:

$$\log P(\text{sequence}) = \sum_t \log P(\text{token}_t \mid \text{token}_{<t})$$

Since each term is ≤ 0, longer sequences accumulate more negative log-probability, biasing beam search toward shorter outputs; a length-normalization term (e.g. dividing by n^γ for sequence length n and some hyperparameter γ) counteracts this. Beam search is computationally heavier (must track k paths) and, since it still targets the single most likely sequence, lacks diversity/creativity – it is mainly used for tasks like machine translation where a single high-likelihood output is desirable.

Sampling

Instead of taking the most likely token, **sample** from the output probability distribution – introducing controlled randomness and variety. To avoid sampling very low-probability (poor-quality) tokens, sampling is typically restricted to a subset of high-probability candidates:

- **Top-k sampling**: restrict to the k highest-probability tokens, then sample among them.
- **Top-p (nucleus) sampling**: restrict to the smallest set of highest-probability tokens whose cumulative probability exceeds threshold p, then sample among them.

From Logits to Probabilities: Temperature

The final decoder layer projects the d_{model}-dimensional token representation into a vector of size $|V|$ (vocabulary size), then applies a **softmax with temperature** T:

$$P(\text{token} = i) = \frac{\exp(x_i/T)}{\sum_j \exp(x_j/T)}$$

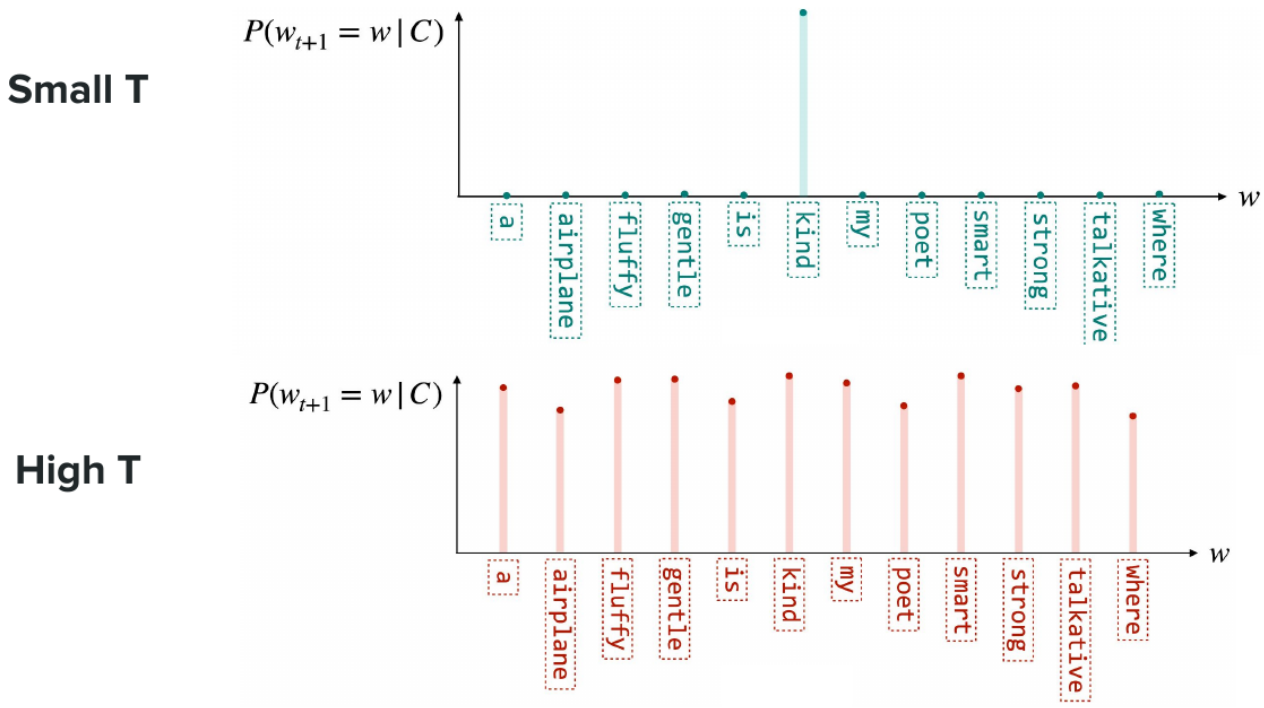
Let $k = \arg\max_i x_i$. Factoring out $\exp(x_k/T)$ from numerator and denominator:

$$P(\text{token} = i) = \frac{\exp\big((x_i - x_k)/T\big)}{\sum_j \exp\big((x_j - x_k)/T\big)}$$

Since $x_j - x_k \leq 0$ for all j (with equality only at $j = k$):

- As $T \to 0^+$: for $i = k$, the exponent is $0/T = 0$, giving a term of 1; for $i \neq k$, the exponent $\to -\infty$, giving a term of 0. The distribution collapses to a **spike** at the highest-logit token (equivalent to greedy decoding).
- As $T \to +\infty$: every exponent $\to 0$, so every term $\to 1$, yielding a **uniform distribution** over the vocabulary.

Practically: **low temperature** $\to$ more deterministic, "safe" outputs; **high temperature** $\to$ more diverse, creative outputs. Note that the transformer's internal computations are entirely deterministic – sampling is the *only* non-deterministic step in generation (though in practice, even $T = 0$ can yield slightly different outputs across runs due to floating-point operation ordering on hardware, a subtle effect covered in optional supplementary reading on non-determinism in LLM inference).



"Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

Figure 2: Low T (top) concentrates probability mass on one token; high T (bottom) flattens the distribution across the vocabulary.

Guided (Constrained) Decoding

To force output into a specific format (e.g. valid JSON), rather than naively generating and retrying until valid, **guided decoding** filters out invalid next tokens at each generation step based on the target grammar (e.g. via a finite state machine or context-free grammar), only allowing tokens consistent with the required structure.

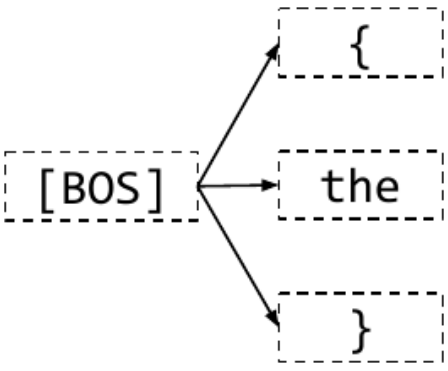


Figure 3: Guided decoding: only tokens consistent with the target grammar are ever allowed as candidates.

For example, given the prompt "Generate a description of my 33-year-old teddy bear who likes reading. Do this in JSON format," guided decoding would only ever allow tokens consistent with valid JSON syntax at each step (e.g. after `{`, only a valid key-opening quote is permitted, not arbitrary words), reliably yielding something like `{"first_name": "teddy", "last_name": "bear", "age": 33, "hobby": "reading"}` rather than requiring generate-and-check retries.

2 Part 2: Prompting Strategies and Inference Efficiency

Context Length

The number of input tokens an LLM can process is called **context length** (also: context size, window size). Modern LLMs support context lengths in the tens of thousands to millions of tokens, with typical orders of magnitude varying both by input type and by model.

Q: Is the context length exactly what's shown to the model during self-attention?
A: Yes – and recall from Lecture 2 that this is also precisely why techniques for managing the $O(n^2)$ complexity of self-attention (e.g. sliding-window attention) matter more and more as context length grows: they're what make it computationally feasible to support ever-larger context windows.

Context Rot

More context is not strictly better: a phenomenon dubbed **context rot** (Hong et al. 2025, *"Context Rot: How Increasing Input Tokens Impacts LLM Performance"*) shows that a model's ability to retrieve a specific piece of information (tested via "needle in a haystack" experiments, where an answer is buried in increasingly long text) degrades as context length grows – especially when **distractor** content is present. This motivates carefully targeting relevant context (e.g. in retrieval settings) rather than simply maximizing context length.

Prompt Structure

While there is no formal theory of prompt design, prompts commonly contain four elements:

- **Context**: sets the scene (e.g. current date/time, persona).
- **Instructions**: the task to perform.
- **Inputs**: the data the instructions operate on.
- **Constraints**: restrictions on the output (e.g. safety guidelines), sometimes invisible to the end user.

In-Context Learning (ICL)

Adapting model behavior via the prompt alone (no weight updates) – introduced and popularized in Brown et al. 2020, *"Language Models Are Few-Shot Learners"* (the GPT-3 paper):

- **Zero-shot**: only the task query is given, no examples; performance depends heavily on the base model's own capability.
- **Few-shot**: example input/output pairs are provided in the prompt before the actual query, steering the model toward the desired task/format; typically improves performance.

Few-shot examples generally improve performance by clarifying intent, but at the cost of more tokens (hence more compute/latency) and the effort of curating good examples. With increasingly capable reasoning models, well-written natural-language **instructions** can sometimes match or exceed few-shot performance, since examples can over-constrain the model to the specific distribution of the examples

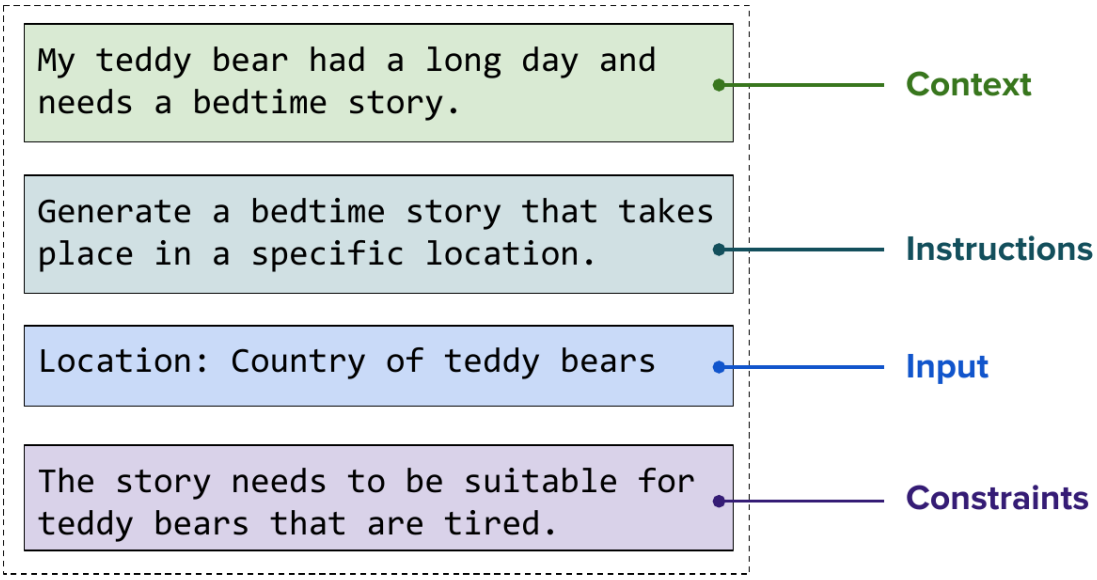


Figure 4: The four common elements of a prompt, illustrated on a bedtime-story request.

shown, hurting generalization to unseen inputs – an active area of ongoing research (e.g. "plan-and-solve" style prompting).

Chain of Thought (CoT)

Prompting the model to produce a **rationale** before its final answer measurably improves performance (Wei et al. 2022, *"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"*) – e.g. "how old will this bear be next year?" answered by first working out its current age, rather than jumping straight to a bare number. CoT can be combined with few-shot examples (showing reasoning-then-answer pairs) and aids **interpretability/debugging**: since the reasoning is expressed in natural language tokens rather than opaque weights, errors can often be traced directly from the visible reasoning chain. The trade-off is increased inference latency and cost from generating more tokens.

Self-Consistency

An extension of CoT (Wang et al. 2022, *"Self-Consistency Improves Chain of Thought Reasoning in Language Models"*): sample the model **multiple times in parallel**, parse out the final answer from each reasoning chain, and take a **majority vote** over the resulting answers. This is more robust than a single sample, at the cost of extra parallel compute – a direct trade-off between performance and added cost.

Q: Do you need a benchmark with ground-truth labels to confirm self-consistency actually helps?
A: Yes, absolutely – you need both a benchmark and ground-truth labels to verify that majority voting over multiple samples is indeed more accurate than a single sample. (A related practical question is *how* you locate the answer within each reasoning chain to vote on – common approaches include instructing the model, via the prompt, to place its final answer last so it can be reliably extracted, using regex-based extraction, or even using another LLM call to pull out the answer.)

Q: Are the multiple samples generated sequentially, each depending on the previous one?
A: No – they're sampled independently, in parallel, from the same starting prompt (sampling from the output distribution at each step, same as ordinary generation, just run as several independent branches). None of the branches see each other's outputs; the answers are only combined *after* all

branches finish, via the majority vote. Because of this, total latency is roughly bounded by whichever branch takes longest, not by the sum of all branches.

Inference Efficiency Techniques

Techniques are split into **exact** methods (identical computation, executed more efficiently) and **approximate** methods (trading some exactness for speed).

KV Cache (Exact)

During autoregressive decoding, generating a new token only requires its own new query, but keys and values for all *previous* tokens must be reused. Rather than recomputing them, the **KV cache** stores previously computed key and value matrices, avoiding redundant computation. (This concept doesn't arise during training, where teacher forcing processes the full target sequence in one pass.)

Q: Do we use KV caching during training too?
A: No – during training, **teacher forcing** feeds the entire input sequence to the model at once (a single forward pass over the full target sequence with the causal mask), so there's no notion of incrementally generating and caching keys/values token by token the way there is at inference time. The KV cache is purely an inference-time optimization.

Grouped Query Attention (Exact, Recap)

As covered in Lecture 2, sharing key/value projection matrices across groups of heads (**GQA**, with **MQA** as the extreme single-group case, and standard multi-head attention as the no-sharing case) directly reduces the size of the KV cache that must be stored.

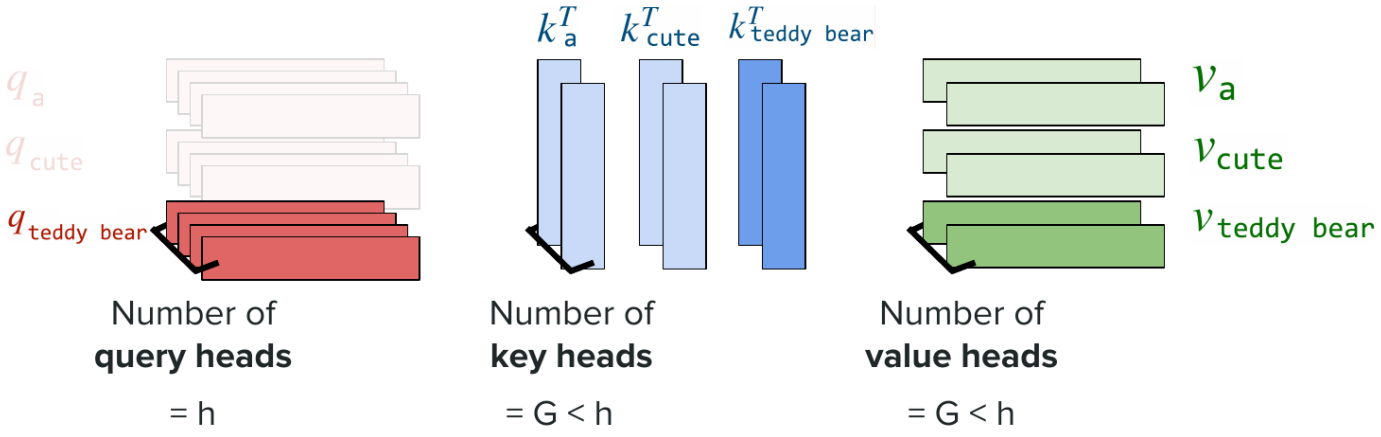


Figure 5: With GQA, the number of cached key/value heads ($G < h$) is smaller than the number of query heads (h), directly shrinking the KV cache.

PagedAttention (Exact)

Naively, an inference server might reserve a memory block sized to the *maximum* context length for every request, even though decoding stops early (at the end-of-sequence token) – wasting substantial memory

(**internal fragmentation**: reserved-but-unused space) and complicating memory allocation across many concurrent requests (**external fragmentation**: gaps left by non-contiguous block allocation). **PagedAttention** (Kwon et al. 2023, *"Efficient Memory Management for Large Language Model Serving with PagedAttention,"* powering the vLLM inference engine) instead stores keys and values in **non-contiguous** memory, allocated in small fixed-size blocks (e.g. size 16) as generation proceeds, with a mapping (similar to virtual memory paging in operating systems) from token positions to their corresponding cache blocks – substantially reducing fragmentation.

Multi-Latent Attention (Exact/Approximate Hybrid)

Introduced by DeepSeek (DeepSeek 2023, *"DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model"*), this technique compresses the KV cache further. In standard multi-head attention, each of the h heads has its own key/value projection, requiring h separate cached vectors per token per matrix. Multi-latent attention factorizes the projection through a lower-dimensional **latent** bottleneck (compress, then decompress), and – notably – **shares the compression matrix across keys, values, and all h heads**, so only a single compact latent representation per token needs to be cached per transformer block (decompression matrices remain separate to recover distinct key/value representations). Beyond memory savings, the DeepSeek-V2 paper reports this also yields a performance benefit, plausibly acting as a form of regularization.

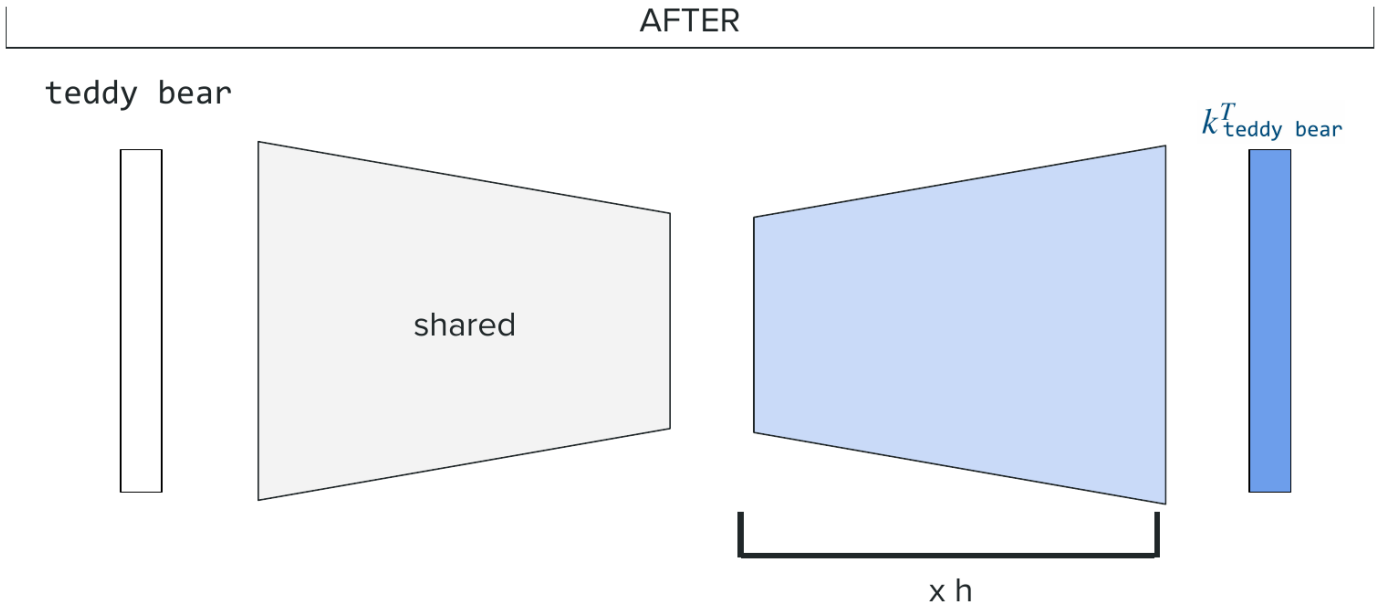


Figure 6: Multi-latent attention: a single shared, compressed latent vector per token replaces h separate key vectors.

Q: Is the low-rank latent dimension something you tune, and is it the same for every token/head?
A: The dimension is a fixed design choice, decided ahead of time rather than tuned per instance – and yes, it's the same latent dimension used throughout (it could in principle differ across parts of the model, but in practice it's typically kept the same).

Speculative Decoding (Approximate)

Uses a small, fast **draft model** to propose several candidate next tokens quickly, which are then verified in a single forward pass of the larger **target model** (Chen et al. 2023, *"Accelerating Large Language Model Decoding with Speculative Sampling"*).

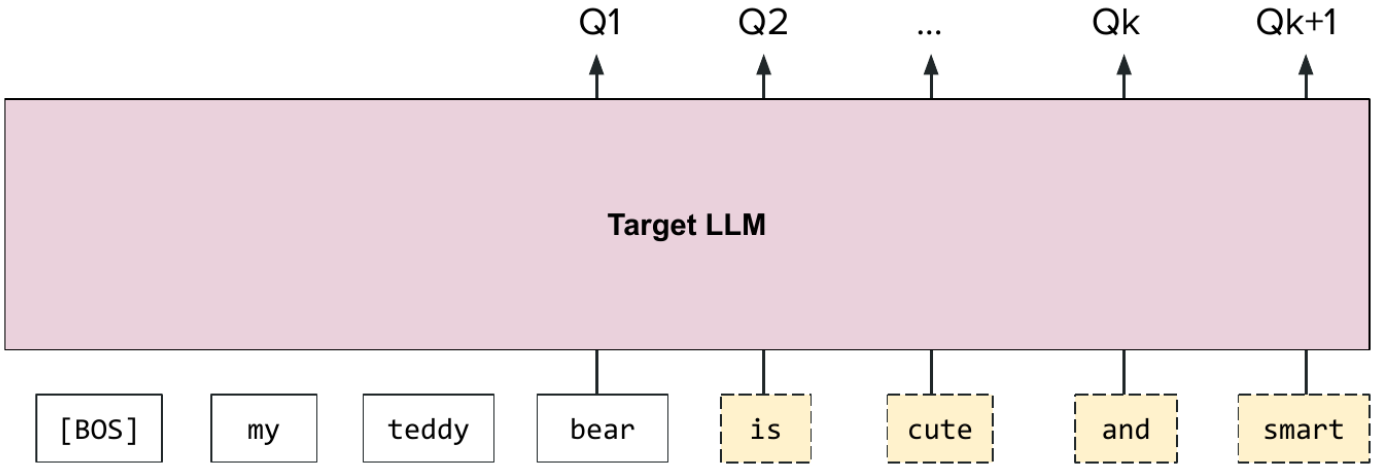


Figure 7: The target LLM verifies all draft tokens in a single forward pass, producing $Q_1, \ldots, Q_{k+1}$ at once.

Denoting the draft model's probability for a proposed token as $P_i(\text{token})$ and the target model's probability for that same token as $Q_i(\text{token})$, the acceptance rule is:

- If $Q_i(\text{token}) \geq P_i(\text{token})$: accept the token.
- Otherwise: accept with probability $Q_i(\text{token})/P_i(\text{token})$, and reject with the complementary probability $1 - Q_i(\text{token})/P_i(\text{token})$.

If a rejection occurs, generation resumes from the last accepted token by resampling from the residual distribution $[Q_i - P_i]_+$ (i.e. $\max(Q_i - P_i, 0)$, renormalized) and the process exits back to drafting from there. This rule guarantees the resulting accepted-token distribution exactly matches what the target model alone would have produced (derivable via the law of total probability). Because a single forward pass over multiple draft tokens costs roughly the same as one pass over a single token (inference being memory-bound rather than compute-bound), this can advance generation by multiple tokens per expensive target-model pass when acceptance is high. As a bonus, the same forward pass also yields the target model's distribution for the token immediately following the draft sequence.

Multi-Token Prediction (Approximate)

A related idea (Gloeckle et al. 2024, *"Better & Faster Large Language Models via Multi-Token Prediction"*), but with the "draft" capability built directly into a single model: multiple prediction heads are attached on top of the last decoder layer's representation, and the model is trained to predict several future tokens at once (rather than only the next one) – the same model plays the role of both draft and target. At inference time, all heads' predictions serve as draft tokens, which are then verified/accepted against the primary (first) head's predictions, similar in spirit to speculative decoding but without needing a separate draft model – using a greedy, rather than distribution-matching, acceptance scheme, since the modified training objective changes the theoretical guarantees available.

Summary

This lecture formally defined LLMs and their scale, introduced **mixture of experts** as a way to grow model capacity without proportionally growing inference compute (with routing, load balancing, and placement in the FFN), and covered how the next token is actually chosen from a probability distribution (greedy decoding, beam search, sampling with top-k/top-p, and the role of temperature) as well as guided decoding for structured outputs. We then covered prompt structure and in-context learning (zero-/few-shot, chain of thought, self-consistency), and closed with a tour of inference-efficiency techniques spanning exact methods (KV cache, GQA, PagedAttention, multi-latent attention) and approximate methods (speculative decoding, multi-token prediction).

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 4: Pretraining at Scale and Supervised Fine-Tuning

Afshine Amidi, Shervine Amidi

Recap of Lecture 3

We covered **mixture of experts** (sparse vs. dense, gating/routing, load balancing) as a way to scale LLM capacity without proportionally scaling inference compute, then the mechanics of **next-token generation**: greedy decoding, beam search, and sampling (with temperature controlling how spiky vs. uniform the output distribution is), plus inference-time optimizations such as the KV cache. Today we turn to how LLMs are actually **trained**: pretraining at scale, and the first post-pretraining stage, supervised fine-tuning (SFT).

Midterm Logistics

This lecture immediately preceded the midterm (covering Lectures 1–4), prompting a round of exam-format questions:

Q: Is the midterm closed notes?
A: Yes.

Q: What is the format of the multiple-choice questions?
A: Something like a question with three or four possible answers, where you choose the correct one; there are also free-form questions requiring an answer in your own words.

Q: Are we allowed to bring anything (notes, formula sheets, etc.)?
A: No – it's closed book. Just a pen.

Q: Is a calculator needed?
A: No calculator will be needed.

---

1 Part 1: Pretraining at Scale

Transfer Learning as the LLM Training Paradigm

Historically, ML practitioners trained a separate model from scratch for each task (e.g. spam detection, sentiment extraction). **Transfer learning** instead starts from a model already trained on a broad task and adapts it to a specific one. LLM training follows this paradigm via two stages: **pretraining** (learn the general structure of language and code from vast data) followed by **tuning** (adapt the pretrained model to specific behaviors/tasks).

What Pretraining Involves

Pretraining is the most expensive stage in terms of compute, cost, and data. The objective is next-token prediction over enormous, broad corpora – natural language in many languages, code in many languages, essentially anything written. Common data sources include **Common Crawl** (billions of web pages per month), Wikipedia, social media (e.g. Reddit), and code repositories/forums (GitHub, Stack Overflow). Dataset scale is measured in tokens, typically hundreds of billions to tens of trillions (e.g. GPT-3: 300 billion tokens (Brown et al. 2020, *"Language Models Are Few-Shot Learners"*); Llama 3: 15 trillion tokens (LLaMA team 2024, *"The Llama 3 Herd of Models"*)).

FLOPs vs. FLOPS

Two related but distinct notations appear constantly in this space:

- **FLOPs** (floating-point operations): a unit of *total compute*. Training a modern LLM requires on the order of 10^{25} FLOPs, roughly a function of (number of tokens) $\times$ (number of parameters), modulated by architecture (e.g. MoE models need less compute per token than equivalently-sized dense models, since only some parameters are active).
- **FLOPS** (floating-point operations *per second*): a unit of *hardware speed*, commonly listed in GPU spec sheets.

Papers don't always capitalize these consistently, so context matters.

Scaling Laws and the Chinchilla Rule

Research (Kaplan et al. 2020, *"Scaling Laws for Neural Language Models"*) found that more compute, more data, and more parameters all monotonically improve next-token prediction performance, and that larger models tend to be more **sample-efficient** (better performance per token processed). This drove a multi-year trend of ever-larger models. But compute is finite, which raises the question: for a fixed compute budget, what is the optimal split between model size and dataset size? Follow-up work (Hoffmann et al. 2022, *"Training Compute-Optimal Large Language Models"* – the **Chinchilla** paper) found a rule of thumb:

$$\text{Training tokens} \approx 20 \times \text{Number of parameters}$$

By this measure, GPT-3 (175B parameters, 300B tokens) was substantially **undertrained**. Model architecture itself was found to play a comparatively minor role relative to the data/parameter trade-off – essentially all modern LLMs are transformer-based decoder-only models.

Q: Do these scaling-law findings depend on the specific neural architecture used?
A: By now, essentially everyone agrees LLMs are transformer-based, decoder-only models, so "LLM" can be assumed to mean that throughout. Within that shared architecture, the Chinchilla paper's own claim is that architecture changes matter comparatively little – what dominates is the amount of training tokens and the model size.

Q: Is there some kind of transfer learning between successive versions of the same model (e.g. does GPT-4 reuse anything learned by GPT-3)?
A: For most of these models, which are closed-source, providers don't reveal these details, so there's no general answer that can be given with confidence. What is consistently disclosed, though, is that pretraining cost is always in the millions of dollars at minimum, regardless.

Chinchilla law.

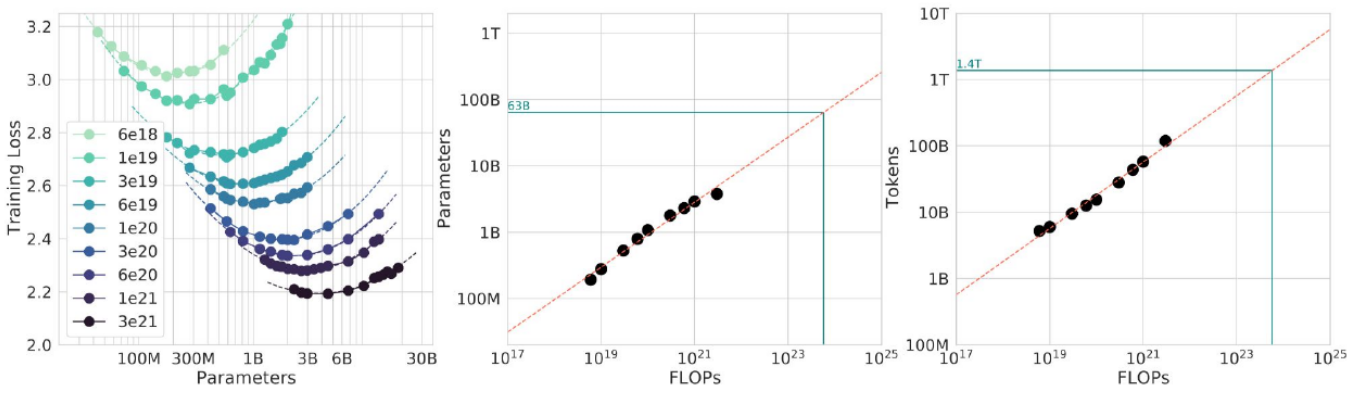


Figure 1: The Chinchilla law: for a fixed compute budget (FLOPs), there is a compute-optimal model size and token count.

Pretraining Challenges

- **Cost**: typically millions to hundreds of millions of dollars, plus environmental/ecological costs.
- **Knowledge cutoff**: the model only knows what was in its training data up to the date the dataset was assembled (the **knowledge cutoff date**, always listed in model cards). Editing/injecting new knowledge into weights afterward is difficult without causing regressions elsewhere.
- **Memorization/plagiarism risk**: since the model is trained to predict text it has seen, there is a risk of reproducing training data verbatim.

The Mechanics of Training: Memory Pressure

Training an LLM (a decoder-only transformer) on GPUs (Google uses its own TPUs; most other models are GPU-trained) involves three passes:

Q: Does the same GPU setup work for both training and inference?
A: This part of the lecture is focused specifically on training; GPU requirements differ somewhat between training and inference (e.g. training additionally needs memory for gradients and optimizer state, not just activations).

---

(1) **Forward pass**: compute **activations** at each layer, needed to compute the loss. Memory cost depends on model size, batch size, and context length (recall the $O(n^2)$ self-attention cost in sequence length n).

(2) **Backward pass**: compute **gradients** of the loss with respect to each parameter, also requiring memory.

(3) **Weight update**: apply the gradient-based update, typically via an optimizer like **Adam** (Kingma & Ba 2014, *"Adam: A Method for Stochastic Optimization"*) – or its common variant **AdamW** (Loshchilov & Hutter 2017, *"Decoupled Weight Decay Regularization"*) – which additionally maintains running estimates (first and second moments) of the gradients – more memory still.

GPU memory is limited (e.g. an H100 has 80GB), far too little to hold all of this for a model with billions of parameters on a single device.

Distributing Training Across GPUs

Data Parallelism (DP) and ZeRO

Data parallelism splits a training batch across GPUs, each holding a full copy of the model; forward/backward passes run independently, and gradients are averaged across devices via inter-GPU communication before the update. This reduces batch-related memory pressure but introduces **communication overhead**, slowing training, and still requires each device to fit a full model copy.

Q: If each GPU computes independently, how is the gradient update handled?
A: The gradient used for the update is just the *average* of the gradients computed independently on each GPU – this requires communication between the GPUs to aggregate those gradients before applying the update.

Q: Is scaling up to more and more GPUs simply always a win, or are there downsides?
A: It's not free: you do gain the ability to scale up memory (and each device still needs to fit a full model copy), but you also incur a real **communication cost** from synchronizing gradients across GPUs – so overall training becomes slower per step, even as it becomes more scalable in aggregate.

ZeRO (Rajbhandari et al. 2019, *"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"*) reduces the resulting duplication by **sharding** (partitioning, rather than replicating) quantities across GPUs:

- **ZeRO-1**: shard optimizer states.
- **ZeRO-2**: additionally shard gradients.
- **ZeRO-3**: additionally shard model parameters themselves.

Each stage trades more communication cost for lower per-GPU memory footprint; the right stage depends on model size and sensitivity to training time.

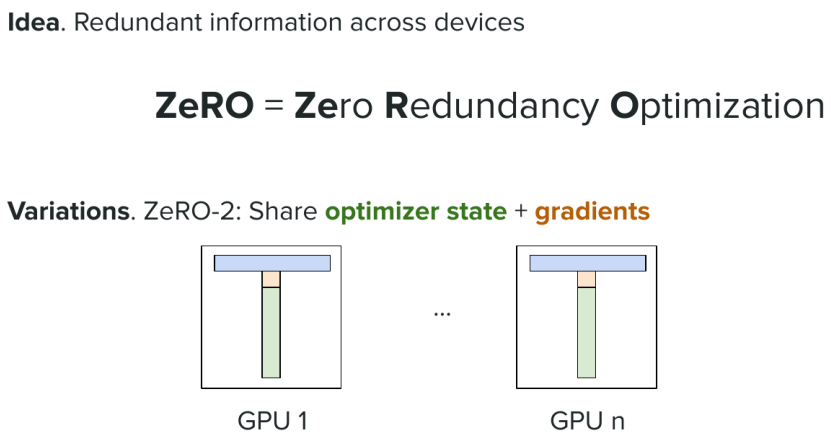


Figure 2: ZeRO-2: optimizer state and gradients are sharded (partitioned, not duplicated) across GPUs.

Model Parallelism

Rather than splitting data, **model parallelism** splits the computation itself. Common variants (see HuggingFace team 2025, *"The Ultra-Scale Playbook: Training LLMs on GPU Clusters"* for a thorough treatment):

- **Expert parallelism (EP)**: distribute different MoE experts across different devices.
- **Tensor parallelism (TP)**: split large matrix multiplications across devices.
- **Pipeline parallelism (PP)**: assign different contiguous groups of layers to different devices (e.g. GPU 1 handles layers 1–3, GPU 2 handles layers 4–6, etc.).
- **Sequence parallelism (SP)** and **context parallelism (CP)**: split the sequence-length dimension itself across devices, useful for very long input sequences.

Flash Attention

Developed at Stanford (Dao et al. 2022, *"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"*), flash attention speeds up the exact (non-approximated) self-attention computation by exploiting GPU memory structure: **HBM** (High Bandwidth Memory – large, tens of GB, but relatively slow) and **SRAM** (on-chip, much smaller – tens of MB – but roughly 10× faster).

Naively computing $\text{softmax}(QK^\top/\sqrt{d_k})V$ requires repeatedly reading from and writing to HBM at each intermediate step (matrix multiply, softmax, another matrix multiply), and this HBM traffic becomes the bottleneck. The obstacle is that softmax normalization is row-wise, seemingly requiring the full row before normalizing. Flash attention's key insight (**tiling**): the softmax of a full row can be decomposed into a combination of partial softmaxes over sub-blocks, up to a computable rescaling factor:

$$\text{softmax}(\text{full row}) = \text{combine}\big(\text{softmax}(\text{block}_1), \text{softmax}(\text{block}_2), \dots\big)$$

This lets small blocks of Q, K, V be loaded into the fast SRAM, fully processed end-to-end there, and only the final result written back to HBM – with the rescaling factors updated incrementally (an exact, iterative computation, not an approximation).

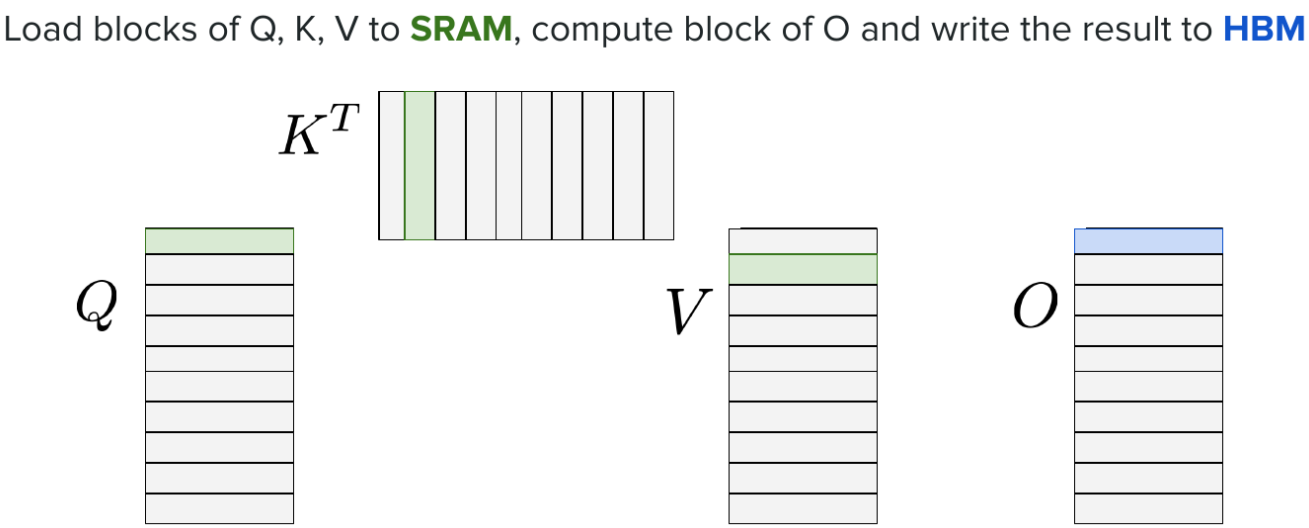


Figure 3: Flash attention: blocks of Q, K, V are loaded into fast SRAM, processed there, and only the output block O is written back to slow HBM.

Q: When tiling, do you take a whole row at once, or just a portion of it?
A: You can take a portion – the whole-row framing is mainly for illustrative purposes. In practice you can think of the matrix as a full grid, tiled in both directions, and blocks are multiplied together accordingly.

Q: Are the rescaling factors (e.g. α) computed exactly on the fly, or are they some kind of estimate/approximation?
A: They're computed exactly, in an iterative fashion: as the output is populated block by block, an additional running quantity is maintained and updated to correctly account for the rescaling needed – it's not an approximation. The original flash attention paper works through the exact formula for this in detail, for anyone who wants to verify it directly.

Recomputation

A second idea from the same line of work: since flash attention makes computing activations cheap, it can be *faster overall* to **discard** forward-pass activations and **recompute** them during the backward pass, rather than storing them. Despite doing strictly more floating-point operations (recomputation adds compute), this reduces HBM reads/writes so much (observed roughly a 10× reduction in one example) that both memory *and* runtime improve simultaneously – an unusually favorable trade-off. Follow-up variants (Flash Attention 2, 3) adapt the technique to newer GPU generations.

Quantization and Mixed Precision Training

Model weights are floating-point numbers, encoded with bits allocated to sign, exponent, and **mantissa** (granularity). Common formats include FP64, FP32, FP16, and BF16 (Brain Float 16), each trading off precision for memory footprint and compute speed (e.g. FP32 roughly doubles achievable compute throughput over FP64, and FP16 over FP32). **Quantization** is the process of converting a number's precision from one format to a lower-precision one to save memory and increase speed, ideally with minimal performance loss.

Mixed precision training (Micikevicius et al. 2017, *"Mixed Precision Training"*) combines granularities strategically: weights are kept in high precision (FP32) for numerically stable updates, while forward/backward pass computations run in a lower precision (FP16), and weight *updates* are still applied in FP32. Intuition: forward-pass activations are computed on inherently noisy data batches, so they don't need extreme precision (they mainly indicate an update *direction*), whereas the weights themselves benefit from high precision to avoid accumulating quantization error over many updates. Whether these techniques (and the Chinchilla-style compute/data/parameter trade-offs) apply uniformly across setups is not settled – teams like the Llama 3 authors ran their own small-scale experiments to derive relationships specific to their training setup before scaling up.

Q: Do you apply quantization/mixed precision to all weights of all layers, or only to some?
A: Not necessarily to everything – there's a body of research on exactly this question, and the answer is that some parts of the model tend to be more sensitive (i.e. more important to keep at higher precision) than others. The high-level strategy described here is a reasonable default, but the specifics vary by setup.

Q: Is there an "optimal" precision to target, similar to how Chinchilla gives an optimal token-to-parameter ratio?
A: There isn't one universally agreed-upon answer – different teams use different settings. What some

teams do (e.g. the Llama 3 authors) is run their own smaller-scale experiments to work out relationships specific to their own setup, then extrapolate those findings when scaling up to the full model, rather than relying on a single fixed rule that's assumed to transfer across all setups.

Q: Is there anything you can do about the *range* of values when quantizing?
A: Yes – there are several distinct quantization techniques that address exactly this, such as **zero-point quantization** and **absmax quantization**, which handle how the range of original values gets mapped into the lower-precision representation.

---

2 Part 2: Supervised Fine-Tuning (SFT)

Why Pretraining Alone Isn't Enough

A purely pretrained model is a next-token predictor, not a helpful assistant. Example: asked "can I put my teddy bear in the washer?", a pretrained-only model might continue with plausible-sounding but unhelpful text (e.g. describing what teddy bears are made of), since it has learned to *continue* text, not to *respond helpfully* to it.

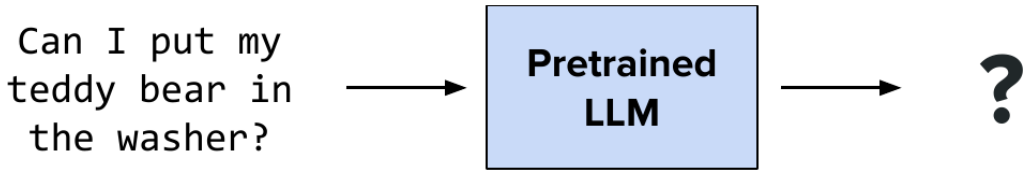


Figure 4: A purely pretrained model has no guarantee of producing a helpful response.

Fine-tuning (specifically **SFT**, supervised fine-tuning) addresses this.

Q: What would a purely pretrained model actually output in response to that question?
A: A good guess: it might just continue with *another question*, or some other plausible-sounding continuation, rather than an answer – since it was trained purely to predict likely next tokens, not to be a helpful assistant. In this case it might drift toward describing what teddy bears are made of, since "washer" and "teddy bear" co-occur in similar contexts (e.g. care instructions) it may have seen in training.

Q: Is that response satisfying as a user?
A: No – and that dissatisfaction is exactly the motivation for the next stage, fine-tuning.

The SFT Objective

SFT uses labeled input/output pairs (hence "supervised"), starting from pretrained weights. Unlike pretraining – where the loss applies to predicting every token in the corpus – in SFT the **input** (e.g. the user's query) is fixed context, *not* something to predict; the loss only applies starting from the **output** tokens the model should generate. The subcategory of SFT that turns a base model into a helpful assistant is called **instruction tuning**.

Q: So we don't put a loss over the input tokens, right?
A: Correct. You start from the input (fixed context), and loss calculation only starts from the point where the model begins predicting the output. The input itself is never something the model is trained to reproduce or "predict."

SFT Data

Unlike pretraining data (raw internet-scale text), SFT data consists of curated instruction/response pairs spanning categories like story writing, poem creation, list generation, explanation, and general assistant dialogues.

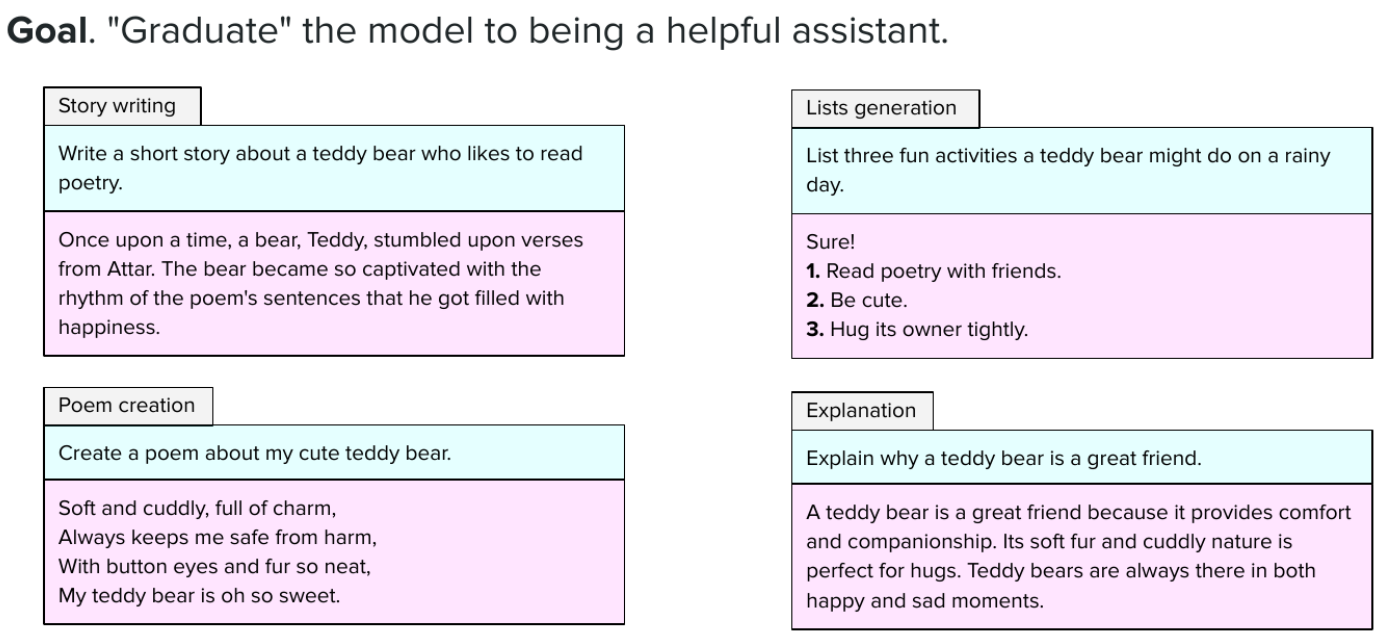


Figure 5: Illustrative SFT (instruction-tuning) examples spanning several task categories.

Historically hand-written by expert linguists, such data can now also be partly **LLM-generated** (sample from a strong existing model, then have a human or another LLM review quality) to speed up curation. SFT data mixtures also typically include **safety**-oriented examples: teaching the model to reject harmful requests (a learned behavior, not a hardcoded regex) and to **hedge** appropriately rather than making unwarranted blanket statements.

Q: If an SFT example (e.g. "write a story") doesn't specify things like which kind of poetry or style to use, how can the model generalize to prompts that do specify those details?
A: This relies on the model's knowledge accumulated during *pretraining* – e.g. it already knows about many kinds of poetry from pretraining data. Fitting the concept of "write a story" during SFT teaches the model the target behavior/format, and it can then generalize that behavior to more specific variants (different genres, styles, constraints) it never explicitly saw paired with that instruction, drawing on what pretraining already gave it. This adaptability to natural language variation is one of the more "magical" aspects of LLMs.

Q: Is this post-pretraining process what's referred to as the model being "aligned"?
A: Getting ahead of the material a bit, but yes – this is covered explicitly in a few slides: the combination of SFT and (later) preference tuning is what's collectively called **alignment**.

Scale

SFT datasets are measured in number of examples rather than tokens, and are orders of magnitude smaller than pretraining corpora (e.g. GPT-3: ∼13K examples; Llama 3: ∼10M examples) – reflecting a shift from "learn general language structure" to "align behavior for the target task," where quality and precision matter more than sheer volume. The subcategory of SFT that specifically "graduates" a model into a helpful assistant is often called **instruction tuning** (Wei et al. 2022, *"Finetuned Language Models Are Zero-Shot Learners"* – the FLAN paper).

After instruction tuning, the teddy-bear-washer example yields a genuinely helpful answer (e.g. recommending hand-washing) – the model now responds to, rather than merely continues, the prompt.

Challenges of SFT

- **Data cost**: high-quality curated data (with human involvement) is expensive and time-consuming to produce, though it can be reused/extended over time.
- **Prompt distribution alignment**: if the SFT data's prompt distribution doesn't closely match the target inference-time distribution (e.g. very different genres or styles from what was seen during fine-tuning), generalization can suffer.
- **Generalization vs. memorization**: with a non-zero sampling temperature, a fine-tuned model won't reproduce training examples verbatim even on the exact same prompt, though it may produce similar "flavor" – data diversity in the SFT mixture is key to encouraging broader generalization rather than narrow repetition.
- **Difficult to evaluate**: covered in detail just below.
- **Computationally expensive**: fine-tuning a full LLM still requires substantial compute, motivating the parameter-efficient methods (LoRA/QLoRA) covered at the end of this lecture.

Q: If you feed the model back one of its own SFT training prompts, will you get back the exact same story?
A: Most likely not word-for-word, because sampling uses a non-zero temperature – the story might have a similar "flavor" (especially if the prompt is very specific, like a particular sub-genre of poetry), but it depends on what else the model saw during pretraining, and for something as creative and open-ended as story generation, a different sample can easily land in an entirely different region of the output space.

Q: Does how much the model "wanders" depend on the temperature parameter?
A: Yes – higher temperature is generally associated with more creative, more varied outputs, precisely because it's more willing to sample tokens that weren't the single most likely choice.

Q: Beyond adjusting temperature, is there another way to control how much variation the model produces for a given kind of prompt?
A: The more robust (if harder) solution is on the **data** side: adding more SFT examples in that category, so the model sees a broader distribution of what's acceptable for that kind of query, rather than relying solely on inference-time temperature tuning to manufacture variety.

Evaluating Fine-Tuned Models

Benchmarks

Standard benchmarks target specific capabilities: general language understanding (**MMLU**, spanning ∼50+ tasks), basic reasoning (**ARC-Challenge**), math reasoning (e.g. **GSM8K**, grade-school math word problems), and code generation (e.g. **HumanEval**), among others. A key caveat, highlighted in Dominguez-Olmedo et al. 2024 (*"Training on the Test Task Confounds Evaluation and Emergence"*), is that comparing models fairly on a benchmark requires knowing whether each model's training mixture specifically included data resembling that benchmark's task domain – otherwise the comparison doesn't isolate genuine capability differences.

Chatbot Arena (LMArena) and Its Limitations

Chatbot Arena (now branded **LMArena**) presents users with blind, paired model responses and collects pairwise preference votes (an A/B test over user prompts) to produce a ranking – essentially a "vibes" score aggregated at scale. Limitations include:

- **Early ranking noise / "cold start"**: a new model's initial, unequal exposure disproportionately influences its eventual ranking.
- **Gameability**: research has shown rankings can be manipulated (Huang et al. 2025, *"Exploring and Mitigating Adversarial Manipulation of Voting-Based Leaderboards"* – e.g. detecting which model is being evaluated from simple self-identifying responses, and selectively engaging).
- **Mismatch with factual quality**: users often cannot accurately judge important aspects such as factuality in a confident, well-formatted answer (unlike expert-curated benchmarks with known ground truth).
- **Stylistic/personal-preference bias**: general users' aggregate preferences (e.g. for or against emoji use) reflect a non-representative distribution that may not match what domain experts or the intended product would prioritize.
- **Safety penalization**: users tend to downvote responses that decline to answer, even when refusal is the intended, correct behavior.

The overall lesson: evaluation is a hard problem in itself – no single number captures "how good" a model is; it requires combining multiple signals suited to the actual use case.

Terminology: Alignment

The combination of SFT and preference tuning (covered next lecture) is collectively referred to as **alignment**. A newer, related stage called **mid-training** has also emerged, sitting between pretraining and fine-tuning: it uses the same next-token-prediction objective as pretraining, but on data curated to better match the tasks ultimately of interest.

Efficient Fine-Tuning: LoRA

Fully fine-tuning all weights is expensive. **LoRA** (Low-Rank Adaptation; Hu et al. 2021, *"LoRA: Low-Rank Adaptation of Large Language Models"*) instead freezes the pretrained weight matrix W_0 and learns a low-rank update:

$$W = W_0 + BA$$

where W_0 is frozen, and B (dimensions matching W_0's rows × rank r) and A (rank r × W_0's columns) are the only trained parameters. Since r (typically $O(10)$) is much smaller than W_0's dimensions

(typically hundreds to thousands), this results in dramatically fewer trainable parameters.

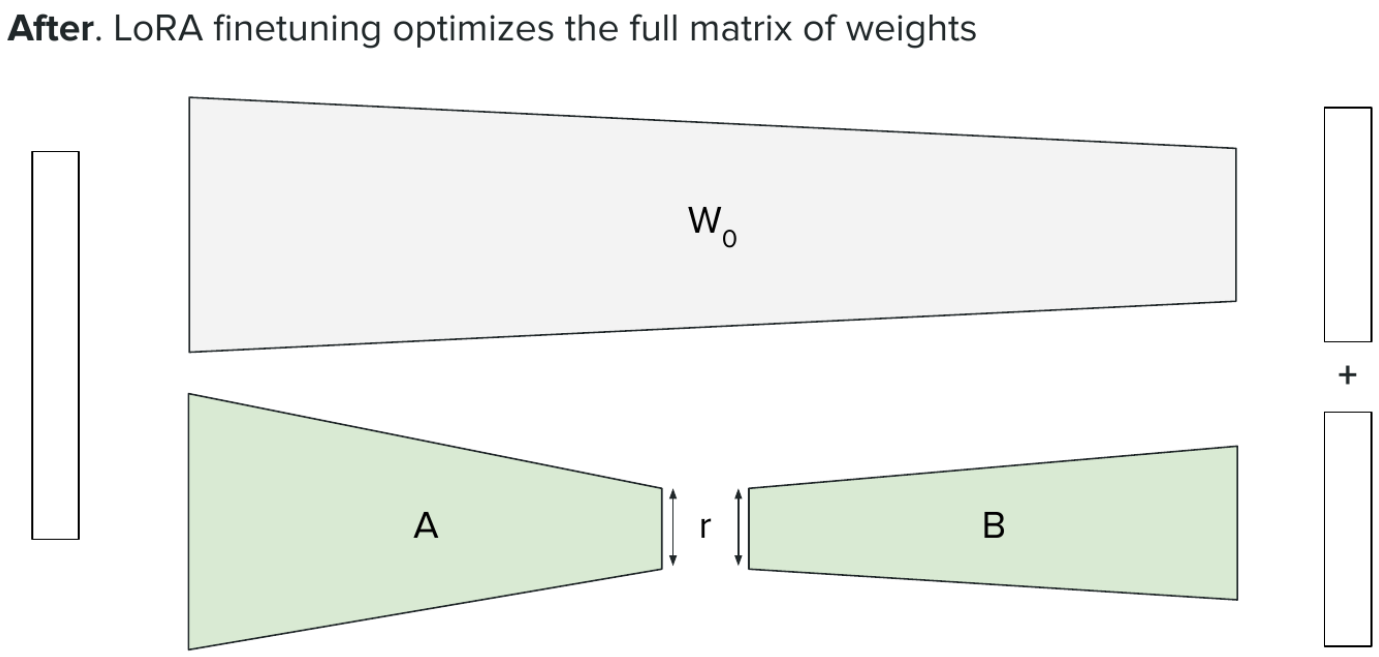


Figure 6: LoRA: the frozen pretrained matrix W_0 (top) plus a trainable low-rank update BA of rank $r \ll \dim(W_0)$ (bottom).

Different tasks (e.g. spam detection vs. sentiment extraction) can each have their own small, task-specific A/B pair layered on the same frozen base model.

Where to Apply LoRA

The original LoRA paper applied it only to attention matrices; more recent guidance (Schulman et al. 2025, *"LoRA Without Regret"*) instead identifies the **feedforward (FFN)** blocks as the most important location. Modern practice often applies LoRA to both, with most of the performance gain attributable to the FFN component. (Other parameter-efficient alternatives include **prefix tuning** and **adapters**.)

Practical Notes

- LoRA fine-tuning typically benefits from a **higher learning rate** (e.g. $\sim$10$\times$ higher than full fine-tuning) – an empirical finding (Schulman et al. 2025, *"LoRA Without Regret"*), plausibly because the low-rank parameterization constrains the region of weight space being explored.
- LoRA can perform worse with **larger batch sizes** – also an empirical finding from the same paper, plausibly due to different training dynamics for a low-rank factorized update versus a full-rank one.
- Rank r is a design choice/hyperparameter (a value of 4 is common); the initial reduction in trainable parameters from using LoRA at all tends to dominate further gains from tuning r precisely.

Q: Do you grid-search the rank r?
A: You could, but in practice a lot of the work of figuring out a good rank has typically already been done by others, so a popular value (e.g. 4) is often just adopted directly. Since the jump from full fine-tuning to LoRA already reduces trainable parameters by orders of magnitude, further gains from

precisely tuning r tend to be comparatively small – so treating r for a given setup as more of a design choice than a make-or-break hyperparameter is reasonable.

QLoRA: Combining LoRA with Quantization

QLoRA (Dettmers et al. 2023, *"QLoRA: Efficient Finetuning of Quantized LLMs"*) quantizes the frozen base weights W_0 into a compact format called **NF4** (a 4-bit format assuming normally-distributed weights, using quantile-based bucketing rather than fixed-size buckets, so each bucket holds roughly equal numbers of values), while the trainable LoRA matrices A, B remain in full precision.

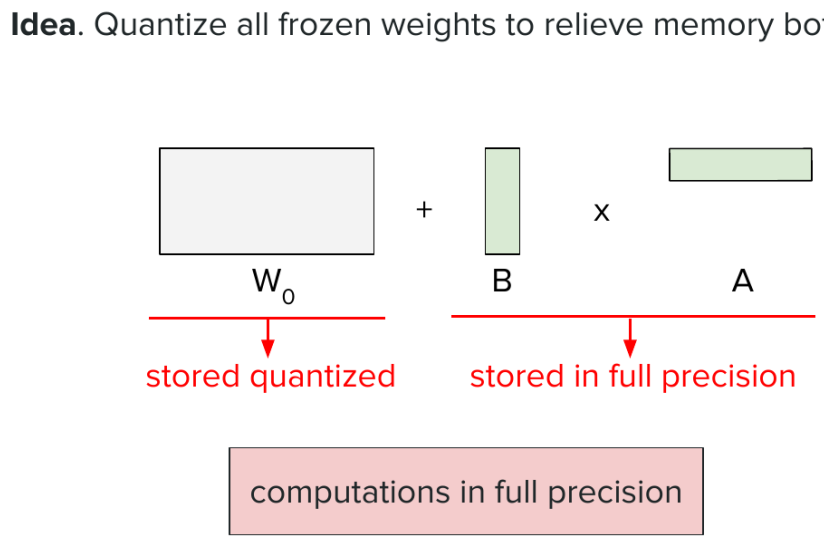


Figure 7: QLoRA: the frozen matrix W_0 is stored quantized, while the trainable LoRA matrices A, B stay in full precision; all computations run in full precision.

A second refinement, **double quantization**, additionally quantizes the small quantization-constant values generated during the (de)quantization process itself. On LLaMA 65B, QLoRA was reported to yield roughly **16× VRAM savings** during fine-tuning from NF4 quantization, with double quantization providing a further ∼6% savings on top – together enabling fine-tuning on much smaller GPUs, at a better trade-off between memory resources and quality, and often faster as well.

Summary

This lecture covered the two ends of the LLM training pipeline discussed so far. **Pretraining**: the compute/data/parameter scaling relationships (Chinchilla's 20:1 token-to-parameter rule), and the systems techniques that make training at this scale feasible – data parallelism and ZeRO sharding, model parallelism, flash attention's memory-hierarchy-aware exact computation (plus activation recomputation), and mixed-precision/quantized training. **Supervised fine-tuning**: how it differs from pretraining (loss only on outputs, much smaller but higher-quality data), why evaluation is hard (benchmark contamination concerns, Chatbot Arena's biases), and **LoRA**/**QLoRA** as parameter- and memory-efficient fine-tuning techniques.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 5: Preference Tuning – RLHF, PPO, and DPO

Afshine Amidi, Shervine Amidi

Recap of Lecture 4

Lecture 4 covered the first two training stages: **pretraining** (teaching the model language/code structure via next-token prediction over massive data, with systems techniques – data parallelism, ZeRO, model parallelism – to make this tractable), and **SFT** (supervised fine-tuning), which adapts a pretrained model to a specific behavior (e.g. being a helpful assistant) using a much smaller, higher-quality curated dataset, with the loss applied only to the desired output tokens. We also covered **LoRA**, a parameter-efficient fine-tuning method that freezes the pretrained weights and learns a small low-rank update. Today: the third stage, **preference tuning** – aligning a fine-tuned model with human preferences (e.g. tone, safety, friendliness) beyond what SFT alone achieves.

Why a Third Stage?

Consider an SFT model asked to "suggest a new activity I could do with my teddy bear," which responds "I would suggest you to not spend much time with your teddy bear at all." This is a plausible assistant-style response, but not one we want. A **preference pair** captures this: the same prompt paired with both the undesired output and a desired rewrite (e.g. suggesting fun activities with the teddy bear).

Why not just add more/better examples to the SFT data instead of introducing a new stage?

- **Comparison is easier than generation**: rating which of two responses is better is much easier to collect at scale than writing a great response from scratch (e.g. ranking two poems vs. writing an excellent poem).
- **Prompt-distribution risk**: adding new SFT examples requires care about not skewing the model's response distribution toward the specific prompt patterns being patched.
- **SFT data must stay very high quality**: patching every observed misstep directly into the SFT set is costly and can degrade the dataset's quality; sometimes model misbehavior is actually a signal that the *existing* SFT data has issues, and preference tuning is not a substitute for fixing that.
- **Negative signal**: SFT only teaches the model what *to* generate; it cannot teach the model what *not* to generate short of providing a hand-corrected replacement for every bad case. Preference tuning explicitly injects this negative signal.

Q: We saw LoRA as a parameter-efficient method for SFT – is there an equivalent for preference tuning?

A: LoRA is about *how many* parameters you tune (reducing that count via a low-rank update); preference tuning is more about using a *different objective function* to train the model. These two ideas are not incompatible – you could very well use LoRA *together with* a preference-tuning objective. This will become clearer once the specific loss functions are introduced below.

---

1 Collecting Preference Data

Given a prompt and candidate response(s), preference data can be collected in three ways:

- **Pointwise**: assign an absolute score to a single response. Hard for humans to calibrate consistently (what does a "0.7" mean?).
- **Pairwise**: given two responses, say which is better. Much easier for humans (and models) to do reliably.
- **Listwise**: rank a list of n responses. More informative than pairwise, but harder to produce than pairwise.

Pairwise is the most commonly used in practice.

Generating Pairwise Data

Given a prompt x (drawn from logs or a target prompt distribution), two responses can be sampled from the model using a positive temperature (yielding two different completions). The pair is then compared using human ratings, **LLM-as-a-judge** (covered in a later lecture), or rule-based metrics (e.g. BLEU/ROUGE, less common today). A simple **binary** comparison (better/worse) is more common than finer-grained scales (e.g. "much better," "slightly better"), since finer distinctions tend to be subjective and noisy. Alternatively, a bad logged response can be directly **rewritten** into a good one to form a pair – more labor-intensive, but directly targeted.

2 RLHF: Reinforcement Learning from Human Feedback

RLHF was popularized for LLM alignment in Ouyang et al. 2022, *"Training Language Models to Follow Instructions with Human Feedback"* (the InstructGPT paper).

RL Basics, Mapped to LLMs

In standard RL, an **agent** in a **state** takes an **action** according to a **policy** $\pi_\theta(a \mid s)$, and receives a **reward**. Mapped to LLMs:

- **Agent** = the LLM.
- **State** = the input/context so far.
- **Action** = predicting the next token.
- **Environment** = the vocabulary of possible tokens.
- **Policy** = the LLM's output probability distribution over the next token (from a forward pass).

The goal is to learn θ such that π_θ aligns with human preferences, using the reward signal derived from preference data.

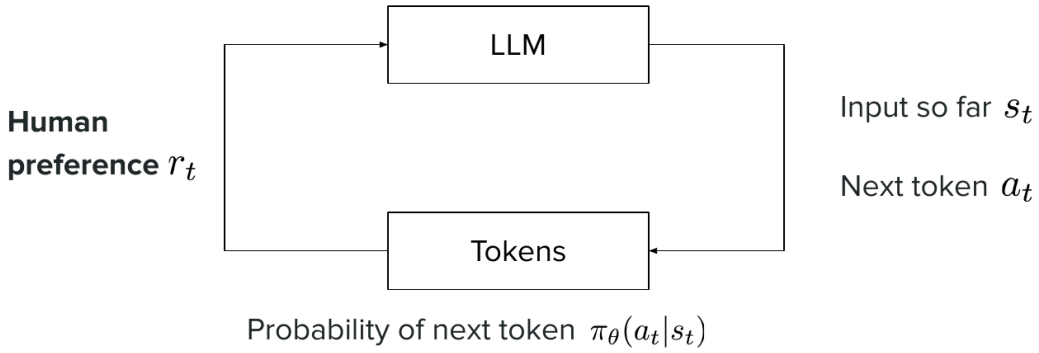


Figure 1: The RL loop mapped onto an LLM: the policy $\pi_\theta(a_t|s_t)$ predicts the next token, and human preference supplies the reward.

Q: Concretely, what are "state" (s_t) and "action" (a_t) in this LLM mapping?
A: s_t is the state you're in – for an LLM, that's the input received so far. a_t is the action you take – which token to generate next, given that input.

Q: Wouldn't computing rewards for every pair be expensive?
A: Yes, this is a genuinely expensive training procedure – in practice people work in batches, similar to other training setups. There's nothing intrinsically *more* expensive about the reward computation itself, but the overall RL pipeline does add real cost compared to SFT alone.

Q: Is the reward signal from RLHF actually strong/informative enough to meaningfully change the LLM?
A: It's a real concern – RLHF is often described as having a sparser signal than SFT. In SFT, every partial input directly teaches the model what token to generate next (a signal at essentially every token). In RLHF, you typically get roughly one signal *per completion* (the reward for a full generated rollout), so it is comparatively sparse, and this is a general property people flag about RL-based approaches for LLMs.

Q: Is the reward applied per token, or to the whole completion?
A: For the whole thing, at least at this level of description – though as we'll see, the value function used in PPO does provide a token-level estimate that feeds into the advantage computation.

The Two Stages of RLHF

(1) **Reward modeling**: train a model that, given a prompt and response, outputs a score reflecting how good it is. Input: prompt + response; output: a scalar score.

(2) **RL stage**: use the reward model to tune the policy (the LLM) toward higher-reward outputs.

(The "Human Feedback" in RLHF refers to the human-labeled preference pairs used to train the reward model; using AI-generated labels instead is sometimes called RLAIF.)

Stage 1: Training the Reward Model

We want a reward model r such that a preferred ("winning") response scores higher than a dispreferred ("losing") one. This uses the **Bradley-Terry** formulation (Bradley & Terry 1952, *"Rank Analysis of*

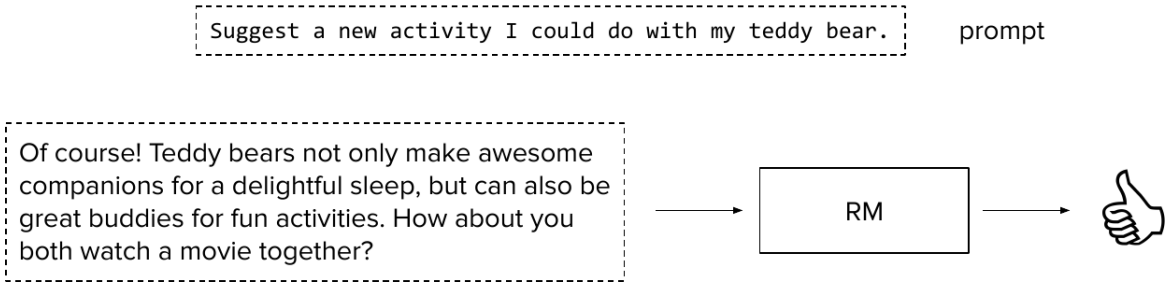


Figure 2: Step 1: the reward model (RM) scores a (prompt, response) pair.

Incomplete Block Designs: I. The Method of Paired Comparisons" – originally a statistics paper on ranking from pairwise comparisons, long predating its LLM application):

$$P(y_i \succ y_j) = \frac{\exp(r_i)}{\exp(r_i) + \exp(r_j)} = \sigma(r_i - r_j)$$

where σ is the sigmoid function, $\sigma(x) = 1/(1 + e^{-x})$. To make $P(y_i \succ y_j) \to 1$, we want $r_i \gg r_j$ when i is preferred.

Given a preference dataset of (winning, losing) pairs $(x, \hat{y}_w, \hat{y}_l)$ assumed independent, maximizing the likelihood of the observed preferences (via Bradley-Terry) and taking the negative log yields the training loss:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}\Big[\log \sigma\big(r(x, \hat{y}_w) - r(x, \hat{y}_l)\big)\Big]$$

This is a form of **binary cross-entropy**. Although trained in a **pairwise** fashion, the resulting reward model is used **pointwise** at inference: it scores a single (prompt, response) pair at a time. Reward scores are typically **normalized/rescaled** (e.g. across a batch) rather than interpreted on an absolute scale.

Q: Where does that binary cross-entropy-style loss actually come from?
A: It falls out of maximum likelihood estimation: given a preference dataset of independent (winning, losing) pairs, you want to find parameters that maximize the probability of the *observed* preferences under the Bradley-Terry model – i.e. maximize the product, over all pairs, of $\sigma(r_w - r_l)$. Taking the log of a product of probabilities turns it into a sum of logs (numerically much safer, since a raw product of many probabilities can underflow toward zero), and since optimization is conventionally framed as *minimizing* a loss, you negate the sum – giving exactly $-\mathbb{E}[\log \sigma(r_w - r_l)]$, which is precisely the cross-entropy loss form.

Q: Is the reward model itself pairwise or pointwise – does it need a pair of responses to make a prediction?
A: That's the interesting part: it's *trained* pairwise (via the loss above, which compares two rewards at a time), but the resulting model is *pointwise* – at inference, it takes a single prompt and a single response and outputs one score. You don't need a pair to get a prediction out of it.

Practical Notes

Reward models are typically built from an existing decoder-only LLM with a classification head attached (or, historically, an encoder-only model like BERT with a head on the CLS token). Datasets are typically on the order of 10,000 preference pairs, labeled by human raters (the "HF" in RLHF). **RewardBench** (Lambert et al. 2024, *"RewardBench: Evaluating Reward Models for Language Modeling"*) is a common benchmark for evaluating reward model quality. Reward models are often trained per **dimension** of interest (e.g. usefulness, safety, friendliness) rather than a single monolithic score, and human-rating guidelines must be kept as objective as possible to reduce label noise.

Q: If different tasks care about different things (usefulness in one case, friendliness in another), how does that map onto reward models?
A: Rewards are typically defined with respect to a given **dimension** – is the output useful? Is it friendly? Is it safe? – and you can train a separate reward model per dimension, or combine them into a more holistic score. Either way, you need to explicitly define which dimension(s) you're quantifying.

Q: Is training a reward model a regression task or a classification task?
A: It's a bit of both in flavor: the reward itself can be interpreted as a continuous score, but the loss is framed probabilistically (via Bradley-Terry/binary cross-entropy) over binary preference labels (better/worse), so it's best characterized as a probabilistic formulation rather than a pure regression or pure classification task.

Q: Do you normalize the reward score?
A: Typically yes – there are various ways to do this. Since the reward isn't produced on some fixed, pre-defined scale (it's free-form, unlike a regression task with a known target range), some rescaling (e.g. across a batch) is common in practice.

Q: Concretely, what does the output of a reward model look like?
A: Think of it on a continuous scale where good outputs might land around, say, 1, and bad outputs around -2 or -3 – there's no fixed universal range; it's whatever the model learns, typically rescaled/normalized downstream as needed.

Stage 2: The RL Stage

General recipe: the policy (LLM) generates a **rollout** (completion) for a prompt; the (frozen) reward model scores the (prompt, rollout) pair; that reward is used to update the policy's weights. The reward model is **not** trained further here – only the policy is.

Why Not Deviate Too Far from the Base Model?

The objective is to maximize reward *while* staying close to the initial (SFT) model, for several reasons:

- **Catastrophic forgetting**: the base model already encodes substantial useful knowledge; drifting too far risks losing it.
- **Reward hacking**: the reward model is an imperfect proxy for what we actually want. Over-optimizing against it can maximize the score without fulfilling the true objective – analogous to optimizing a lecture for applause volume rather than actual informativeness, which could inadvertently reward jokes over substance.
- **Training instability**: large, unconstrained policy updates can destabilize training.

RL-stage datasets are typically larger than reward-model datasets (100K+ observations), with labels effectively supplied by the reward model rather than direct human ratings.

Q: Why not just run SFT directly on the preference data, instead of setting up this whole RL pipeline?
A: SFT can teach the model "given this input, generate this" – but it has no natural way to teach "given this input, do *not* generate this" unless you rewrite the bad response into a perfect one and train on that instead (which is exactly what happens for the SFT data itself). Preference tuning is specifically about injecting that negative signal – "I prefer this over that" – which is what makes RL (or RL-derived methods like DPO, covered later) a more natural fit than plain SFT for this stage.

PPO: Proximal Policy Optimization

PPO (Schulman et al. 2017, *"Proximal Policy Optimization Algorithms"*) is the standard RLHF algorithm; "proximal" reflects the goal of not deviating too far from a reference point.

PPO = Proximal Policy Optimization

$$\mathscr{L}(\theta) = -\Big[r(x,\widehat{y}) - \lambda \mathrm{KL}(\pi_\theta(\widehat{y}|x)||\pi_{\text{ref}}\,(\widehat{y}|x))\Big]$$

Figure 3: The PPO-KL objective: maximize reward $r(x, \hat{y})$ while penalizing KL divergence from the reference policy π_{ref}.

Advantage, Not Raw Reward

Rather than directly maximizing reward, PPO maximizes an **advantage** – how much better an output is than what would be *expected* on average – which reduces gradient variance and speeds up training. The advantage is estimated using a learned **value function**: a token-level estimate of the expected future reward if generation continued to follow the current policy from that point. This value function is trained **jointly** with the policy (as a regression head), and advantages are computed via **Generalized Advantage Estimation (GAE)** (Schulman et al. 2015, *"High-Dimensional Continuous Control Using Generalized Advantage Estimation"*), beyond this course's scope in full mathematical detail.

PPO-Clip

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\big(R(\theta)\,A,\ \text{clip}(R(\theta), 1-\epsilon, 1+\epsilon)\,A\big)\Big]$$

where A is the advantage, and

$$R(\theta) = \frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)}$$

is the probability **ratio** between the current policy and the policy at the previous RL iteration (not the reward, despite the letter R – a notoriously confusing notational choice). L denotes a quantity to be *maximized*, despite the "loss" naming convention. Intuition:

- If $A > 0$ (the output should be reinforced), increasing $R(\theta)$ increases L, but the clip caps this increase beyond $1 + \epsilon$, preventing overly large updates.

- If $A < 0$ (the output should be discouraged), decreasing $R(\theta)$ increases L, but the clip caps this decrease below $1 - \epsilon$, again bounding the update size.

This clipping mechanism keeps policy updates within a controlled trust region from one iteration to the next.

Q: Outside the clipped region, does the objective just look like a ReLU?
A: Essentially yes – the shape is a linear function of $R(\theta)$ up to the clip boundary, then flat beyond it, which is the same qualitative shape as a ReLU (or its mirror image, for the $A < 0$ case).

Q: Concretely, how do you compute $R(\theta)$?
A: It's the probability of the token that was actually generated, according to the current policy, divided by that same probability under the previous-iteration policy – both read directly off the probability distribution output by the LLM (post-softmax) for that token, given the input so far.

PPO-KL Penalty

An alternative/complementary formulation directly penalizes the **KL divergence** between the current and reference policy:

$$L^{\mathrm{KL}}(\theta) = \mathbb{E}\big[R(\theta)\, A\big] - \beta \cdot \mathrm{KL}(\pi_{\mathrm{ref}} \,\|\, \pi_\theta)$$

Recall the KL divergence between distributions P, Q:

$$\mathrm{KL}(P \,\|\, Q) = \sum_i p_i \log \frac{p_i}{q_i} \geq 0$$

(non-negativity follows from Jensen's inequality; equality holds iff $P = Q$). In the original 2017 PPO paper, the "old" policy referred to the previous RL iteration; in modern LLM RLHF practice, this KL term is typically computed against the **reference (SFT) model** instead, and implementations often combine both the clipping and KL-penalty mechanisms.

Q: Why use the reference/base model here rather than the previous RL iteration?
A: Because these serve two different purposes: the clipping mechanism (against the *previous iteration*) keeps updates from being too extreme from one training step to the next, while the KL penalty (against the *base/reference model*) keeps the policy from drifting too far from where it started overall. In modern practice, both constraints are typically used together rather than picking one.

Challenges of PPO/RLHF, and Alternatives

- **Multi-stage dependency**: reward model issues discovered after RL training require redoing both stages.
- **Many hyperparameters**: e.g. β (KL weight), ϵ (clip range), and GAE's own hyperparameters.
- **Training instability**, requiring careful constraint of update sizes.
- **Harder monitoring**: unlike pretraining/SFT's clean cross-entropy loss, average reward is a less direct/reliable training-health signal.

- **Need for exploration**: generations must remain diverse enough during training to actually discover better/worse completions to learn from.
- **Many models required**: the policy, the value function, the (frozen) reward model, and the (frozen) reference/base model – four models to keep in memory, raising the question of whether this cost is really worth it.

This has motivated a number of PPO alternatives that avoid needing all four models – e.g. **REINFORCE**-style methods and **GRPO** (Shao et al. 2024, *"DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models"* – covered in detail next lecture), among others. This is an example of **on-policy** training: the policy generates its own rollouts, which are then used to update itself – in contrast to **off-policy** training (like SFT), which trains on fixed data not generated by the current model.

3 Best-of-N (BoN)

For teams with a reward model but wanting to avoid full RL training: generate N completions for a prompt (with positive temperature), score each with the reward model, and return the top-scoring one at inference time.

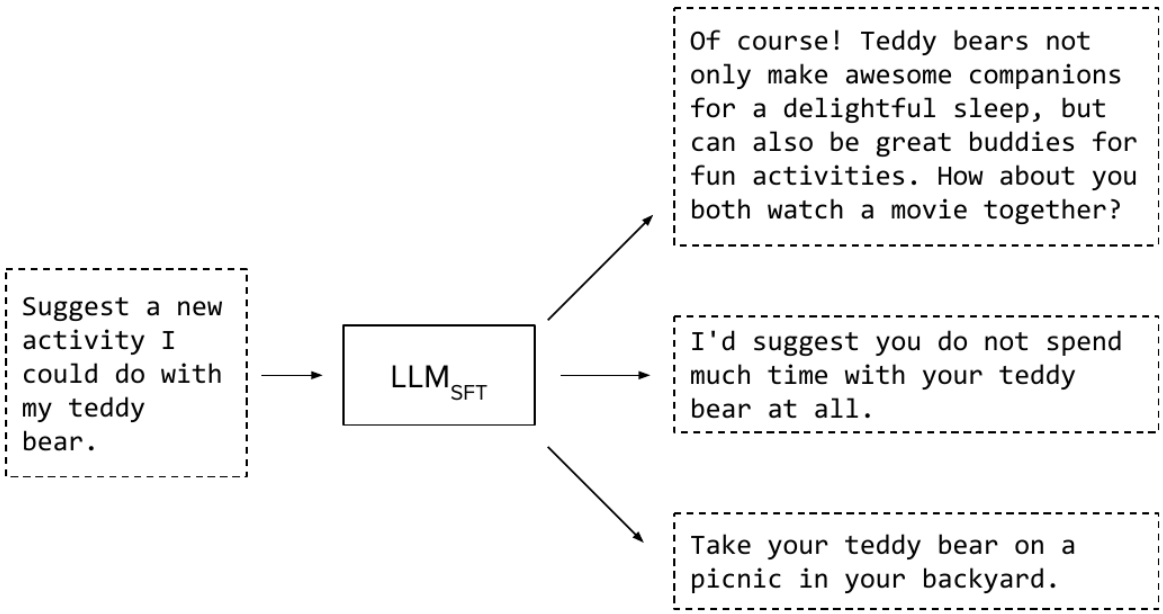


Figure 4: Best-of-N: sample multiple completions, score each, keep the best.

Drawbacks:

- If the underlying model rarely produces good completions, more sampling may not help enough.
- **Inference cost**: querying the model N times per request is expensive, especially at scale/high traffic – this approach shifts cost from training-time to inference-time.
- **Latency**: even generating all N completions in parallel, total latency is bounded by the *slowest* of the N generations; since latency distributions are typically right-skewed, the expected maximum over N draws shifts noticeably later than a single draw's expected latency.

Q: If the underlying model is bad, doesn't Best-of-N just return the least-bad of several bad options?
A: That's a fair concern – although if you generate enough completions with a high-enough temperature,

you get some diversity, which helps. But it's true that Best-of-N can't manufacture quality the base model fundamentally doesn't have; the model needs to be at least somewhat capable to start with.

Q: Doesn't querying the model N times per request make this expensive?
A: Yes – that's the main drawback. You skip RL training, but you push that cost onto *inference* instead, completing the prompt multiple times for every single request, which adds up quickly under real traffic. Whether this is worth it depends on your inference traffic volume versus what training-time RL would have cost.

Q: Is the reward model here using the sigmoid formulation from Bradley-Terry?
A: No – Best-of-N assumes the reward model is already trained (using that pairwise, sigmoid-based formulation during *its own* training). At Best-of-N *selection* time, you're just using the trained reward model to score each of the N completions independently and picking the highest score – no sigmoid/pairwise comparison happens at this stage.

Q: What if all N generated responses are bad?
A: That's a real and fair concern – Best-of-N can only select among what was actually generated, so if none of the N samples are good, the method has no way to produce a good output.

Q: Does the scale of the reward scores matter for Best-of-N?
A: Not really – since you're just picking the highest-scoring completion, any monotonic rescaling of the scores (e.g. normalizing them) won't change which one ends up on top. Scale matters much more when the reward feeds into a *loss function* for training (as in RL), which is not the case here.

---

4 DPO: Direct Preference Optimization

Motivated by RLHF's complexity (four models, unstable multi-stage training), **DPO** (Rafailov et al. 2023, *"Direct Preference Optimization: Your Language Model Is Secretly a Reward Model"*) replaces the two-stage reward-model-then-RL process with a single supervised loss directly over preference pairs:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(\hat{y}_w \mid x)}{\pi_{\text{ref}}(\hat{y}_w \mid x)} - \beta \log \frac{\pi_\theta(\hat{y}_l \mid x)}{\pi_{\text{ref}}(\hat{y}_l \mid x)}\right)\right]$$

Notably, no explicit reward model appears – reflecting the paper's core insight (captured in its title) that the implicit reward is expressed directly as a function of the policy itself.

Derivation Sketch

Starting from the PPO-style objective (maximize reward subject to a KL penalty against the reference model, weighted by β), one can solve for the **optimal policy** π^* in closed form as a function of the reward r and a normalizing **partition function** $Z(x)$. Rearranging this expression lets r be written as a function of π^* instead. Substituting this expression for r into the Bradley-Terry formulation (used for the reward model) eliminates r entirely, leaving a loss purely in terms of the policy π_θ, the reference policy π_{ref}, and β – exactly the DPO loss above. Turning this probability-maximization into a loss follows the same negative-log-likelihood step used for the original reward model. β is a hyperparameter (often around 0.1) controlling how strongly the policy is anchored to the reference model.

DPO vs. RLHF

- **Models needed**: DPO needs only π_θ (trained) and π_{ref} (frozen) – two models, vs. RLHF's four (policy, value function, reward model, reference model).

- **Simplicity**: DPO is a single supervised loss over fixed preference pairs (like SFT), avoiding on-policy generation and RL training instability.
- **Performance**: empirically, PPO tends to outperform DPO, but often at much greater tuning/compute cost – with no absolute consensus on the gap, since it varies by task and is sensitive to implementation details (Xu et al. 2024, *"Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study"*). DPO's simplicity comes at the cost of a **distribution shift** risk: since DPO fits directly on static preference pairs rather than the model's own on-policy generations, the fitted distribution may not perfectly match what the model would generate at inference time (a challenge intrinsic to the DPO formulation, sometimes mitigated by also running SFT on the preference data, at additional cost).

Choosing between them is a trade-off: DPO for a simpler pipeline with good (if not top) results; PPO for maximum performance when the team has the expertise and budget to tune RL training carefully.

Q: Is θ being optimized with respect to the base/reference model, or something else?
A: With respect to the *current* trainable policy, not the base model directly – there are two distinct sets of weights: π_{ref} stays fixed throughout (it only appears inside the loss, as the anchor being compared against) while π_θ is what's actually being updated at each step, starting from a copy of the same weights as π_{ref}.

Q: Instead of using the SFT model as the reference, could you use a different (e.g. larger) model?
A: That's an interesting direction, but it would effectively be a different algorithm, and it's not obvious it would behave sensibly: the whole point of preference tuning is to align the responses the model *already* produces (from its own learned distribution) with what's preferred, so starting the reference from a different model's distribution could undermine that loss's intended meaning. The natural choice remains starting from – and preference-tuning – the same checkpoint's own distribution, though this exact variant hasn't been definitively ruled out as potentially useful.

Q: In practice, what are the trade-offs, and which of PPO/DPO is actually used?
A: It depends on compute budget and how much appetite there is for babysitting a harder-to-tune training process. RL (PPO) is difficult to get exactly right, but tends to give the best results; DPO is much easier to get running and can get close to PPO's results with far less effort. If you want a quick, solid preference-tuning setup, DPO is the friendlier choice; if you're an RL expert chasing the last bit of performance and have the budget for it, PPO is likely the better bet.

Revisiting the Teddy Bear Example

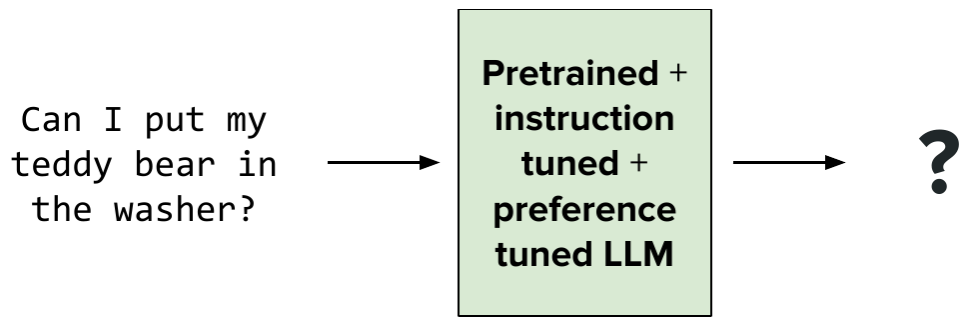


Figure 5: The fully aligned model: pretrained, then instruction-tuned, then preference-tuned.

Recall the instruction-tuned model's answer to "can I put my teddy bear in the washer?": "No, it might get damaged. Try hand washing it instead." Factually correct, but blunt/unfriendly in tone. After preference tuning, a response like "It's better not to – your teddy bear could get hurt. A gentle hand wash is safer" conveys the same facts more gently – exactly the kind of behavioral/tonal alignment preference tuning targets, distinct from teaching new facts.

Summary

This lecture covered the third LLM training stage, **preference tuning**: why it's needed beyond SFT (negative signal, easier data collection, avoiding SFT-data distribution risk), how preference data is collected (pairwise comparisons), and two families of methods to use it – **RLHF** (reward model via the Bradley-Terry formulation, then PPO-based RL with clipping and/or KL-penalty mechanisms to stay close to the reference model), and **DPO** (a simpler, directly supervised alternative derived from the same underlying objective, trading some performance for far less training complexity), alongside **Best-of-N** as an inference-time alternative to training-time preference optimization.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 6: LLM Reasoning – GRPO and the DeepSeek R1 Pipeline

Afshine Amidi, Shervine Amidi

Recap of Lecture 5

Lecture 5 covered **preference tuning**, framed as reinforcement learning: the LLM as agent/policy, the input-so-far as state, next-token prediction as action, and human preferences as reward. We covered **RLHF**'s two stages (reward modeling via Bradley-Terry, then RL-based policy tuning) and **PPO**, whose loss combines an **advantage**-maximization term (with a baseline to reduce variance) and a term discouraging the policy from deviating too far from the previous iteration *and* from the base (SFT) model – via clipping (**PPO-Clip**, using the ratio $R(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$) and/or a **KL-divergence penalty** (**PPO-KL**), typically applied against the SFT reference model in modern practice. Today: **reasoning models** – a major 2024–2025 development that reuses these RL foundations.

Strengths and Weaknesses of "Vanilla" LLMs

The vanilla LLMs discussed so far – prompt in, direct response out – excel at tasks like debugging, code generation, and creative writing. But they have notable weaknesses:

- **Limited reasoning**: sophisticated multi-step problems (e.g. advanced math) are hard for a model trained purely on next-token prediction without explicit incentive to work through steps.
- **Static knowledge**: pretraining data has a fixed cutoff date, so the model cannot know about events after that date (covered further next lecture, via RAG).
- **No ability to take action**: the model can only respond in text, not directly execute tasks (also covered next lecture, via tool calling/agents).
- **Hard to evaluate**: free-form text generation resists traditional rule-based metrics (covered in Lecture 8).

Today's focus is the first weakness: **reasoning**.

1 What Is a Reasoning Model?

There is no single agreed-upon definition, but a useful working one: **reasoning** is the ability to solve problems (typically math or coding) that require a **multi-step process** – breaking a hard problem into steps, working through them, and arriving at a final answer (analogous to working through a non-trivial exam question). Contrast a knowledge question ("what is the course code of Stanford's transformers and LLM class?" – CME 295, a simple lookup) with a reasoning question ("a bear was born in 2020; how old is it in 2025?" – requires a step of computation).

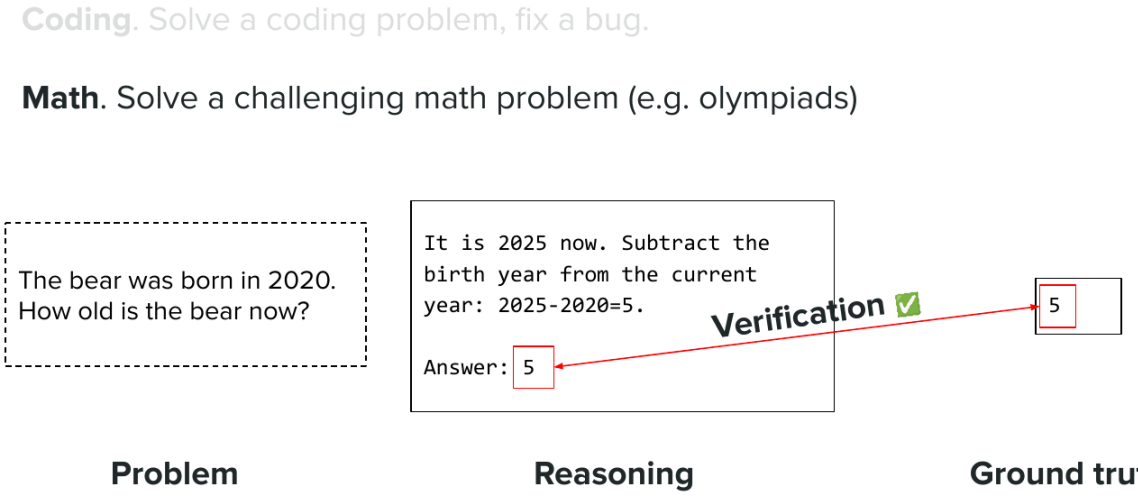


Figure 1: A reasoning benchmark item: a problem, a reasoning chain, and a verifiable ground-truth answer.

Q: For a question like "how old is this bear in 2025," doesn't the model need some external context (e.g. knowing today's actual date) to answer correctly?
A: Good point – there are two parts to this. For something like the current date specifically, LLMs are typically given this as part of a **preamble** (context injected before the user's query), so that particular piece of information is usually already available to the model. For other cases where the needed information genuinely isn't available to the model, that's where techniques like RAG (covered next lecture) come in – fetching external information the model doesn't already have. For the purposes of today's lecture, that's treated more as an extension on top of reasoning models, rather than something foundational to how they work.

From Chain of Thought to Reasoning Models

Recall **chain of thought (CoT)** prompting (Wei et al. 2022, *"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"*; Lecture 3): encouraging step-by-step reasoning before the final answer, originally via in-context examples. **Reasoning models** push this further – "do CoT, but at a much larger scale" – training the model at scale to produce an explicit **reasoning chain** before its answer – not just via prompting, but baked into the model's trained behavior. Two intuitions for why this helps:

- Hard problems are unlikely to appear verbatim in training data; decomposing them into smaller, more tractable sub-problems lets the model draw on patterns it *has* seen.
- Generating more tokens before the answer effectively gives the model more **compute** (each token requires a full forward pass) – sometimes called the model's **compute budget**.

The model's output is therefore **reasoning chain + final answer**, rather than just the answer.

Timeline

Reasoning models are a very recent development: OpenAI's `o1-preview` (September 12, 2024) kicked off the trend, followed by Google's Gemini 2.0 Flash Thinking (December 19, 2024), then DeepSeek's **R1** paper (January 20, 2025) – notable for matching frontier reasoning performance while publishing

its methodology – followed in short order by Grok 3 Beta (February 19, 2025), Claude 3.7 Sonnet (February 24, 2025), and Mistral's Magistral (June 10, 2025), among others.

User-Facing Behavior

Chat interfaces that show a "thinking" indicator are using a reasoning model; the displayed "thought summary" is typically a *summary* of the raw reasoning chain, not the chain itself (raw chains may be less human-legible, users may not want to read lengthy raw reasoning, and providers may wish to avoid directly exposing chains that could otherwise be used to train a competitor's model via distillation). Reasoning tokens are billed as **output tokens**, giving users an incentive to seek maximal reasoning quality per reasoning token spent.

2 Benchmarks for Reasoning

Coding

Given a problem, produce a solution verified by passing **test cases** – an objective, automatic check. Benchmarks include **HumanEval** (human-written problems), **CodeForces** (competitive programming), and **SWE-bench** (derived from real GitHub issues).

Math

Given a problem, produce an answer that can be **parsed** (e.g. via a required output format, such as a boxed final answer) and compared against ground truth. Benchmarks include **AIME** (US math olympiad qualifier) and **GSM8K** (grade-school math).

The pass@k Metric

pass@k (Chen et al. 2021, *"Evaluating Large Language Models Trained on Code"* – the Codex/HumanEval paper) estimates the probability that **at least one** of k sampled attempts succeeds. This matters because in domains with automatically verifiable correctness (like coding/math), if resources allow generating multiple attempts, the actual quantity of interest is whether *any* of them is correct.

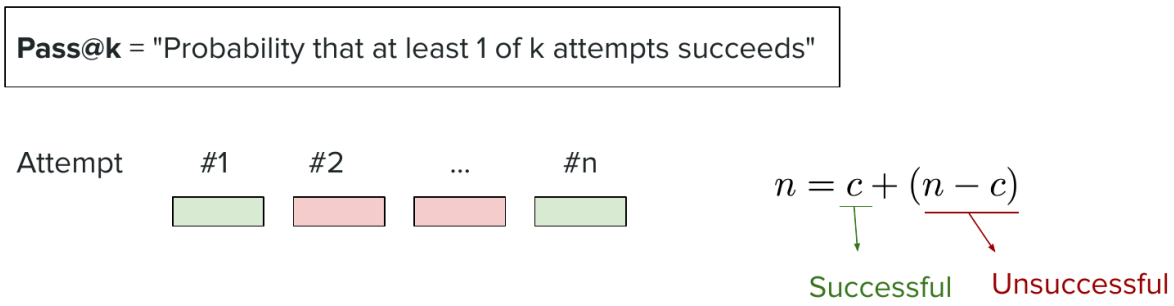


Figure 2: Of n total attempts, c succeed and $n - c$ fail; pass@k asks: what's the probability at least one of k randomly chosen attempts is among the successful ones?

Derivation

Given n total sampled attempts, of which c succeeded, we want the probability that, selecting k of the n at random, **at least one** is correct:

$$\text{pass@}k = 1 - P(\text{all } k \text{ selected are incorrect})$$

Computing the probability of drawing k incorrect attempts without replacement from the $n-c$ incorrect ones:

$$P(\text{all } k \text{ incorrect}) = \frac{n-c}{n} \cdot \frac{n-c-1}{n-1} \cdots \frac{n-c-k+1}{n-k+1} = \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

so:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

The special case $k = 1$ simplifies to pass@1 $= c/n$, the simple proportion of successful attempts.

Temperature and pass@k

Sample diversity (controlled by temperature) matters: at very low temperature, samples are high-quality but nearly identical, so pass@k barely improves with more samples k. At very high temperature, samples become diverse but individually lower-quality, hurting pass@k. An intermediate temperature (e.g. around 0.4–0.8, dataset-dependent) tends to work best, which is why papers explicitly report the temperature used for benchmark results.

Q: Do you choose a particular temperature when computing pass@k?
A: Yes – and it matters quite a bit. Very low temperature gives good but nearly-identical samples, so increasing k barely helps pass@k. Very high temperature gives diverse samples, but individual quality suffers enough to hurt pass@k too. An intermediate temperature (often somewhere around 0.4–0.8, dataset-dependent) tends to work best – which is exactly why papers reporting pass@k results always specify the temperature used.

Q: Why does pass@k make sense as a metric in the first place – why not just look at pass@1?
A: It reflects a realistic use case: if you can afford to generate more than one attempt (e.g. for coding, where correctness can be automatically verified), then what actually matters is whether *any* of your k attempts succeeds – not just whether a single attempt does. This is conceptually similar to Best-of-N from last lecture: if you can spend more compute generating multiple candidates, and you have a cheap way to check correctness, it can be worth doing so to raise your effective success rate.

A related metric, **consensus@k** ("cons@k", DeepSeek-AI 2025, *"DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"*; closely related to the **self-consistency** technique from Lecture 3), selects the answer that appears most frequently among k samples via majority vote and compares *that* consensus answer against ground truth. In practice: pass@k suits use cases where checking correctness is easy and higher latency is acceptable; pass@1 suits use cases where only a single generation is produced and used.

3 Training Reasoning Models

Why RL, Not SFT, for Reasoning Chains

Naively, one might try SFT on human-written reasoning chains. Three arguments favor RL instead:

(1) Writing high-quality, especially long, reasoning chains by hand is very difficult and expensive – and without existing chains to imitate, SFT has nothing to learn from.

(2) The way a model reasons internally may differ from how humans reason, so human-written chains may not be the ideal training signal even if available.

(3) Domains like coding and math have **naturally verifiable rewards** (tests passing; answer matching ground truth) – exactly the ingredient RL needs, without requiring a trained reward model.

The Reward Signal

Two rewards combine to incentivize both reasoning *and* correctness:

- **Format reward**: check for the presence of designated reasoning markers (e.g. `<think>...</think>` tags) indicating a reasoning chain was produced.
- **Correctness reward**: check the final answer against ground truth (tests passing for code; exact match for math).

Applying RL with only these two **verifiable rewards** (no learned reward model needed) has been shown to substantially improve performance on reasoning benchmarks (e.g. AIME) as RL training progresses – this is exactly what DeepSeek's R1-Zero experiment demonstrated (covered below).

Controlling How Much a Model Thinks

Not all prompts warrant equal amounts of reasoning. Open problems and partial solutions include:

- **Dynamic budgets**: e.g. a lightweight classifier estimating how much reasoning a given prompt warrants (Han et al. 2024, *"Token-Budget-Aware LLM Reasoning"*).
- **Context-window awareness / budget forcing**: reasoning consumes context length, requiring the model to balance depth of reasoning against remaining budget; **budget forcing** (Muennighoff et al. 2025, *"s1: Simple Test-Time Scaling"*) inserts tokens (e.g. "wait") mid-generation to force additional reasoning, or forces an answer once time/budget is up.
- **"Continuous" thoughts / latent reasoning**: an active research direction (Hao et al. 2024, *"Training Large Language Models to Reason in a Continuous Latent Space"*) representing reasoning steps as continuous hidden representations rather than discrete language tokens, potentially more compressed and expressive.

4 GRPO: Group Relative Policy Optimization

GRPO (introduced 2024) has become the standard RL algorithm for reasoning training. Like PPO, it maximizes advantage while constraining policy updates to stay close to both the previous iteration and the reference model – but it computes the **advantage differently**.

Key Difference from PPO: No Value Function

PPO estimates advantage using a reward (at the completion level) combined with a jointly-trained **value function** (a token-level baseline estimate), via Generalized Advantage Estimation – an expensive extra model to train. GRPO instead samples **multiple completions** (a **group**) for the same prompt, computes a reward for each, and defines the advantage of each completion **relative to the group's statistics**:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

This requires no value function at all – only sampling G completions and scoring each. Intuition: a correct answer to a hard problem (where most of the group fails) should be more strongly reinforced than a correct answer to an easy problem (where most of the group succeeds) – the group-relative comparison naturally captures this.

Q: Why depart from PPO's value-function approach at all – what's wrong with the traditional RL setup?
A: Traditional RL algorithms like PPO were developed well before LLMs became the dominant use case for RL, so the quantities they rely on (like a learned value function) may make a lot of sense in other RL settings but aren't necessarily the most natural or efficient choice for language. GRPO's designers instead asked: what's a way to tell how good a completion is, purely in a *relative* sense, without the overhead of training and maintaining a separate, expensive joint value model? Sampling several completions for the same prompt and comparing rewards within that group is the answer they landed on – and it's a design choice that specifically reflects what LLMs are good at (cheaply sampling many completions), not a universal RL principle.

Walkthrough

A query is passed through the policy model to generate G completions. Each (query, completion) pair is scored by the reward model (or, in the reasoning case, by the verifiable reward – no trained reward model needed at all), giving G rewards, from which the group-relative advantages are computed as above. These advantages tune the policy, together with a KL-divergence term against the reference model. (In PPO, by contrast, only *one* completion is generated per prompt; its reward, together with a per-token value-function estimate, feeds into GAE to produce the advantage.)

Comparing the Loss Functions

GRPO and PPO share: (1) both operate on the ratio of current-to-old policy probabilities, and (2) both use a clipping mechanism to bound update size, following the same intuition as PPO-Clip (reinforce/discourage an output, but not too abruptly). They differ in: (1) GRPO includes the KL-divergence term **explicitly** in the objective, whereas PPO's KL term is typically folded into the per-token reward used for advantage computation; and (2) how the advantage itself is computed (group-relative statistics vs. reward + value function).

Which Models Are Needed?

Both GRPO and PPO require a frozen **reference model** for the KL term. In the reasoning setting specifically, **no reward model** is needed at all (rewards are verifiable). The key difference in what gets *trained*: GRPO trains only the **policy**; PPO trains both the policy **and** a value function.

Q: What models are actually involved in a GRPO training run – which are frozen and which are trained?
A: The frozen models are the same role as in PPO: a **reference model** used for the KL-divergence term, and (in the general preference-tuning case) a **reward model**. But specifically for reasoning training, there *is no reward model at all* – correctness is verifiable directly (tests passing, answer matching), so that piece simply isn't needed. As for what's actually **trained**: GRPO only trains the policy, whereas PPO trains both the policy and a separate value function – that's the key practical difference in how many models you need to maintain and update.

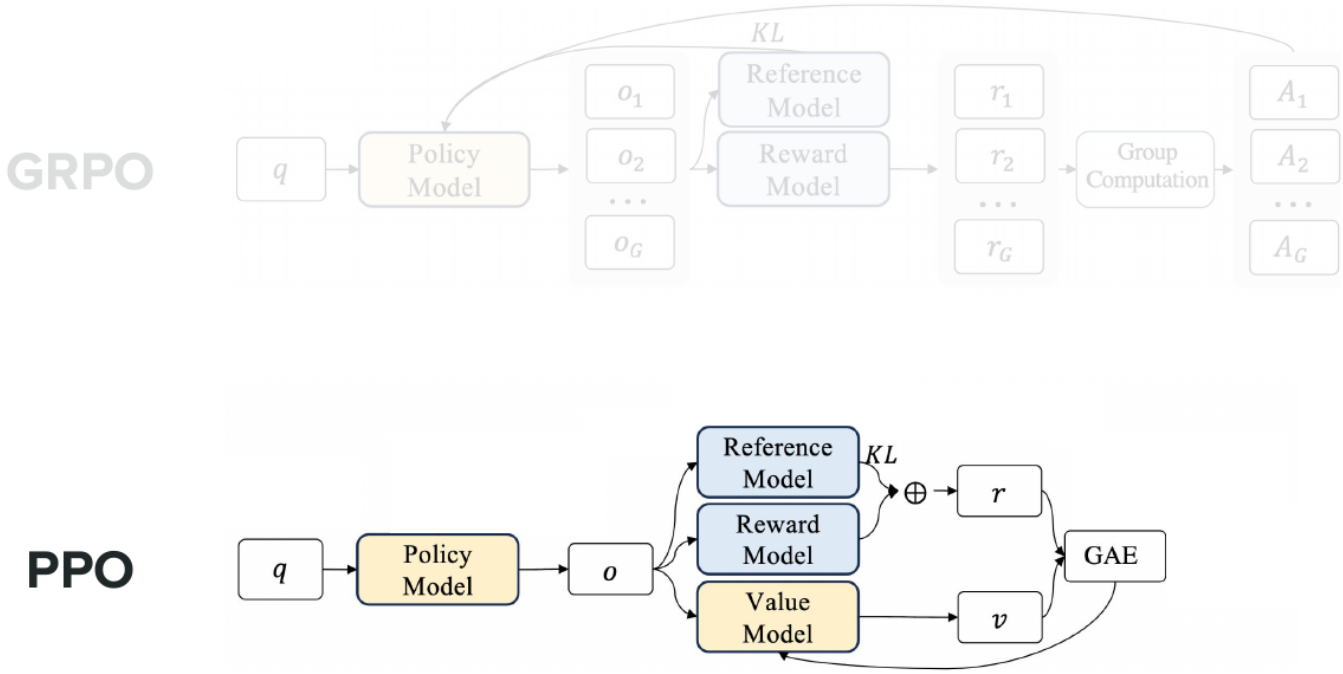


Figure 3: GRPO (top) computes advantage via group-relative reward comparison; PPO (bottom) requires an additional value model and GAE.

5 The Length Bias Problem and Its Fixes

Empirically, as GRPO-based RL training progresses, average output length keeps increasing – initially correlated with improving performance, but the length continues growing even after performance plateaus, indicating a problem.

Diagnosis

The (simplified) GRPO objective includes a per-completion normalization by $1/|o_i|$ (the length of completion i), applied *outside* the token-level sum:

$$J_{\text{GRPO}} \propto \sum_i \frac{1}{|o_i|} \sum_t \left(\text{ratio}_t \cdot A_i - \text{KL term}\right)$$

Because this factor depends only on which completion (short or long) a token belongs to, tokens in **short** completions receive a *larger* per-token weight than tokens in **long** completions. When $A_i < 0$ (an incorrect/bad completion), this means short bad completions get penalized *more heavily* than long bad completions – an unintended incentive for the model to prefer generating long bad outputs over short bad ones, gradually inflating output length regardless of quality.

Fixes

- **DAPO** (Yu et al. 2025, *"DAPO: An Open-Source LLM Reinforcement Learning System at Scale"*): replaces the per-completion length normalization with a single normalization constant shared across all tokens in the batch, equalizing each token's contribution regardless of which completion (short or long) it belongs to.
- **Dr. GRPO** (Liu et al. 2025, *"Understanding R1-Zero-Like Training: A Critical Perspective"* – "GRPO Done Right"): removes the length-normalization term entirely.

Observation. Response length keeps on increasing with RL training

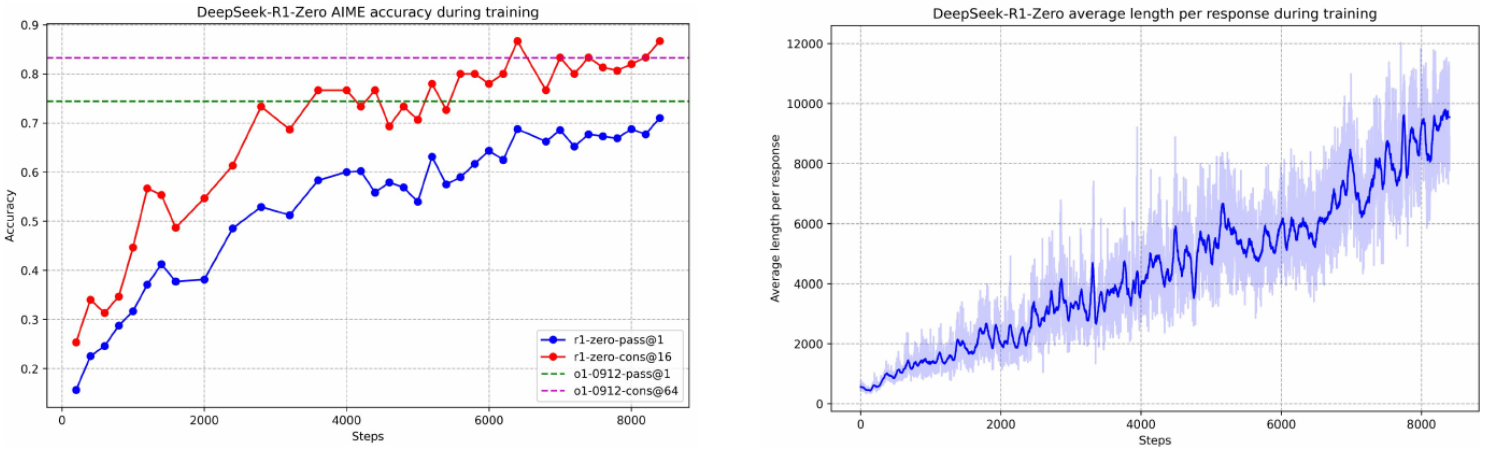


Figure 4: DeepSeek-R1-Zero training: AIME accuracy (left) and average response length (right) both climb over RL training steps.

Empirically, these fixes stop the runaway length growth: correct-completion lengths track the original GRPO behavior, but incorrect-completion lengths shrink substantially relative to unmodified GRPO – confirming the length-normalization term was the source of the bias.

Other Refinements

- The **standard-deviation** normalization in the advantage formula can itself introduce a **difficulty bias** (noted in Liu et al. 2025's Dr. GRPO analysis): for very hard problems (where most completions fail), the standard deviation of rewards is small, distorting the advantage signal.
- **Encouraging diversity** and **asymmetric clipping** (explored in Yu et al. 2025's DAPO paper): giving the same ϵ to both the lower and upper clip bounds can be unfair when a token's probability is already very low, since the multiplicative clip range $[(1-\epsilon)\pi_{\text{old}}, (1+\epsilon)\pi_{\text{old}}]$ then allows very little absolute room to grow – motivating asymmetric bounds and other diversity-preserving adjustments, among others.

6 Case Study: The DeepSeek R1 Pipeline

R1-Zero: RL From Scratch

Starting from DeepSeek-V3's pretrained base (**V3-Base** – a modern MoE architecture with roughly **671B total parameters, ∼37B active** per token, per DeepSeek-AI 2024, *"DeepSeek-V3 Technical Report"* – plus multi-latent attention and pre-norm), R1-Zero (DeepSeek-AI 2025, *"DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"*) skips SFT entirely and applies GRPO directly using only format and correctness rewards, with a simple templated prompt instructing the model to reason inside `<think>` tags and answer inside `<answer>` tags. This alone substantially improved AIME-style benchmark accuracy over RL training – demonstrating that RL with purely verifiable rewards can induce strong reasoning **without any prior supervision**.

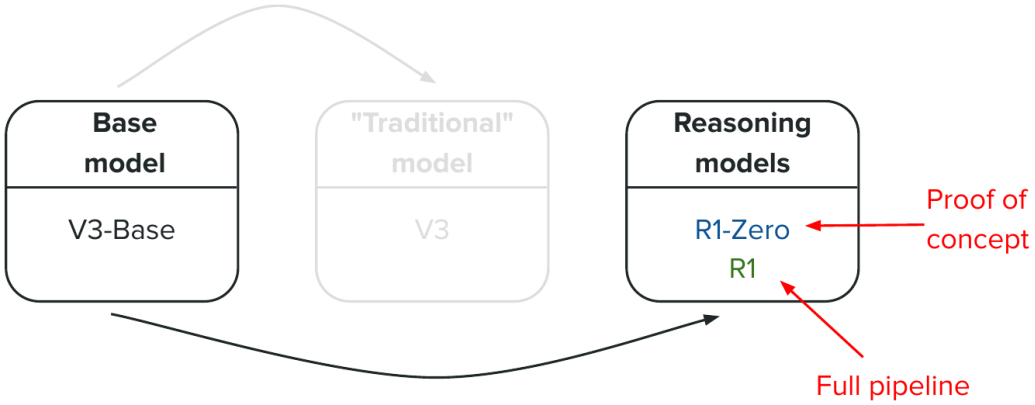


Figure 5: From base model (V3-Base) to reasoning models: R1-Zero (proof of concept) and R1 (full pipeline).

Issues

Inspecting R1-Zero's reasoning chains revealed problems: **language mixing** (switching languages mid-chain) and **poor formatting/syntax** – plausibly because the model had no strong supervised anchor to draw on when optimizing an otherwise unconstrained objective.

R1: The Full Pipeline

To address R1-Zero's issues, the full R1 pipeline adds several stages around the same core RL idea:

(1) **Cold-start SFT**: reasoning chains generated during the R1-Zero experiment were **human-rewritten** to fix formatting and language-consistency issues, then used as a (relatively small) SFT dataset to give the model a clean supervised starting point.

(2) **RL stage** (as in R1-Zero, plus a new **language-consistency reward**): a simple heuristic (e.g. the fraction of tokens in the target language within the reasoning chain) discourages language mixing.

(3) **Rejection-sampling SFT**: a larger-scale SFT stage mixing reasoning and non-reasoning data, where reasoning data (∼600K pairs spanning math, coding, and logic) was generated via **rejection sampling** – prompting the model trained so far, then filtering (via rule-based checks plus a V3-based judge) to keep only high-quality outputs – and non-reasoning data (∼200K pairs, "general" data) was recycled from DeepSeek-V3's own (non-reasoning) instruction data.

(4) **Final RL stage**: mixing reasoning rewards (as before) with **helpfulness** and **harmlessness** rewards on non-reasoning data, aligning the model as a general assistant. The harmlessness reward is applied over the *entire* output (including the reasoning chain, not just the visible answer), since the reasoning itself should also be harmless.

Results

On reasoning benchmarks, models cluster into two tiers: non-reasoning models score notably lower than reasoning-trained ones. R1 was found to be competitive with closed-source reasoning models of the time, a notable result given its openly published methodology.

Distilling Reasoning into Smaller Models

For teams without the resources to run a very large MoE reasoning model, **distillation** (yielding the **R1-Distill** model family) offers a route to smaller reasoning-capable models. Recall standard distillation (Lecture 2): fitting a student to the teacher's full **output probability distribution** on fixed data.

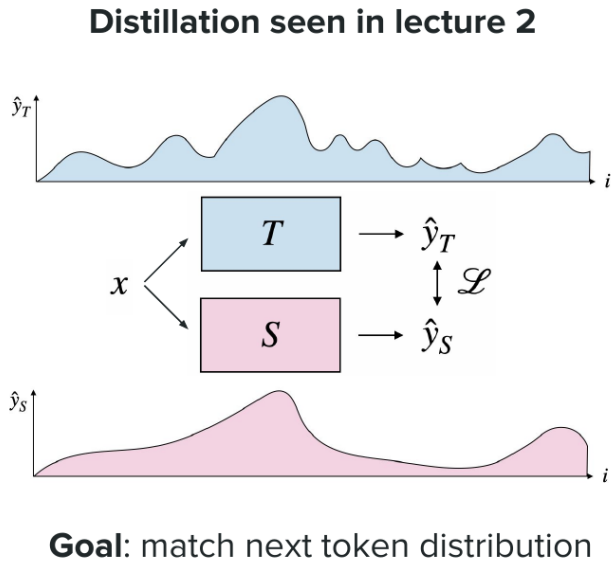


Figure 6: Standard distillation: the student S is trained to match the teacher T's output distribution $\hat{y}_T$ on the same input x.

Here, since there's no pre-existing reasoning SFT data, the process instead: (1) uses the trained teacher (R1) to **generate** entire sample responses (including reasoning chains) for a set of prompts *offline*, then (2) fits a smaller **student** model via standard SFT to reproduce those **exact output sequences** (rather than distilling a full probability distribution token-by-token) – i.e. the goal shifts from "match next-token distribution" to "SFT-learn the reasoning traces." Results were reported to be both **competitive** with closed-source small reasoning models (e.g. o1-mini) and a **good use of compute**. Notably, at smaller model scale, this distillation approach was found to be **more effective** than training a small model with RL from scratch – the teacher's demonstrated reasoning patterns transfer more efficiently than rediscovering them via RL at limited scale.

Summary

This lecture introduced **reasoning models**: what distinguishes reasoning from knowledge tasks, why explicit reasoning chains help (decomposition into tractable sub-problems, additional inference-time compute), and the benchmarks (coding, math) and metrics (**pass@k**, with its full combinatorial derivation) used to evaluate them. We covered why **RL** rather than SFT is the natural training approach given verifiable rewards, and **GRPO** as the key algorithm – computing advantage via group-relative reward comparison rather than a learned value function – including the **length-bias** problem this introduces and its fixes (DAPO, Dr. GRPO). Finally, the **DeepSeek R1** case study showed how these pieces combine into a full pipeline (cold-start SFT, RL, rejection-sampling SFT, final alignment RL), plus how reasoning capability can be **distilled** into smaller models via direct sequence-level SFT on teacher-generated data.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 7: RAG, Tool Calling, and Agents

Afshine Amidi, Shervine Amidi

Recap of Lecture 6

Lecture 6 covered **reasoning models**: letting an LLM produce a hidden reasoning chain before its final answer, trained via RL (specifically **GRPO**, which computes advantage by comparing a group of sampled completions' rewards rather than using a learned value function). Applying GRPO with format and correctness rewards was shown to measurably improve performance on reasoning benchmarks (e.g. AIME), though output length tended to grow unboundedly during training – a **length bias** traced to a per-completion normalization term in the GRPO loss, addressed by follow-up work (DAPO, "GRPO Done Right"). Reasoning was one of several **weaknesses** of vanilla LLMs identified before that lecture; today we address two more: LLMs' **static knowledge** (fixed at the pretraining cutoff) and their inability to **take action** in the world. This lecture covers **RAG**, **tool calling**, and **agents** as solutions.

1 Part 1: RAG – Retrieval-Augmented Generation

Motivation: The Knowledge Cutoff Problem

A pretrained LLM only knows what was in its training data up to its **knowledge cutoff date** (always listed in model cards, e.g. GPT-5's is September 30, 2024). Asked about events after that date, the model cannot answer correctly on its own.

Why Not Just Keep Retraining or Stuff Everything in the Prompt?

- **Continual retraining** to inject new knowledge is risky: it's difficult to update a model's knowledge without causing regressions elsewhere, and impractical to redo for every fine-tuned downstream use case.
- **Put everything relevant in the prompt**: even though modern context windows are large (e.g. hundreds of thousands of tokens, roughly hundreds of pages of text), this isn't a full solution:
  - **Context rot / needle-in-a-haystack degradation**: experiments burying a fact in a long prompt and asking the model to retrieve it (Kamradt 2023, *"Needle in a Haystack – Pressure Testing LLMs"*) show that retrieval accuracy degrades as prompt length grows, and is especially poor when the relevant fact sits in the first half of a long document – irrelevant context actively *confuses* the model, it doesn't just sit inertly.
  - **Cost**: LLM API pricing is per token (e.g. order of $1 per million tokens for a model like GPT-5), so padding every prompt with unnecessary content adds up.

The better approach: only include the **relevant** information in the prompt. This is the idea behind **RAG** (Retrieval-Augmented Generation; Lewis et al. 2020, *"Retrieval-Augmented Generation for*

Knowledge-Intensive NLP Tasks").

The Three Steps of RAG

(1) **Retrieve**: fetch relevant documents/chunks from a knowledge base given the prompt.

(2) **Augment**: insert the retrieved information into the prompt.

(3) **Generate**: have the LLM answer using the augmented prompt.

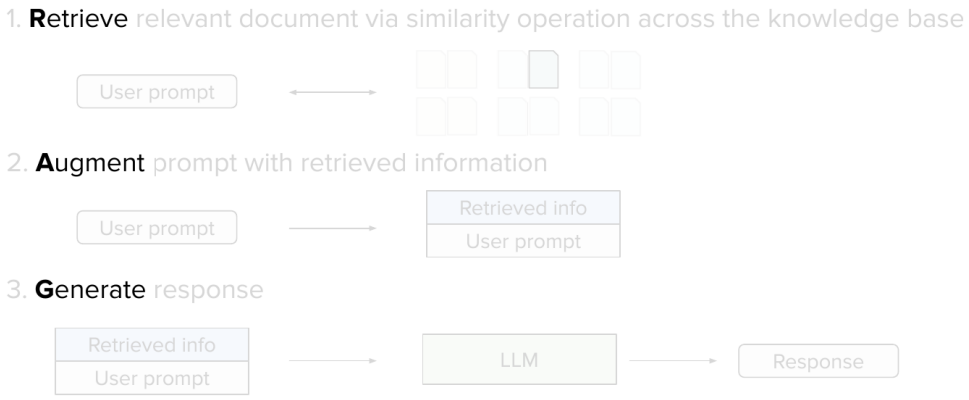


Figure 1: Retrieve, Augment, Generate: the three steps of RAG.

The retrieval step is where most of the engineering complexity (and risk of failure) lies – if retrieval surfaces the wrong information, generation quality suffers regardless of the LLM itself.

Q: Couldn't the retrieval step just do a bad job and surface the wrong documents?
A: Yes, exactly – and that's precisely why the retrieval stage deserves so much attention: it's the part of the pipeline most prone to failure, and no amount of downstream generation quality can compensate for the wrong information being retrieved in the first place. Evaluating and improving retrieval quality (covered later in this section) is a first-class concern in building a good RAG system, not an afterthought.

Building the Knowledge Base

Documents are collected and split into **chunks** (subsets of a document up to a maximum length, typically on the order of hundreds of tokens), each represented by a precomputed **embedding**. Key hyperparameters:

- **Embedding size**: bigger captures more nuance but costs more space/compute (typical order of magnitude: thousands, e.g. $\sim$1500).
- **Chunk size**: too small loses context; too large dilutes the embedding's meaningfulness (typical: $\sim$hundreds of tokens, e.g. $\sim$500).
- **Chunk overlap**: some overlap between consecutive chunks preserves cross-chunk context (typical: low hundreds of tokens).

Q: Do you train your own embedding model for this, or use an existing one?
A: Both are options – you can use a pretrained embedding model (which is what people typically do),

or train your own if you have the data and need for it.

Q: What exactly is the embedding model trying to achieve?
A: It's trying to represent chunks (and queries) in a way that serves the end goal: fetching relevant documents. Concretely, that means chunks with similar meaning should end up close together in embedding space, so that a similarity search (e.g. cosine similarity) reliably surfaces the right chunks for a given query.

Stage 1: Candidate Retrieval

Analogous to techniques from recommendation systems and search, retrieval proceeds in two stages, the first prioritizing **recall** over precision.

Semantic (Embedding-Based) Search

Both the query and each chunk are embedded (using an encoder-only model, in a **bi-encoder** setup: query and chunk encoded independently, then compared), and **cosine similarity** ranks chunks by relevance, keeping the top (e.g. ∼100) candidates. Because the knowledge base can be huge, exact search is replaced with **approximate nearest neighbor (ANN)** methods, which partition the embedding space to avoid a costly linear scan. A common way to train such embeddings is **Sentence-BERT** (Reimers & Gurevych 2019, *"Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks"*): an extension of BERT with a loss function that pushes cosine similarity high for relevant pairs and low for irrelevant ones.

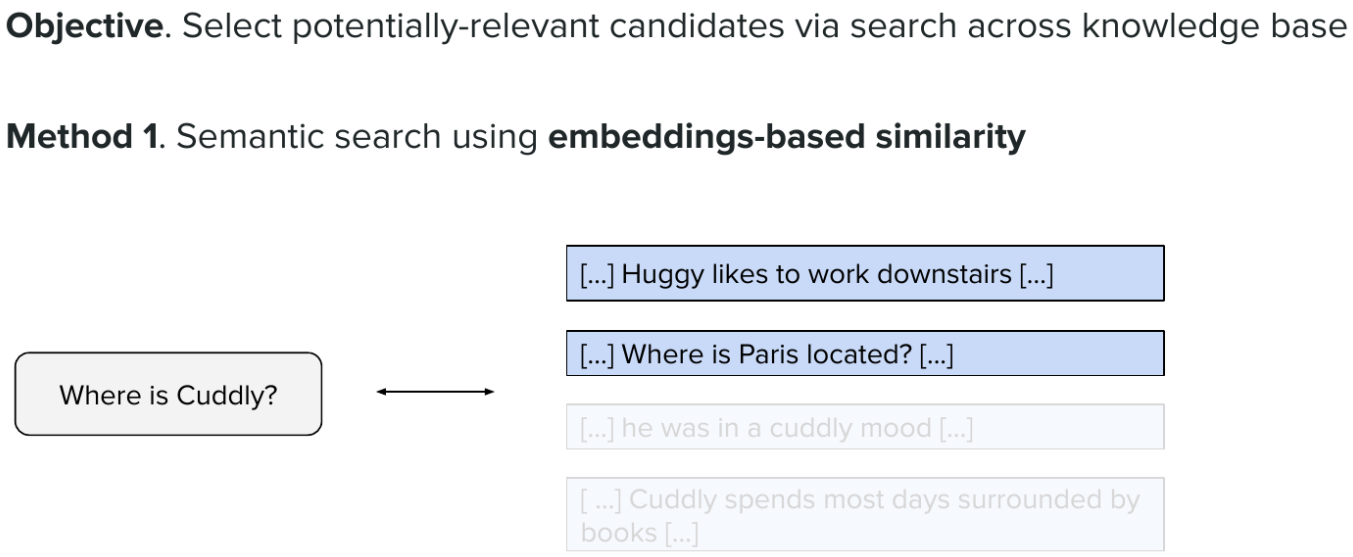


Figure 2: Semantic search for "Where is Cuddly?" surfaces semantically-similar-but-wrong matches (mentioning a different bear, "Huggy," or an unrelated question about Paris) alongside the two genuinely relevant chunks about Cuddly.

Q: Is cosine similarity the default/standard way to compute similarity between embeddings?
A: Yes, typically – though different implementations sometimes use other distances, such as L2 distance. These are related in practice: for example, once vectors are normalized to unit norm, L2 distance and cosine similarity become closely related (differing by a simple, deterministic transformation), so the

choice often comes down to implementation convenience rather than a fundamentally different notion of similarity.

Keyword-Based Search (BM25)

Semantic search finds meaning-similar chunks but offers no guarantee of literal keyword overlap. **BM25** is a heuristic relevance score based on query/document term overlap, useful when exact keyword matches genuinely matter (e.g. searching for a specific named entity like "Cuddly" the teddy bear, where a semantically similar but differently-named entity like "Huggy" shouldn't necessarily rank first). In practice, teams often use a **hybrid** of embedding-based and BM25 scores.

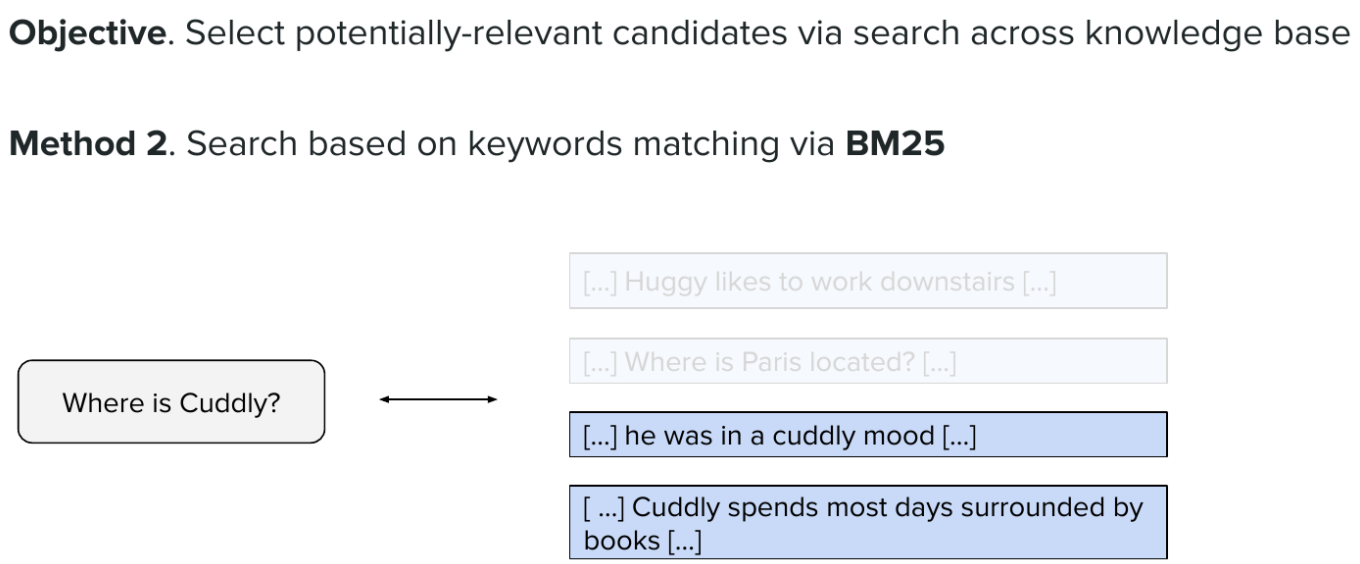


Figure 3: BM25 keyword matching for the same query correctly surfaces only the chunks that literally mention "Cuddly."

Query/Document Mismatch: HyDE

Queries (short, often questions) and documents (long, declarative) differ stylistically, making their embeddings less directly comparable even when semantically related. **HyDE** (Hypothetical Document Embeddings; Gao et al. 2022, *"Precise Zero-Shot Dense Retrieval Without Relevance Labels"*) addresses this by first prompting an LLM to generate a plausible (possibly fabricated) *document* answering the query, then embedding *that* to search – better matching the style of what's being searched for. (An alternative is training separate query and document encoders, though this adds maintenance overhead.)

Contextual Chunking

Chunks taken out of context may become ambiguous on their own.

Q: Is chunking always done naively (e.g. by fixed token count), regardless of the document's actual structure?
A: Not necessarily – and this is a genuinely important question. Contextual chunking (below) is one way to mitigate the problem of context-free chunks. But it also matters what kind of document you're chunking in the first place: a JSON file, a Markdown document, or code all have their own internal structure, and being aware of that structure when deciding where to split (e.g. not splitting in the middle

of a JSON object) can meaningfully improve chunk quality, beyond just picking a fixed token-count boundary.

One fix, **contextual retrieval** (Anthropic team 2024, *"Introducing Contextual Retrieval"*): use an LLM call to generate a short context summary (based on the full source document) to prepend to each chunk before embedding/search. Since this requires many LLM calls with a shared document prefix, **prompt caching** makes it economical: because decoder-only models process a shared prompt prefix identically each time, providers can cache and reuse those computed activations rather than recomputing them, often at a fraction of the normal per-token price (e.g. roughly 1/10th for cached input tokens with some providers) – so structuring prompts to share long common prefixes is a useful cost-saving pattern generally, not just here.

Stage 2: Ranking (Re-Ranking)

An optional but often valuable second stage refines the initial candidate list using a more computationally expensive but more accurate method: a **cross-encoder**, which feeds the query and chunk *together* into the same encoder (allowing attention between them), producing a more precise relevance score than the bi-encoder's independently-computed embeddings.

Q: Does the cross-encoder actually compute attention between the query and the chunk?
A: Yes, absolutely – that's precisely what distinguishes it from the bi-encoder setup: the query and chunk tokens are fed into the encoder together, so self-attention lets them directly interact, unlike the bi-encoder's independently-computed embeddings that are only compared *after* the fact.

Q: How is the cross-encoder/re-ranker actually trained – e.g. with a contrastive loss?
A: That's one option among several possible loss functions used to train these models; the Sentence-BERT paper works through and compares a few different loss formulations in detail (including contrastive-style objectives), and is a good reference for anyone wanting to see the specifics.

Evaluating Retrieval Quality

Given a set of retrieved (ranked) chunks and ground-truth relevance labels, several metrics quantify retrieval quality:

- **NDCG@k** (Normalized Discounted Cumulative Gain): rewards placing relevant chunks *higher* in the ranking.

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}, \qquad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

  where $\text{rel}_i \in \{0, 1\}$ indicates whether the chunk at rank i is actually relevant, and IDCG@k is the DCG of the ideal (best possible) ranking – normalizing by it ensures a perfect ranking scores exactly 1.

- **RR@k** (Reciprocal Rank at k; averaged across queries, this becomes **MRR**, Mean Reciprocal Rank): the inverse of the rank of the *first* relevant chunk (e.g. if the first relevant chunk is at rank 2, the contribution is 1/2) – simpler than NDCG, ignoring anything past the first hit.

- **Recall@k**: of all truly relevant chunks, what fraction appear in the top k?

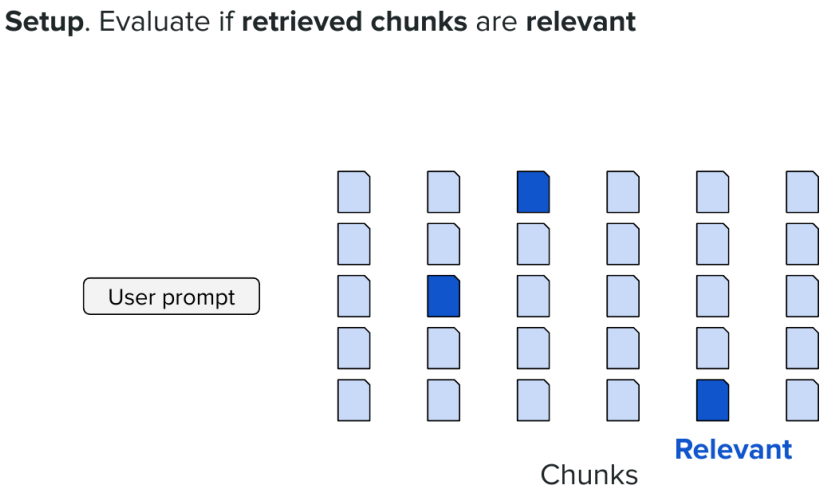


Figure 4: Evaluation setup: for a given prompt, which of the retrieved chunks are actually relevant?

- **Precision@k**: of the top k retrieved chunks, what fraction are truly relevant?

The **Massive Text Embedding Benchmark (MTEB)** is a popular benchmark suite for evaluating retrievers/embedding models on these metrics.

Q: In the NDCG (and related) formulas, how is "relevance" actually determined for each retrieved chunk?
A: You can think of candidate retrieval and re-ranking as together producing a set of chunks with scores; sorting them by score gives you the ranking to evaluate. "Relevance" itself is the ground-truth label – for each retrieved chunk, is it *actually* relevant to the query or not (as determined by whatever labeling process built the evaluation set)? Those ground-truth labels, cross-referenced against the ranking your system produced, are what feed into the NDCG/precision/recall formulas.

---

2 Part 2: Tool Calling

Definition

Tool calling (function calling), per IBM's definition, lets autonomous systems complete complex tasks by dynamically accessing – and potentially acting upon – external resources (IBM, *"Tool Calling Overview"*) – filling knowledge or capability gaps a pretrained LLM cannot fill on its own. Tools are typically presented to the LLM as Python-like function signatures with documentation (though the underlying language isn't fixed to Python), **predefined in advance** by developers, not generated on the fly by the LLM.

Anatomy of a Tool

A tool definition includes: a **description** (so the model understands its purpose), a well-specified set of **input arguments**, and a **structured output** format (e.g. a typed class) so the result is machine-interpretable. Crucially, the LLM sees only the function signature and documentation – *not* the implementation – since its job is only to decide *which* function to call and *with what arguments*, not to simulate its internal logic.

Q: Are the function APIs pre-computed/predefined, or generated on the fly?
A: Predefined – you (the developer) define these APIs beforehand. They are not generated by the LLM on the fly; the model only ever selects among, and populates arguments for, tools that already exist.

The Three-Stage Process

(1) **Tool prediction**: given the available function API(s) (in the preamble) and the user query, the LLM outputs the function name and arguments to call (e.g. inferring a user's location from context to populate a coordinate argument).

(2) **Execution**: the function is actually run (outside the LLM) with the predicted arguments, producing a structured result.

(3) **Response synthesis**: the original query, the tool call, and its structured result are fed back to the LLM, which produces a natural-language response for the user.

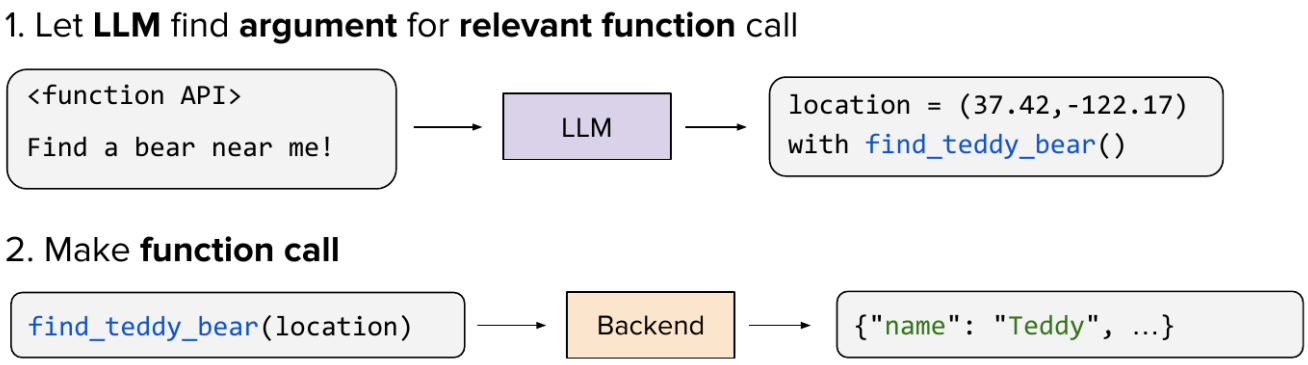


Figure 5: Steps 1–2: the LLM predicts the function call and arguments, then the backend actually executes it.

Training a Model to Use Tools

Two SFT stages are typically involved: (1) pairs mapping (query + function API) → (correct function call), covering varied phrasings, multiple tools, and multi-turn contexts; and (2) pairs mapping (conversation history + tool result) → (final natural-language response), so the response is appropriately formatted rather than just echoing raw structured data.

Q: If there are multiple tools available, do you need some kind of tool selection mechanism, or more training examples?
A: Both can help. On the SFT side, you can include multi-tool examples directly in the training data, showing the model how to choose correctly among several available functions. But at larger scale, this is exactly what motivates a dedicated **tool selector/router** (covered shortly), which narrows down the candidate tools *before* generation, rather than relying purely on SFT to teach discrimination among many tools at once.

An Alternative to SFT: Prompt-Based Tool Use

Since modern LLMs already have strong general reasoning and code-manipulation ability (from pre-training), it's sometimes possible to skip dedicated tool-use SFT and instead rely on a well-written **natural-language explanation** of how to use a new tool.

Q: Couldn't you just use few-shot learning (showing input/output examples in the prompt) instead of writing a full explanation?
A: That's a reasonable idea, and it is indeed one accepted practice. The trade-off is that few-shot examples can have generalization challenges: since you're showing specific input/output pairs, the model may fit narrowly to those particular cases rather than generalizing well across the full range of ways a query might be phrased.

Q: What about just asking the model to reason its way through choosing the tool call, instead of relying on examples?
A: Also a great direction – but writing a prompt that elicits good reasoning for this, in a way that actually generalizes well, is itself hard to do by hand. In practice, rather than writing that reasoning-eliciting prompt yourself from scratch, it works well to treat existing labeled (query, correct call) pairs as an evaluation set, and have a strong reasoning model iteratively refine the explanation/prompt based on where it currently succeeds or fails – offloading the hard part of prompt-writing to the reasoning model itself.

Since hand-crafting such an explanation well is hard, one practical approach: use existing labeled (query $\rightarrow$ correct call) examples as an **evaluation set**, iterate a draft explanation against it, and feed the pattern of successes/failures back to a strong reasoning model to *improve the explanation itself* – an offline, one-time process (the resulting fixed explanation is then used at inference time for all queries), which can work surprisingly well in practice.

Q: Is this explanation-writing process something that happens at training time, or at inference time for each query?
A: Training time (offline) – you iterate on and finalize the explanation once, ahead of time, using the evaluation set. The resulting fixed explanation is then reused as-is at inference time, alongside the function API, for every incoming query; it isn't regenerated per query.

Categories of Tool Use

- **Informational**: fetching real-time or external data (search, weather, stock prices).
- **Computational**: converting a query into executable code (e.g. for precise math) rather than relying on the model's own arithmetic via reasoning chains.
- **Action-taking**: performing real-world side effects on the user's behalf (e.g. sending an email).

Scaling to Many Tools: Tool Selection / Routing

Putting *every* available tool's definition into every prompt runs into the same needle-in-a-haystack degradation seen with RAG, wastes context budget, and can dilute performance across too many

supported use cases. A **tool selector** (or **router**; Robert et al. 2024, *"Automatic Tool Selection to Reduce Large Language Model Latency"*) addresses this in two steps: given the query and a lightweight list of tool names/short descriptions, an LLM first **selects** a relevant subset of tools; only those full tool definitions are then included in the actual generation prompt. (This selection step could itself be implemented via a RAG-style retrieval over tool descriptions.)

Q: Isn't this tool-selection step basically just RAG?
A: It *could* be implemented via RAG-style retrieval over tool descriptions – that's a great observation – but it doesn't have to be. The selection step could just as well be its own dedicated LLM call, given instructions and the lightweight list of tool names/descriptions, without necessarily going through an embedding-based retrieval pipeline. RAG is one valid implementation choice, not the only one.

Standardization: MCP (Model Context Protocol)

Without a standard, every tool must be re-implemented per LLM/host, causing duplicated work. **MCP** (Anthropic 2024, *"Introducing the Model Context Protocol"*; see also *"About MCP: Architecture Overview,"* Anthropic 2024) standardizes how tools are exposed to models, with key vocabulary:

- **MCP server**: the instance that serves tools for a given piece of infrastructure/provider.
- **Tools**: the actual callable functions.
- **Prompts**: templates showing how to use the tools.
- **Resources**: external data sources the tools can draw on.
- **MCP client**: the infrastructure on the LLM-host side that connects to an MCP server (a one-to-one connection).

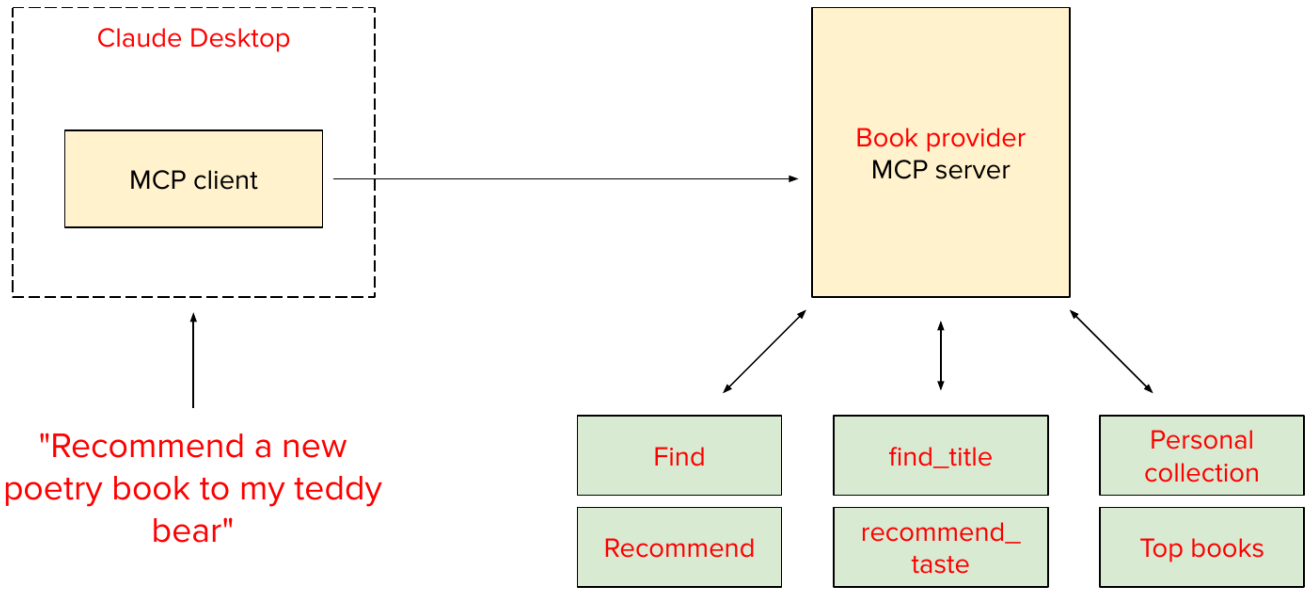


Figure 6: MCP architecture: an MCP client (inside the host, e.g. Claude Desktop) connects to an MCP server exposing tools.

Example: for book recommendations, an LLM host (e.g. Claude) connects to a book provider's MCP server, which exposes tools for finding/recommending books, prompts showing how to search by title

or by taste, and resources like a user's personal book collection or bestseller lists.

3 Part 3: Agents

Definition

An **agent** is a system that **autonomously pursues a goal and completes tasks on a user's behalf**. Compared to a single tool call, an agent adds **reasoning** and the ability to **loop** – iterating through multiple steps (potentially multiple tool calls) rather than a single request/response.

ReAct: Reason + Act

The **ReAct** framework (Yao et al. 2022, *"ReAct: Synergizing Reasoning and Acting in Language Models"*) decomposes complex tasks into repeating cycles, commonly labeled **observe**, **plan**, **act** (naming and exact ordering vary by paper/implementation, but the high-level loop structure is consistent):

- **Observe**: interpret the current situation/state and what's needed.
- **Plan**: decide the next sub-goal or action needed.
- **Act**: execute a tool call (or generate a response) to pursue that sub-goal.

Worked Example

Query: "My teddy bear is cold. Please do something."

(1) **Observe**: the bear is cold, likely due to room temperature, which is currently unknown.

(2) **Plan**: need to determine the room's temperature.

(3) **Act**: call `get_current_room_temperature()`.

(4) **Observe**: temperature returns 65°F – about 5°F less than an average/comfortable temperature.

(5) **Plan**: need to raise the temperature.

(6) **Act**: call `set_temperature(target=70)`.

(7) **Observe**: temperature is now correctly set – goal achieved; exit the loop.

(8) **Output**: report back to the user (e.g. "I've raised the temperature by 5 degrees").

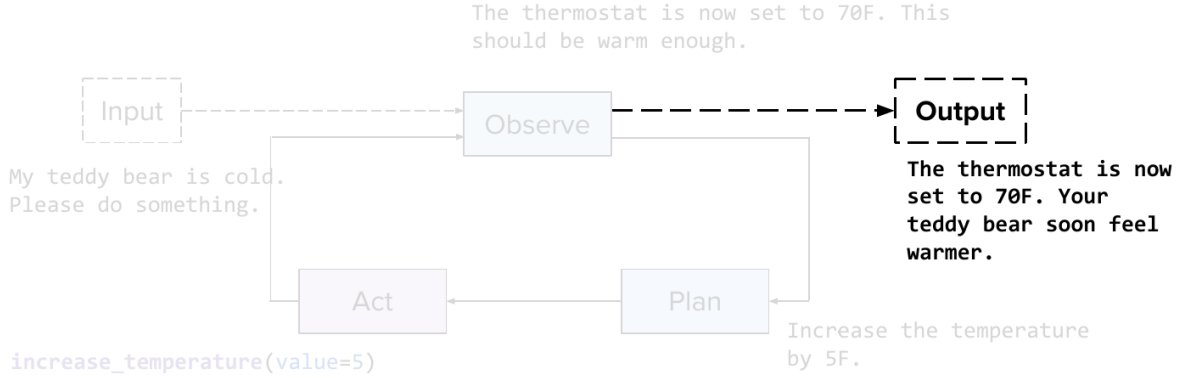


Figure 7: The ReAct loop's final iteration: after observing the room is now warm enough, the agent reports back to the user.

This loop of goal-directed reasoning and tool use, repeated until the goal is met, is what distinguishes an **agentic** workflow from a single tool call.

Multi-Agent Systems

Different agents can specialize (e.g. a thermostat agent, an air-quality agent, an energy-management agent, an occupancy agent) and potentially need to **communicate with one another** – motivating standardization analogous to MCP, but for agent-to-agent communication. Google's **Agent2Agent (A2A)** protocol (Google 2025, *"Announcing the Agent2Agent Protocol (A2A)"*; full spec in *"Agent2Agent (A2A) Protocol Official Specification,"* Google 2025) standardizes what an agent exposes: an **AgentCard** (identity/metadata), a set of **AgentSkills** (capabilities, each with usage examples for other agents to understand them), and an **AgentExecutor** defining how it **executes** a given request and how a **cancel** mechanism works to stop an in-progress action.

Q: Is each agent essentially just an LLM given some context to perform specific tasks?
A: That can very well be the case, yes – in the example given here, each specialized agent (thermostat, air quality, etc.) operates independently, with its own reasoning loop; the input/output between agents is what ties them together, but each one runs its own process internally.

Q: With multiple independent agents each doing their own reasoning, couldn't your token budget spiral out of control?
A: That's a fair and important concern – and it's exactly why budget restrictions are typically put in place in real deployments. That said, each agent operates on its *own* budget rather than eating into another agent's allotment; the aggregate cost across many agents can still add up (hitting your overall money/compute budget), but it's not a case of one agent silently consuming another's resources.

Safety

Increased agentic capability (executing real actions) introduces new risks:

- **Data exfiltration**: e.g. a malicious or injected instruction tricking an agent with email access into sending sensitive data (like a password) to an attacker-controlled address (a risk category studied in Ye et al. 2024, *"ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages"*).

Mitigations fall into two categories (see also Chen et al. 2024, *"Towards Tool Use Alignment of Large Language Models"*):

- **Training-time**: incorporating harmlessness-focused data into SFT/RL training mixtures (as seen in the DeepSeek R1 pipeline's final alignment stage).
- **Inference-time safeguards**: e.g. a safety classifier reviewing the conversation/tool calls before execution.

The **Agent Safety Bench** catalogs possible safety hazards and provides a benchmark suite for assessing agent safety. As a real-world illustration of the stakes, Anthropic has publicly documented a large-scale cyberattack that leveraged an LLM's tool-calling and agentic capabilities (Anthropic 2025, *"Disrupting the First Reported AI-Orchestrated Cyber Espionage Campaign"*), along with a detailed account of remediation steps – underscoring that both attack sophistication and defensive measures continue to evolve together as agentic capabilities grow.

Practical Guidance

- **Error accumulation**: at every step of an agentic loop, there's a chance of diverging from a correct trajectory (e.g. poor grounding, incorrect tool arguments) – a key reason large-scale, fully autonomous agents remain limited in practice today.
- **Start small**: begin with a single, simple, well-scoped tool/task before expanding scope.
- **Start with the most capable model** first to establish an upper bound on achievable performance, then optimize for latency/cost afterward.
- **Prioritize correctness before efficiency**.
- **Debug via the reasoning chain**: since agentic systems expose their reasoning as readable tokens (rather than opaque weights), inspecting that chain is a practical way to diagnose failures.
- A particularly valuable current use case: **AI-assisted coding agents**, which can offload substantial implementation work – but the ability to *judge* whether generated code is correct remains an essential skill, arguably more important than ever as generation itself becomes cheap.

Summary

This lecture covered three complementary ways to extend an LLM beyond its static, text-only pretrained capabilities. **RAG** addresses the knowledge-cutoff problem by retrieving (via a two-stage bi-encoder/cross-encoder pipeline, with techniques like BM25, HyDE, and contextual chunking) and injecting only relevant external information into the prompt, evaluated via ranking metrics like NDCG and MRR. **Tool calling** lets an LLM invoke external functions via a predict/execute/synthesize cycle, with **tool selection** and the **MCP** standard addressing scale and interoperability. **Agents** combine reasoning with looped tool use (the **ReAct** framework) to autonomously pursue multi-step goals, with **safety** (e.g. data exfiltration risks) an increasingly important consideration as these systems gain the ability to act in the world.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 8: LLM Evaluation

Afshine Amidi, Shervine Amidi

Recap of Lecture 7

Lecture 7 covered how an LLM can interact with systems outside of itself. We saw **RAG** (Retrieval-Augmented Generation), which lets a model fetch information from an external knowledge base through two stages: candidate retrieval (a bi-encoder setup, e.g. Sentence-BERT, used to filter down relevant candidates) and re-ranking (a more sophisticated cross-encoder stage). We also reviewed metrics to quantify retrieval quality. We then covered **tool calling**, i.e. the ability of a model to decide which tool to call and with which arguments, execute it, and translate the result back into natural language. Finally, we saw that **agentic workflows** combine RAG and tool calling: given an input, the model can make multiple calls to different tools to gather information, as illustrated by the **ReAct** framework (Reason + Act), decomposed into observe, plan, and act steps, with AI-assisted coding as a leading real-world application.

Introduction: What Does "Evaluation" Mean?

The word *evaluation* is overloaded: it can refer to output quality (coherence, factuality), or to system-level metrics (latency, price, uptime). This lecture focuses on **output quality**: given a response, how do we quantify how good it is? This is inherently hard because an LLM is a text-to-text model whose output can be natural language, code, math, and more, making a universal metric elusive.

Human Evaluation: The Ideal (but Impractical) Baseline

The conceptually ideal setup is to have a human rate every LLM output, repeated across many prompts, and to aggregate these ratings into an overall performance measure.

Limitation 1: Subjectivity and Inter-Rater Agreement

Rating tasks can be subjective. For example, asked whether "a teddy bear is almost always a sweet gift, just pick one that feels right for you" is a *useful* response to "what birthday gift should I get," two human raters may disagree. This motivates measuring **inter-rater agreement**.

A naive metric is the **agreement rate**: the proportion of times two raters give the same (e.g. binary) rating. This metric is misleading on its own, since two independent, random raters will already agree part of the time by chance. If rater A rates "good" with probability P_A and rater B with probability P_B, and both rate independently and at random, then:

$$P(\text{agree}) = \underbrace{P_A P_B}_{\text{both say 1}} + \underbrace{(1-P_A)(1-P_B)}_{\text{both say 0}}$$

If $P_A = P_B = 0.5$:

$$P(\text{agree}) = 0.5^2 + 0.5^2 = 0.25 + 0.25 = 0.5$$

So two purely random raters already agree 50% of the time. This means a raw agreement rate cannot be interpreted without a chance baseline.

To correct for this, metrics such as **Cohen's kappa** (Cohen 1960, *"A Coefficient of Agreement for Nominal Scales"*) compare the observed agreement rate p_o to the chance agreement rate p_e – framed as: how much better is our agreement than what we'd expect by pure chance, given how the raters actually use the categories?

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

If $p_o = 1$ (perfect agreement), $\kappa = 1$. If $p_o < p_e$ (worse than chance), κ is negative; if $p_o > p_e$, κ is positive. Extensions to more than two raters include **Fleiss's kappa** (Fleiss 1971, *"Measuring Nominal Scale Agreement Among Many Raters"*) and **Krippendorff's alpha**.

In practice, teams track this agreement as a health metric, and hold "agreement (calibration) sessions" between raters when it is unsatisfactory, to align on rating guidelines.

Limitation 2: Cost and Speed

Human rating does not scale: rating even a thousand outputs takes significant time and money, making per-output human rating impractical at scale.

Rule-Based (Reference-Based) Metrics

Instead of rating every output, humans can instead write **reference** (ideal) outputs once for a fixed set of prompts. Model outputs are then compared against these fixed references using automatic metrics, allowing repeated evaluation without repeated human rating.

METEOR

METEOR (Metric for Evaluation of Translation with Explicit ORdering; Banerjee & Lavie 2005, *"METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments"*), used originally for translation, combines an F-score with an ordering penalty:

$$\text{METEOR} = F_{\text{mean}} \times (1 - \text{Penalty})$$

where F_{mean} is a weighted harmonic mean of unigram precision and recall (precision: proportion of predicted unigrams matching the reference; recall: proportion of reference unigrams matching the prediction), and the penalty rewards contiguous, correctly ordered matches:

$$\text{Penalty} = \gamma \left(\frac{C}{U_m} \right)^{\beta}$$

where C is the number of contiguous matched chunks, U_m is the number of matched unigrams, and γ, β are hyperparameters. A lower C (i.e. long contiguous matches) means better ordering agreement with the reference, hence a lower penalty and a higher METEOR score.

BLEU and ROUGE

BLEU (BiLingual Evaluation Understudy; Papineni et al. 2002, *"BLEU: A Method for Automatic Evaluation of Machine Translation"*) is a precision-oriented metric based on matching n-grams between prediction and reference, with a **brevity penalty** to discourage gaming the metric via overly short outputs. **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation; Lin 2004, *"ROUGE: A Package for Automatic Evaluation of Summaries"*) follows a similar philosophy (with several variants, e.g. ROUGE-N, ROUGE-L) and is typically used for summarization tasks.

Limitations of Reference-Based Metrics

- **No stylistic flexibility**: semantically equivalent but differently worded outputs (e.g. "a plush teddy bear can comfort a child during bedtime" vs. "soft stuffed bears often help kids feel safe as they fall asleep") are penalized despite being equally good.
- **Weak correlation with human judgment**, despite hand-tuned hyperparameters.
- Still requires human-written references to get started.

LLM-as-a-Judge

Given the limitations above, a natural idea is to use another LLM – pretrained on large amounts of data and tuned toward human preference – as the rater. This is called **LLM-as-a-Judge** (LaaJ; Zheng et al. 2023, *"Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena"*).

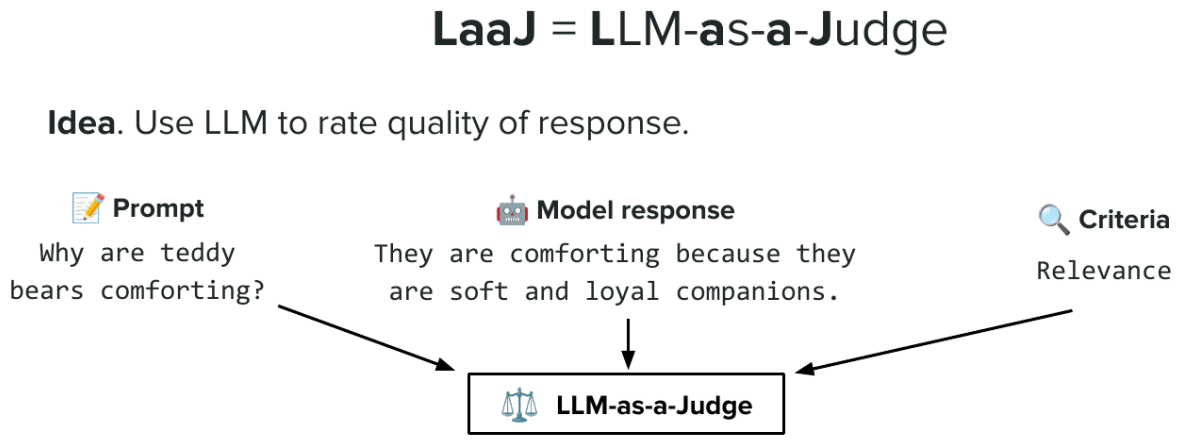


Figure 1: LLM-as-a-Judge: given a prompt, a model response, and grading criteria, the judge produces a rating.

Setup

The judge is given: the original prompt, the model response, and the grading criteria. It returns:

- A **score** (often binary: pass/fail).
- A **rationale** explaining the score – a key advantage over opaque rule-based formulas.

Empirically, asking the judge to output the rationale *before* the score improves quality, mirroring the chain-of-thought behavior of reasoning models: verbalizing the "thought process" before committing to an answer.

Guaranteeing a Parsable Output

Because sampling is probabilistic, a judge is not guaranteed to return a response in the requested (rationale, score) format.

Q: If we give the judge LLM a prompt asking for a rationale and a score, are we guaranteed to get back something we can actually parse?
A: No – because the model has a probabilistic sampling process, there's no hard guarantee it follows the requested format every time.

Q: Is there a technique that would *guarantee* a structured response?
A: Yes – **constrained (guided) decoding**: restrict sampling to only the tokens consistent with a target schema/grammar, so the output is forced to conform to the required structure. Providers expose this as **structured output** (e.g. passing a target class/schema so the response is guaranteed to conform to it).

The fix is **constrained decoding**, i.e. restricting sampling to tokens consistent with a target schema. Providers expose this as **structured output** (e.g. passing a target class/schema so the response is guaranteed to conform to it).

Two Main Variants

- **Pointwise**: judge a single response (good / not good).
- **Pairwise**: given two responses A and B, decide which is better – useful for generating synthetic preference data (e.g. to train a reward model, as in preference tuning).

Benefits

- No reference text or human ratings needed to get started.
- Interpretable via the rationale.

Failure Modes (Biases)

- **Position bias**: the judge favors the response presented first (or second), regardless of quality. *Remedy*: ask the judge both orderings (A vs. B and B vs. A) and take the majority vote (more advanced remedies tweak position embeddings directly).
- **Verbosity bias**: the judge favors longer, more elaborate responses even when they are not more correct. *Remedies*: explicitly instruct the judge to disregard length, add in-context examples demonstrating this, or apply a length penalty to the score.
- **Self-enhancement bias**: a model tends to prefer outputs it generated itself. *Remedy*: avoid using the same model for generation and judging; prefer a different (ideally larger, stronger-reasoning) model as judge.

These three are illustrative, not exhaustive – e.g. a judge may also be systematically misaligned with ground-truth human preference in other ways.

Figure 2: Position bias: swapping the order of the two responses can flip the judge's verdict.

Best Practices

- Write crisp, explicit grading guidelines.
- Prefer a **binary** scale (pass/fail) over granular scales – easier for both the judge and for human calibrators, and removes noise from too many options.
- Output the rationale *before* the score.
- Periodically **calibrate** judge scores against human ratings (e.g. via correlation analysis) since human ratings remain the ground truth the judge approximates.
- Use a **low temperature** (e.g. 0.1–0.2) for reproducible evaluation results.

Overall: LLM-as-a-Judge approximates human ratings without needing per-output human labeling, but one must guard against over-optimizing against this proxy (recall **Goodhart's law**: when a measure becomes a target, it ceases to be a good measure).

Dimensions of Evaluation

Broadly, two dimensions matter:

- **Task performance**: usefulness, factuality, relevance, etc.
- **Alignment / format**: tone, style, safety of the response.

Deep Dive: Factuality

Consider the text: "Teddy bears, first created in the 1920s, were named after President Theodore Roosevelt after he proudly wanted to shoot a captured bear on a hunting trip." (In reality, teddy bears date to the 1900s, and Roosevelt *refused* to shoot the bear.) Factuality is nuanced – a text can be partially correct – so a binary label for the whole passage is too coarse. The standard approach (Wei et al. 2024, *"Long-Form Factuality in Large Language Models"*):

(1) Decompose the text into a list of atomic **facts** (via an LLM call).

(2) Check each fact for correctness (binary), typically via RAG / web search against a knowledge base.

(3) Aggregate with importance weights α_i per fact:

$$\text{Factuality score} = \frac{\sum_i \alpha_i \cdot \mathbb{1}[\text{fact}_i \text{ correct}]}{\sum_i \alpha_i}$$

(weights can all be set equal for simplicity). In the example above, two of the facts are correct, yielding a score of 0.6.

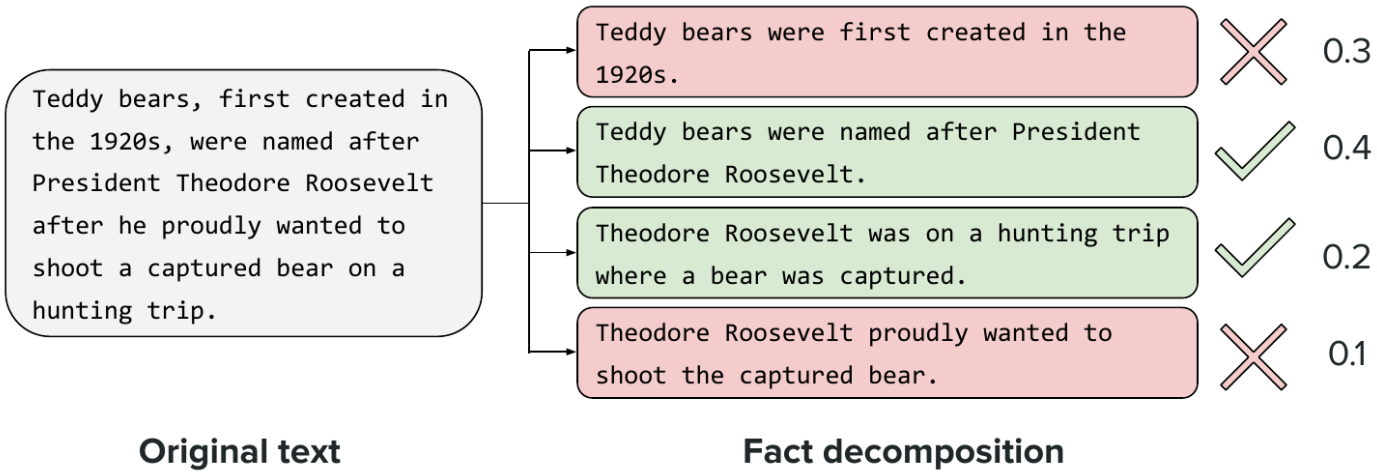


Figure 3: The teddy bear example decomposed into four atomic facts, each checked and weighted: $(0.4 + 0.2)/(0.3 + 0.4 + 0.2 + 0.1) = 0.6$.

Evaluating Agentic Workflows

Consider one loop of a tool call within the **ReAct** framework (Yao et al. 2022, *"ReAct: Synergizing Reasoning and Acting in Language Models"*; Lecture 7), decomposed into: (1) predict the right tool call and arguments, (2) execute the tool, (3) synthesize the result into a response. Each stage has characteristic failure modes.

Tool Prediction Errors

- **Punt / non-use of an available tool**: the model apologizes instead of calling a tool that clearly applies. May be caused by a **tool router / selector** recall error (the tool wasn't even shortlisted) or by the model simply not considering it despite being available (revisit SFT training data or prompt).
- **Tool hallucination**: the model calls a function that does not exist (e.g. calling `find_bear` when only `find_teddy_bear` is defined). Possible causes: the model is too weak or ungrounded (consider upgrading it), or the tool APIs/instructions are poorly written or unclear (improve function names, arguments, and descriptions; clarify top-level instructions that tools should be used).
- **Wrong tool chosen among valid options**: ambiguity about which tool should handle a given request. Fix by refining tool-router recall for that query type and by making each tool's intended scope precise in its API description.
- **Right tool, wrong arguments**: e.g. calling a location tool with placeholder coordinates $(0, 0)$ because the actual location was never supplied in context. Fixes include ensuring necessary context (e.g. location) is available, adding an explicit location-finder tool with actionable errors instead of dummy defaults, or retraining/rewriting the API if the model simply doesn't know what to pass.

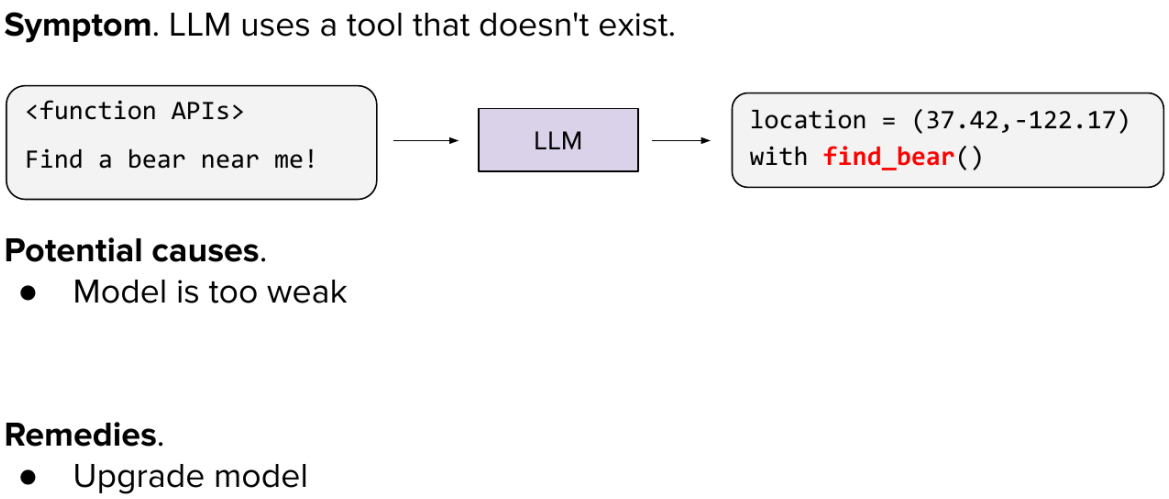


Figure 4: Tool hallucination: the LLM invents `find_bear()`, a function that was never defined.

Tool Execution Errors

- **Wrong/buggy tool output**: a software bug in the tool itself (fix the implementation).
- **No response returned**: especially harmful for action-performing tools (e.g. raising a thermostat), since the model may falsely confirm success. *Guidance*: always return a meaningful structured output, even if empty (e.g. an empty JSON meaning "no results found" is more informative than `None`).

Synthesis Errors

Even with the correct tool output, the model may fail to incorporate it into the final response (e.g. saying "I didn't find any bear" despite a successful match). Causes include: the model's inability to ground on prior context (consider upgrading the model), tool output that is too verbose/noisy for the model to parse the relevant part (trim and clean tool outputs), or output that is not structured meaningfully (use well-named structured objects/attributes rather than raw unstructured text).

Common Themes

Failure modes generally trace back to: (1) the model's reasoning/grounding ability, (2) the relevance of what is placed in context, (3) tool/API modeling (via SFT, prompting, or API descriptions), or (4) bugs in the tool itself. Being systematic about categorizing and grouping errors is essential when debugging agents.

Benchmarks

Broad benchmark categories used to compare LLMs:

Knowledge Benchmarks

Test whether a model can recall facts across many domains – mostly reflecting pretraining quality. **MMLU** (Massive Multitask Language Understanding; Hendrycks et al. 2020, *"Measuring Massive Multitask Language Understanding"*) spans 57 diverse tasks (elementary mathematics, US history, computer science, law, etc.), framed as 4-choice multiple-choice questions so that answers can be extracted and scored in a hard-coded, unambiguous way (avoiding another layer of LLM-as-a-Judge error).

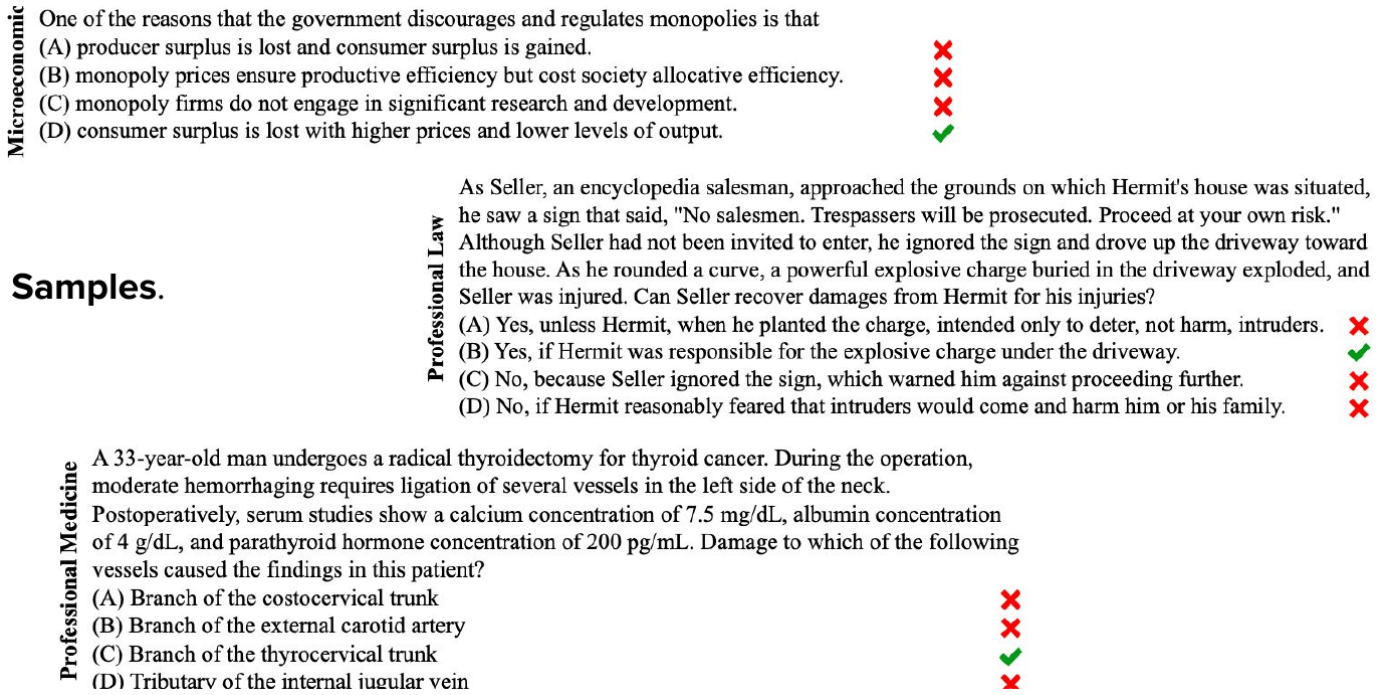


Figure 5: MMLU spans wildly different domains, from microeconomics to professional law and medicine.

Reasoning Benchmarks

- **AIME**: a high-school Olympiad-qualifying math exam, with numeric (three-digit) answers – well-constrained and thus LLM-friendly to grade.
- **PIQA** (Physical Interaction, Question Answering; Bisk et al. 2019, *"PIQA: Reasoning About Physical Commonsense in Natural Language"*): common-sense reasoning grounded in everyday physical situations, framed as 2-choice questions (e.g. vacuuming with a solid seal vs. a hairnet to retrieve something lost on a carpet), with about 20,000 examples.

Coding Benchmarks

Coding matters both for AI-assisted coding use cases and because agentic tools are often themselves written/interpreted as code. **SWE-bench** (Jimenez et al. 2023, *"SWE-bench: Can Language Models Resolve Real-World GitHub Issues?"*) sources 2,294 real software engineering problems from GitHub issues across 12 popular Python repositories, each with a base commit and an already-merged pull request that includes tests; a model is given the codebase and issue, produces a patch, and is scored by whether the previously-failing tests now pass.

Safety Benchmarks

Safety benchmarks are less standardized across providers since safety policies differ by company, making cross-model comparison less meaningful without inspecting benchmark content. **HarmBench** (Mazeika et al. 2024, *"HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal"*) covers 510 unique harmful behaviors (400 text-based, 110 multimodal) across four categories: standard (vanilla harmful behavior), copyright (generating copyrighted content), contextual (text-modality-specific), and multimodal – judged against whether they violate laws and widely-held norms. Since harmful outputs can be open-ended (not matchable via fixed answers), HarmBench relies on a trained classifier to compute an **Attack Success Rate (ASR)** – notably, an attempt counts as "successful" (i.e. unsafe) even if the harmful output itself was low quality.

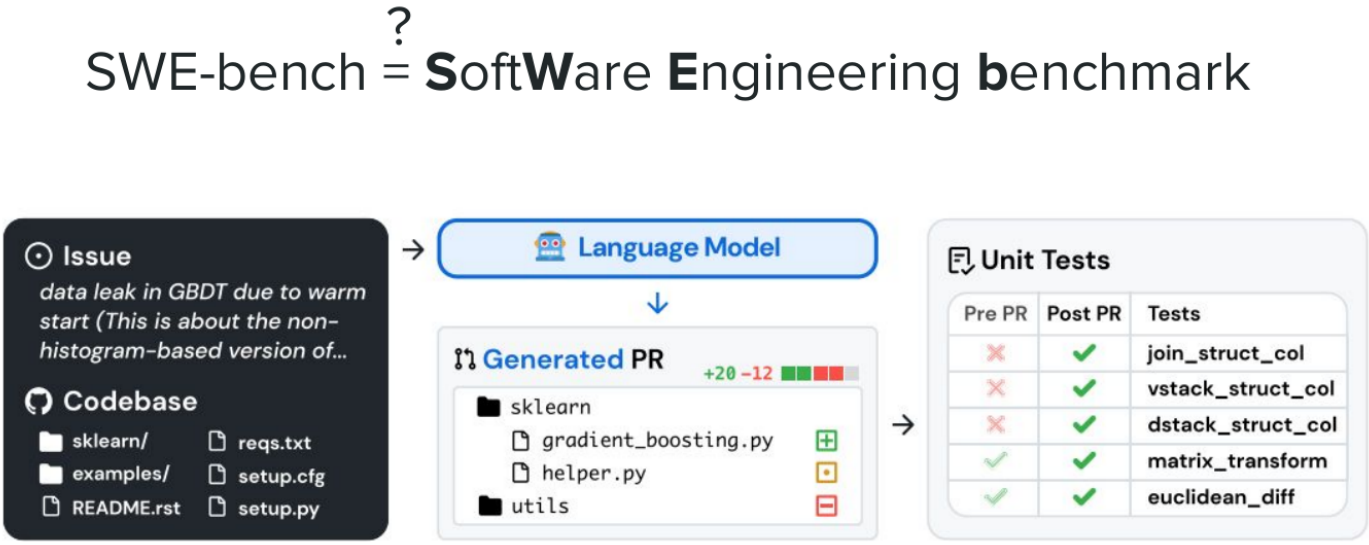


Figure 6: SWE-bench: given a real GitHub issue and codebase, the model generates a PR, scored by whether the associated unit tests now pass.

Agentic Benchmarks

τ-bench ("tau," for Tool-Agent-User [Interaction Benchmark]; Yao et al. 2024, *"τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains"*) evaluates agents across two domains, each with its own database schema, APIs, and policies: **airline** (500 users, 300 flights, 2000 reservations, ~10 tools, 50 tasks) and **retail** (500 users, 50 products, 1000 orders, ~10 tools, 115 tasks), with tasks driven by an LLM-simulated user (since the conversation cannot be scripted – each turn depends on the agent's prior actions). Success is measured via a reward based on whether the resulting database state (and/or action taken) matches the task's goal, combined with **pass^k** ("pass hat k," introduced in the same paper), the probability that *all* k attempts succeed (as opposed to **pass@k**, the probability that *at least one* succeeds) – relevant when reliability and consistency, not just occasional success, are required.

Grounding Benchmarks in Reality

Model release reports (e.g. Pichai et al., *"A New Era of Intelligence with Gemini 3,"* November 18, 2025) typically cite results on benchmarks of exactly these types, often using multilingual or agentic variants (e.g. *global* PIQA, a SWE-bench flavor, and τ^2-bench). Benchmarks characterize a model's *profile* rather than declaring one model globally "best" – different models have different strengths (e.g. some are strong at coding, others are fast and cheap), and personal/task-specific experience should guide model choice. Plotting performance against another axis of interest (e.g. price, safety, context length) traces out a **Pareto frontier** of the best available trade-offs.

Data Contamination and Goodhart's Law

Benchmark validity depends on the model not having seen the benchmark itself during training. Precautions include: using a content identifier such as a **hash** to detect overlap (see Google's BIG-bench documentation, 2025), **blocklisting** benchmark-hosting websites for tool-use benchmarks (as done for the Gemini 3 Pro evaluation methodology, Google 2025), or evaluating on genuinely new problems (e.g. the latest year's math competition, as emphasized in Pichai et al.'s Gemini 3 announcement). Finally, **Goodhart's law** – "when a measure becomes a target, it ceases to be a good measure" (traced to Strathern 1997, *"'Improving Ratings': Audit in the British University System,"* p. 308, rather than

Definition. Pareto curve = set of solutions that "optimizes" a trade-off.

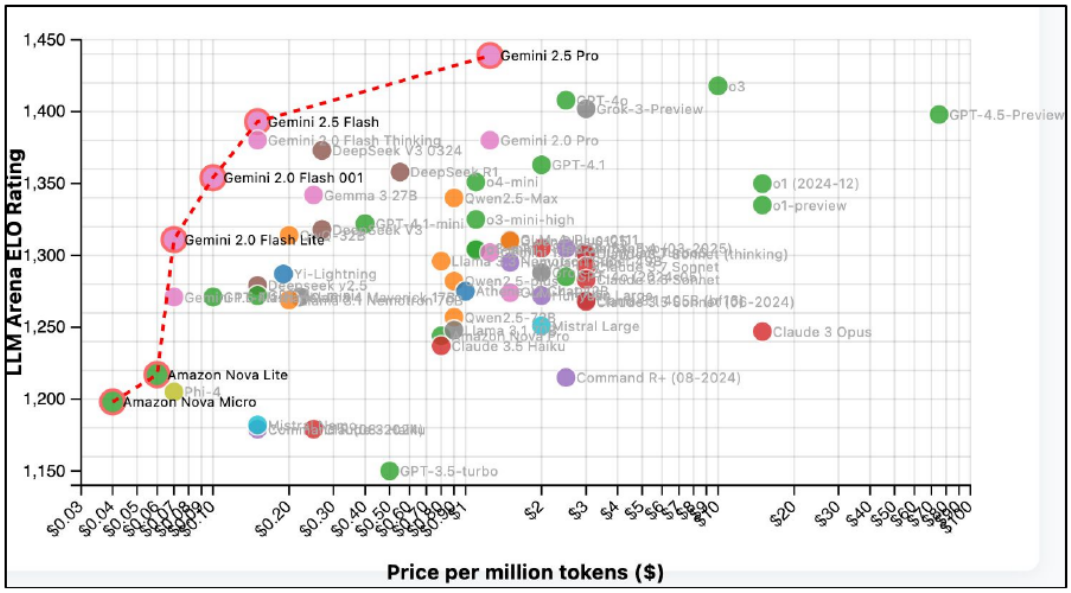


Figure 7: A Pareto frontier of LMArena rating vs. price per million tokens: the dashed line traces the best available trade-offs.

economist Charles Goodhart's original, differently-worded statement) – is a useful reminder that no single benchmark result should be over-optimized against; the ultimate test is whether a model performs well on your own use case, complemented by more organic signals like Chatbot Arena, and simply trying a few models out yourself.

Summary

We moved from the ideal-but-impractical baseline of per-output human rating, to reference-based rule metrics (METEOR, BLEU, ROUGE), to **LLM-as-a-Judge** as a scalable, interpretable proxy for human preference – while remaining mindful of its biases (position, verbosity, self-enhancement) and the need to keep it calibrated against human ground truth. We examined factuality evaluation via fact decomposition and weighted aggregation, catalogued failure modes in agentic tool-calling pipelines, and surveyed the main benchmark families (knowledge, reasoning, coding, safety, agentic) used to characterize model performance, closing with the caution of Goodhart's law: benchmarks are a guide, not the goal itself.

Thank you!

CME 295 – Transformers and Large Language Models

Lecture 9: Course Recap, Trending Topics, and Closing Thoughts

Afshine Amidi, Shervine Amidi

Overview

This is the final lecture of the course, split into three parts: (1) a recap tying together the entire quarter, (2) a look at topics trending in 2025 and beyond, and (3) closing thoughts and pointers for continued learning. Note: the final exam covers lectures 5–8 (the midterm covered lectures 1–4).

1 Part 1: Course Recap

Lecture 1 – From Text to Self-Attention

Before any modeling can happen, text must be processed. **Tokenization** splits input into atomic units; the most common scheme is **subword-level tokenization**, which lets shared word roots be reused and represented efficiently.

Early representation methods (e.g. **word2vec**) learned embeddings from a proxy task (predicting a center word from context or vice versa), but these embeddings were not **context-aware**: the same word had the same representation regardless of sentence. **RNNs** addressed sequential processing by keeping a running internal state, but suffered from **long-range dependency** issues – information from far in the past faded as sequences grew longer.

This motivated **self-attention**: tokens attend directly to one another regardless of their distance in the sequence, via **query**, **key**, and **value** vectors. A query's similarity to each key is computed via a scaled dot product and passed through a softmax, producing a weighted average over the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

This matrix formulation is efficient on modern hardware. The lecture concluded with the **transformer** architecture (encoder + decoder), originally motivated by machine translation.

Lecture 2 – Architectural Refinements and Transformer Variants

Since the original 2017 transformer paper, several refinements have become standard:

- **Rotary Position Embeddings (RoPE)**: rather than encoding *absolute* position (as in the original paper), RoPE rotates queries and keys as a function of their *relative* distance, directly inside the self-attention computation – which is where relative position actually matters.
- **Grouped Query Attention**: projection matrices for keys/values can be shared across groups of heads rather than learning one set per head.

- **Pre-norm vs. post-norm**: the original transformer applied normalization *after* each sublayer (post-norm); most modern models apply it *before* (pre-norm).

The transformer architecture also spawned several derived model families: **encoder-only** (e.g. **BERT**, which produces meaningful embeddings via the CLS token, well-suited to classification but not text generation), **decoder-only** (e.g. **GPT**, autoregressive text generation), and **encoder-decoder** (e.g. **T5**).

Lecture 3 – Large Language Models

Modern LLMs are decoder-only, text-to-text transformers, scaled up considerably. To avoid activating the entire network on every forward pass, **mixture-of-experts (MoE)** architectures replace a single feedforward network with several expert feedforward networks and a **gating mechanism** that sparsely routes each token to a subset of experts – enabling smart hardware placement and parallelization.

At inference, the model outputs a next-token probability distribution; **greedy decoding** always takes the most likely token, while **sampling** introduces randomness for more varied outputs. The **temperature** hyperparameter controls this: low temperature $\rightarrow$ spiky, near-deterministic distribution; high temperature $\rightarrow$ flatter, more random/creative distribution.

Lecture 4 – Training LLMs at Scale

Empirically, larger models trained on more compute and more data perform better, which raised the question: for a fixed compute budget, what is the optimal balance of model size vs. dataset size? Research in the early 2020s found that most models at the time were **undertrained** relative to their size, leading to a widely used rule of thumb:

$$\text{Training tokens} \approx 20 \times \text{Number of parameters}$$

(e.g. a 100B-parameter model should be trained on at least $\sim$2T tokens).

Flash attention improves efficiency by exploiting the GPU memory hierarchy – a large but slow memory (HBM) and a small but fast memory (SRAM) – minimizing reads/writes to HBM by processing computation in small blocks in SRAM. It is an *exact* method (no approximation), and even embraces *recomputing* intermediate results rather than storing them, trading extra compute for fewer slow memory accesses. **Data parallelism** (splitting data across GPUs) and **model parallelism** (splitting a single forward pass across GPUs) further enable training at scale.

Training itself proceeds in three stages:

(1) **Pretraining**: next-token prediction on huge corpora (trillions of tokens), yielding a model that can autocomplete text but is not yet "helpful."

(2) **Supervised fine-tuning (SFT)**: trains the model on the specific input/output behavior we want.

(3) **Preference tuning**: injects *negative* signal (what *not* to do) using pairwise preference data ("prefer this output over that one"), aligning the model along dimensions like usefulness, safety, and tone.

Lecture 5 – Preference Tuning via Reinforcement Learning

Preference tuning was framed through a reinforcement learning (RL) lens: the LLM is a **policy**; the input so far is the **state**; predicting the next token is the **action**; and a **reward** (derived from human preference) evaluates the resulting completion.

Since rewards are only available for limited data, we train a **reward model**. The **Bradley-Terry** formulation models the probability that output i is preferred over output j as a function of their individual scores r_i, r_j; the reward model is trained pairwise (given a preferred/dispreferred pair) but used at inference time on a single output at a time.

The reward model is then used to steer the LLM: given a prompt, the LLM produces a rollout, the reward model scores it, and the LLM's weights are updated to maximize reward – via **PPO** (Proximal Policy Optimization), which uses a learned **value function** to produce a relative **advantage** estimate (how good an output is compared to a baseline). Because the reward model is imperfect, **reward hacking** is a risk; the optimization objective therefore constrains the updated policy to stay close to both the base (SFT) model and the previous RL iteration, acting as a regularizer.

Lecture 6 – Reasoning Models

Reasoning models are trained (largely via the RL techniques from Lecture 5) to output a **reasoning chain** before the final answer, building on the **chain-of-thought** prompting idea from Lecture 3. Performance (e.g. on the AIME math benchmark) improves measurably as this training progresses.

GRPO (Group Relative Policy Optimization) has largely superseded PPO for reasoning training. Key differences:

- GRPO requires **no value model**: instead of a learned baseline, it samples several completions per prompt and computes relative advantages by comparing their rewards directly.
- GRPO is typically applied to tasks with **verifiable rewards** (e.g. math, where the correct answer is known), removing the need to train a reward model at all. Only the policy model and a reference model (for comparison) need to be maintained.

A known issue with vanilla GRPO is a length-normalization term in its loss that under-penalizes longer incorrect answers, incentivizing runaway output length. Extensions such as **GRPO done right (Dr. GRPO)** and **DAPO** address this by adjusting or removing this normalization term.

Lecture 7 – RAG, Tool Calling, and Agents

RAG (Retrieval-Augmented Generation) lets an LLM fetch relevant external documents to overcome its knowledge-cutoff limitation, via two retrieval stages: **candidate retrieval** (bi-encoder semantic search, e.g. cosine similarity over precomputed embeddings) and **re-ranking** (a more precise cross-encoder scoring of query-document pairs), after which the top-k documents augment the prompt.

Tool calling lets an LLM invoke external APIs: the model first decides which function and arguments to use, the function is executed, and the result is fed back to the model to produce a final natural-language answer. **Agentic workflows** combine RAG and tool calling, allowing the model to make multiple calls across a reasoning loop to accomplish a task.

Lecture 8 – LLM Evaluation

Rule-based reference metrics (**BLEU**, **ROUGE**, **METEOR**) predate LLM-based evaluation but fail to reward stylistically different, equally valid phrasings. This motivates **LLM-as-a-Judge**: given a prompt, a model response, and grading criteria, a judge LLM outputs a **rationale** and a **score** (commonly binary: pass/fail), with the rationale output *before* the score (mirroring reasoning-model chain-of-thought, and empirically improving judge quality).

Key biases to watch for: **position bias** (favoring whichever response is presented first), **verbosity bias** (favoring longer responses regardless of quality), and **self-enhancement bias** (a model favoring

its own outputs). Standard **benchmarks** span knowledge, reasoning, coding (important given how many applications are coding-related), and safety, among other dimensions – this is not an exhaustive list.

2 Part 2: Trending Topics in 2025 and Beyond

Beyond Text: The Vision Transformer (ViT)

The transformer's core strength is self-attention: letting any element attend to any other. Since tokens are simply vectors, a natural question is whether transformers work on non-text vectors, such as image patches.

The **Vision Transformer (ViT)** (Dosovitskiy et al. 2020, *"An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale"*), introduced in 2020, answers yes: an image is divided into fixed-size patches (e.g. 3×3), each patch's raw pixel values (e.g. RGB) are linearly projected into a vector, a learned CLS embedding and position embeddings are added, and the sequence is passed through a transformer **encoder** (mirroring BERT). The encoded CLS token – having attended to every other patch – is projected onto the target classes for prediction.

This was notable because ViT largely discards the strong **inductive bias** of convolutional neural networks (which process images via local sliding windows, mimicking human visual inspection). Given enough training data, ViT was shown to outperform CNN-based approaches despite this much weaker inductive bias.

Vision-Language Models (VLMs)

To let an LLM answer questions about an image, both an image representation and text tokens must be fed to the model. Two common strategies:

- **Early fusion (most common)**: image tokens (from an image encoder) are concatenated with text tokens and fed together into a decoder-only, autoregressive LLM (e.g. **LLaVA**, Liu et al. 2023, *"Visual Instruction Tuning"* – a popular open-weight VLM).
- **Cross-attention fusion (less common)**: image tokens are not concatenated into the input sequence but instead interact with text tokens inside dedicated **cross-attention** layers (used, e.g., in **Llama 3**; LLaMA Team 2024, *"The Llama 3 Herd of Models"*).

Beyond vision understanding, transformer components (self-attention) also appear in image *generation* architectures, such as **diffusion transformers** and **multimodal diffusion transformers**, as well as in speech and recommendation domains.

Diffusion-Based LLMs

Standard LLMs are **autoregressive (ARM)**: each token is predicted conditioned on all previous tokens, one at a time. This makes *inference*-time generation inherently sequential (non-parallelizable), even though *training* is parallelizable (via the causal mask over full target sequences).

Diffusion models (Ho et al. 2020, *"Denoising Diffusion Probabilistic Models"*), highly successful for images, generate by starting from noise and learning to reverse a forward "noising" process (gradually corrupting a clean image) – i.e. learning to predict and remove the noise needed to reconstruct the target image. Noise is a natural starting point because it is easy to sample, well modeled by Gaussian distributions, and introduces useful randomness. (Analogy, per Michelangelo: a sculpture is already

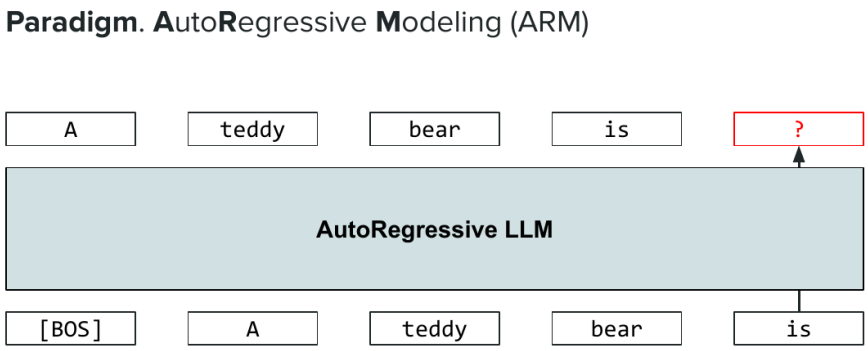


Figure 1: The autoregressive bottleneck: each new token requires a full forward pass conditioned on everything generated so far.

complete within the marble block, and the sculptor's job is simply to chisel away the superfluous material.)

Figure 2: The sculpture analogy: the target (e.g. a teddy bear) is already latent within the raw material; generation reveals it by removing the superfluous parts.

Adapting this to text is nontrivial because **text is discrete** while images are continuous – there is no direct notion of "adding noise" to tokens.

The current approach treats the **mask token** as the text analogue of noise: the forward process progressively masks more of the input until the sequence is fully masked; the model then learns to unmask (reconstruct) the original sequence, conditioned on a prompt. This gives rise to **masked diffusion models (MDM)**, also called **diffusion-based LLMs (DLLM)** (see e.g. Lou et al. 2023, *"Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution"*; Sahoo et al. 2024, *"Simple and Effective Masked Diffusion Language Models"*; Nie et al. 2025, *"Large Language Diffusion Models"* – the LLaDA paper).

Intuition: rather than writing a speech linearly, one might first draft a rough outline and then progressively refine each part – diffusion text generation works in a similar coarse-to-fine manner, rather than strictly left-to-right.

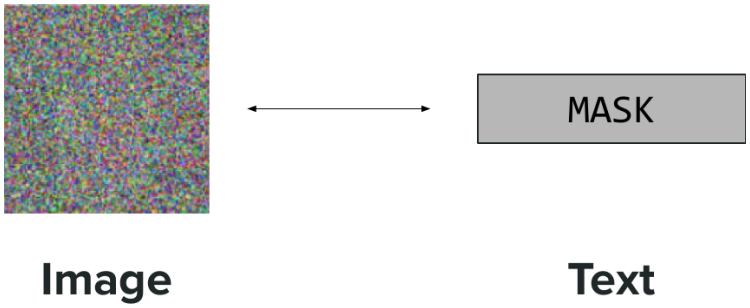


Figure 3: The mask token is text's analogue of noise: a fully-masked sequence plays the same role as a fully-noised image.

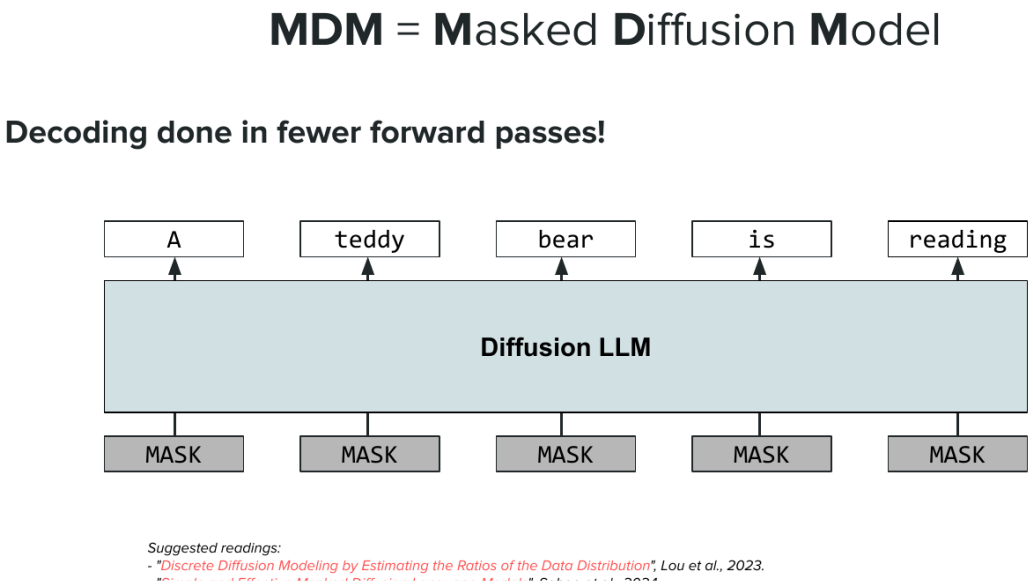


Figure 4: A masked diffusion LLM unmasks every token in parallel, in far fewer forward passes than an autoregressive model.

Advantages:

- **Speed**: decoding requires only as many forward passes as diffusion steps (a fixed, tunable number), not one pass per output token – reported as roughly 10× more output tokens per second compared to ARM, especially valuable for long outputs (e.g. coding).
- **Fill-in-the-middle tasks**: because the model considers the sequence as a whole rather than strictly left-to-right, it is naturally better suited to tasks requiring information from multiple directions.

Current limitations: performance and adapting ARM-based techniques (e.g. reasoning chains) to the diffusion paradigm remain active challenges. Recent industry activity includes an experimental diffusion-based LLM Google showcased at I/O (May 20, 2025), a diffusion LLM covered by TechCrunch (November 6, 2025), and startups such as Inception and ByteDance (announcement July 31, 2025) pursuing this direction.

Cross-Pollination Between Modalities

Ideas increasingly transfer between the vision and text/LLM worlds:

- **Architecture**: "diffusion" training, born in vision, has been adapted for text output (e.g. Nie et

al. 2025's LLaDA); conversely, image diffusion models have increasingly replaced convolutional backbones with **transformer** backbones – **Diffusion Transformers (DiT)** (Peebles & Xie 2022, *"Scalable Diffusion Models with Transformers"*) – improving results.

- **Input representation**: **DeepSeek-OCR** (Wei et al. 2025, *"DeepSeek-OCR: Contexts Optical Compression"*; despite its name) demonstrated that text tokens can be reconstructed from a small number of *vision* tokens representing image patches – suggesting that patch-based visual representations can be a highly information-dense alternative to standard text tokenizers.
- **Positional encoding**: **RoPE**, originally for 1D text sequences, has been extended to **2D**/multimodal variants (e.g. the Multimodal Scalable RoPE used in Wu et al. 2025's *"Qwen-Image Technical Report"*), allowing relative-position computations to remain meaningful when both text and image tokens are present.

The Transformer Architecture Is Still Evolving

Many "settled" design choices are still actively being challenged:

- **Optimizers**: Adam has long been standard, but newer optimizers (e.g. **Muon**, and its refinement **MuonClip**, from the Kimi K2 paper) are emerging as potential successors.
- **Normalization**: beyond the pre-norm/post-norm placement question, the *type* of normalization has shifted from the original **LayerNorm** toward alternatives like **RMSNorm** (fewer parameters).
- **Attention configuration**: grouped-query attention and related variants are not applied uniformly – different papers make different design choices, sometimes even varying by layer.
- **Activation functions**: from ReLU toward ReLU-like variants (e.g. GELU) and beyond, with new activation functions still being proposed.
- **Other hyperparameters**: whether to use MoE, number of layers, number of heads, FFN width, etc., remain open design choices without a single converged standard.

Data: A Critical and Shifting Landscape

Early LLMs benefited from scraping largely human-generated internet text. Today, a large share of newly generated web content is itself LLM-generated, raising the risk of **model collapse** (Shumailov et al. 2023, *"The Curse of Recursion: Training on Generated Data Makes Models Forget"*): training on model-generated text (which tends to be less diverse) shifts the training data distribution and degrades learning. This has motivated growing investment in **data curation** and a new intermediate stage, **mid-training** (training on a large but higher-quality corpus), inserted between pretraining and fine-tuning.

Hardware

GPUs excel at matrix-vector/matrix-matrix operations, but transformer blocks have special needs that don't map perfectly onto this: attention requires frequent key/value reads and writes, and this memory movement (not raw compute) dominates cost on GPUs – exactly the bottleneck that motivated techniques like flash attention (recomputing rather than storing intermediates to minimize slow-memory traffic). Emerging research (Leroux et al. 2025, *"Analog In-Memory Computing Attention Mechanism for Fast and Energy-Efficient Large Language Models"*) proposes "natively" supporting attention operations in hardware – storing the KV cache in dedicated memory cells and using analog signals (leveraging physical properties like Kirchhoff's circuit laws) to perform computation as a side effect of the hardware

itself – reporting up to ∼**100**× **latency savings** and ∼**70,000**× **energy savings** relative to an Nvidia H100.

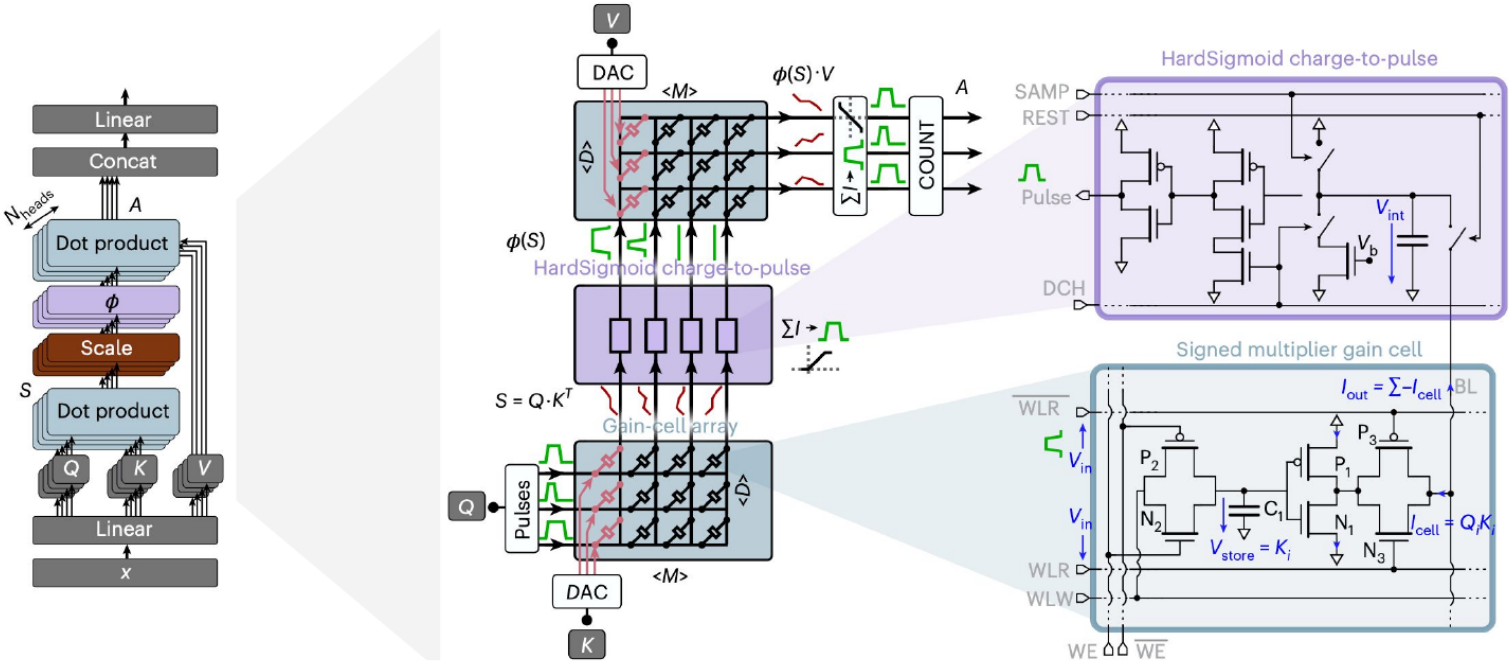


Figure 5: Analog in-memory attention: a gain-cell array stores K/V directly in memory, computing attention as a physical side effect of the circuit itself.

Looking Ahead: Use Cases and Open Questions

Daily life has already been dramatically changed by LLMs, with key applications today spanning coding (both "true" coding and text-to-query interfaces), general-purpose conversational assistance (common facts, web search), creativity, and learning.

More use cases are expected going forward, along a rough timeline:

- **Tomorrow**: democratization of existing agentic capability into mainstream products – e.g. Google Workspace Studio (Google Workspace, *"Introducing Google Workspace Studio: Automate Everyday Work with AI Agents,"* December 3, 2025), which lets everyday users automate work with AI agents.
- **Near term**: agentic browsing (e.g. OpenAI, *"Introducing ChatGPT Atlas,"* October 21, 2025) – and, looking further out, OS-level LLM integration.
- **Long term**: "truly" autonomous agents with vast responsibilities.
- **The hard test case**: whether AI-driven customer service can become genuinely useful (finally) – a good barometer for how far the field has come, since human qualities like empathy and groundedness remain difficult to replicate.

Agentic and AI-browsing capabilities point toward democratizing complex workflows for everyday users via natural language rather than code – though challenges remain, notably **security** (e.g. prompt injection and data exfiltration risks), analogous to how HTTPS eventually became a trust signal for the web.

Ongoing open challenges ("top of mind"):

- **Continuous learning**: today's models have fixed weights post-training (not continuously updated), with RAG/tools used as workarounds – can models learn continuously instead?
- **Hallucination**: arguably a natural consequence of training models to predict the next token rather than to map statements to verified facts.
- **Personalization, interpretability, and safety** remain open, active areas.

3 Part 3: Closing Thoughts and Staying Current

To keep up with the field going forward, useful resources include:

- **Papers**: **arXiv** (Computer Science > Computation and Language), and general ML venues (**NeurIPS**, **ICML**, **ICLR**) alongside NLP-specific venues (**ACL**, **EMNLP**).
- **Code**: authors' **GitHub** repositories linked in their papers, and **HuggingFace's "trending papers"** page (browsing state-of-the-art datasets/methods) – a modern successor to "Papers with Code."
- **Miscellaneous**: **X/Twitter** (academics and industry leaders); YouTube – both theoretical (**Two Minute Papers**, **Yannic Kilcher**) and practical (Google Developers, HuggingFace, **Andrej Karpathy**); and company/academia technical papers and blogs (e.g. Amazon Science, Anthropic, Apple ML, Google DeepMind, Meta AI, Microsoft AI, OpenAI, Stanford NLP).

The course's own study guide and **VIP cheatsheet** will continue to be updated periodically, with translations into multiple languages available.

The instructors closed by thanking students for their engagement throughout the quarter – both those attending in person and those following online – and expressed hope that the course provided a solid foundation for continuing to follow developments in transformers and large language models going forward.

Thank you!